www.ambarella.com

Ambarella CV2FS CV22FS CV2FSA SDK Safety Manual

Summary Description

The purpose of the document is to introduce and provide details related to the safety requirements for Ambarella's CV2FS / CV22FS / CV2FSA chipsets (also known as CV2 ASIL or CV2A).

The document provides a brief product overview and then proceeds to provide assumed safety requirements, technical safety concepts, safety mechanisms, integration, safety analysis, and the development cycle.

Copyright Notice

Copyright © 2025 Ambarella International LP.

The contents of this document are proprietary and confidential information of Ambarella International LP.

The material in this document is for information only. Ambarella assumes no responsibility for errors or omissions and reserves the right to change, without notice, product specifications, operating characteristics, packaging, ordering, etc. Ambarella assumes no liability for damage resulting from the use of information contained in this document. All brands, product names, and company names are trademarks of their respective owners.

For complete Ambarella contact information, please visit www.ambarella.com.

Table of Contents

Contents

Please update the table of contents.

Table of Figures

Please update the table of figures.

1 Purpose and Scope of Document

The purpose of the document is to introduce and provide all the details related to the safety requirements for Ambarella's CV2FS / CV22FS chipsets.

The document provides a brief product overview and then proceeds to assumed safety requirements, technical safety concepts, safety mechanisms, integration, safety analysis, and the development cycle.

2 Product Overview

The CV2FS / CV22FS / CV2FSA chip is a system on a chip (SoC) for automotive markets, including e-Mirror and ADAS applications, and is targeted to comply with ASIL-C according to ISO 26262.

CV2FS / CV22FS / CV2FSA chip contains a quad core Arm® Cortex®-A53 for application processing and embedded accelerators for image processing, video encoding, and computer vision/AI functions—especially neural network processing and optical flow. The accelerators are co-processors to the Arm cores. A dual core Cortex-R52 in lockstep is added for safety critical applications and safety monitoring. All processors and co-processors connect through a single controller to access DRAM.

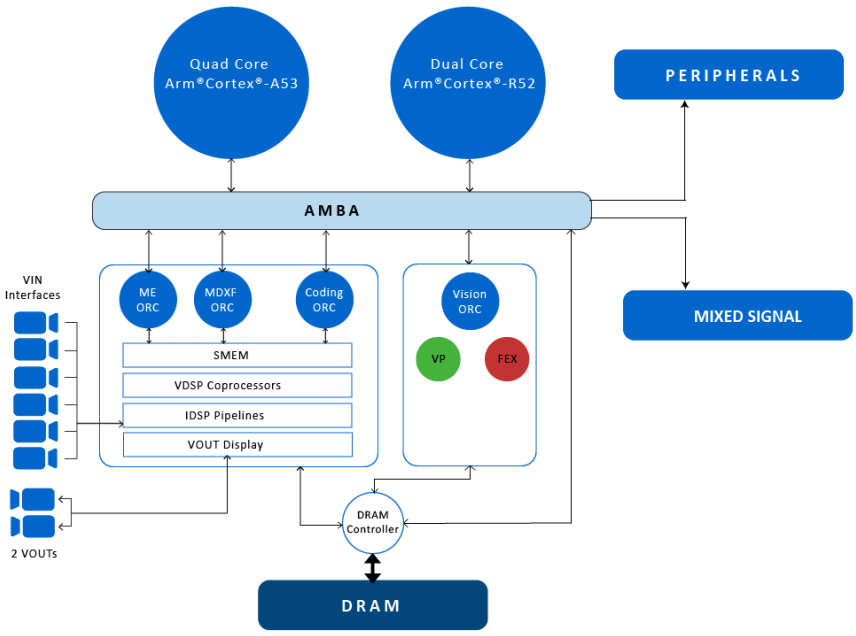


Figure 1 Conceptual View of the Architecture

The following use case descriptions provide overviews of the applications and products in which the SEooC is expected to be used. These use cases are not intended to be exhaustive or complete. Instead, they provide a background for the functional requirements given in the next chapter to enable the specification of SEooC top-level safety requirements.

For a full description of all intended use cases, please refer to *CV2A Product Requirement [D18596_0]*.

2.1.1 Use Case UC-01: Forward-Facing ADAS Camera

Typically, this use case provides a single sensor-based solution, but can be dual or triple sensor-based in certain high-end variants. In this use case, different sensor configurations must be supported. Additionally, if video streaming or recording functionality is required, CV2FS / CV22FS can be configured to output compressed H.264 video as well as object data.

2.1.2 Use Case UC-02: Electronic Mirror System

Electronic mirror systems are also known as camera monitor systems (CMS) as defined under ISO 16505. Basic configuration includes in-cabin rear-view mirror replacement, which provides a video viewing solution. Potential future CV functionality includes blind spot detection (BSD); CV2FS / CV22FS is targeted to support two- and three-channel CMS architectures.

2.1.3 Use Case UC-03: Driver State and In-Cabin Monitoring Systems

This use case can include a single camera or dual camera configuration, typically based on global shutter for driver state monitoring (DSM); RGB-IR rolling shutter HDR sensors are considered to enable in-cabin monitoring solution. The application monitors a driver's attention by detecting the head position, gaze of eyes, as well as the frequency of eyelid movements to determine drowsiness. To ensure that variable lighting conditions including drivers wearing sunglasses do not affect the functionality of the application, the system deploys pulsed LED light sources synchronized with the capturing sequence of the image sensor. This use case also considers the addition of RGB-IR sensors for passengers and the monitoring of "lost objects." In this case, near-infrared LEDs are used for consistent lighting.

2.2 Operating Modes and Modes that Support Safety

Table 1 CV2FS/CV22FS/CV2FSA Operating Modes

Power-down	**CV2FS/CV22FS/CV2FSA is completely unpowered and does not perform any function.**	**Assumed safe state (see TLSS-01 in Section 3.2)**
Reset	CV2FS/CV22FS/CV2FSA is in hardware (power-on) or cold and warm reset and does not perform any function.	Assumed safe state (see TLSS-01 in Section 3.2)
Normal mode (Startup phase)	CV2FS/CV22FS/CV2FSA is initialized and performs some self-tests (under the control of the internal firmware and application software). It is not yet ready to perform safety-related tasks.	It must be ensured by the application software and on system level, that the system safe state is maintained.
Normal mode (Runtime phase)	**CV2FS/CV22FS/CV2FSA can be used to perform safety related tasks.**	**Functional safety is maintained by CV2FS/CV22FS/CV2FSA.**
Error state	CV2FS/CV22FS/CV2FSA has detected an internal error and enters a SEooC top-level safe state (see Section 3.2). It can no longer reliably perform safety-related tasks.	The system must recognize the SEooC safe state and initiate a system safe state.
Normal mode (Shutdown phase)	CV2FS/CV22FS/CV2FSA is de-initialized and performs some self-tests (under the control of the internal firmware and application software). It is no longer available to perform safety-related tasks.	It must be ensured by the application software and on system level, that the system safe state is maintained.
Test mode	This operating mode is intended for manufacturing testing, and it must not be entered by the users.	Prohibited state (see CON-03 in Section 3.4)
Debug mode	This operating mode is intended for software development and debugging, and it must not be entered during normal operation.	Prohibited state (see CON-03 in Section 3.4)

Only in normal mode (runtime phase) the CV2FS/CV22FS/CV2FSA SEooC is available to perform safety-related tasks and the SEooC is responsible for maintaining a safe state by itself. In all other operating modes and phases, the system and application software are responsible for maintaining the system safe state.

3 Assumed Safety Requirements

The SoC is generally built to ASIL-C qualification based on the ISO-26262 standard. The Cortex-A53 is considered QM quality. Some peripherals are built to QM quality if they are included to aid development and system debugging but do not operate during safe operation (e.g. JTAG).

The assumed top-level safety requirements (TLSR) are described in Section 3.1. These are the highest-level safety-related requirements for the SEooC. When integrating and using the SEooC in a system or application, it is the responsibility of the user to confirm that these assumed TLSR match the actual system and application requirements.

The assumed top-level safe states (TLSS) are described in Section 3.2. In case of a potential TLSR violation, the SEooC enters one or several of the assumed TLSS. They represent a local safe state or error indication on the SEooC level. The system must recognize such an SEooC safe state and initiate a system-safe state.

The non-functional requirements (NFR) are described in Section 3.3. It described the environmental conditions, lifetime, quality requirements as well as the target of quantitative analysis of random hardware faults.

The constraints and assumptions (CON) described in Section 3.4 complement the TLSR. Assumptions related to the SEooC itself guide or constrain the way, how the SEooC is used in a safety related application. Other assumptions are directed towards system elements surrounding the SEooC, and the safety related behavior, functionalities, and characteristics required from these in order to achieve functional safety for the SEooC. As for the TLSR, the SEooC user must ensure that all constraints and assumptions are fulfilled during integration and usage.

3.1 Assumed TLSR

This section lists the top-level safety requirements.

[TLSR-1] The SEooC shall safely acquire image data from an external sensor and prepare captured data for processing by the computer vision system. [ASIL C]

The image data signal processing (IDSP) function shall be implemented for capturing any auxiliary/safety information from supported video sensor which system software uses to determine the safety state of the sensor. It prepares image data for processing by the computer vision system including correcting sensor artifacts (e.g. noise and bad pixels), adjusting the color, collect statistics for auto-calibration, and re-sampling. An error in IDSP function may lead to a safety goal violation because video processor or computer vision co-processors receives incorrect image data.

[TLSR-2] The SEooC shall safely encode image sequences captured from each external sensor according to H.264 standard. [ASIL C]

The video data signal processing (VDSP) function shall be implemented to encode the video stream. Video decoding is not safety-related because the decoder is not used for all three use cases defined in product requirement. The SEooC does not require receiving general H.264 encoded bit-streams to decode on-chip before processing through safe component.

An error in VDSP function may lead to a safety goal violation in encode mode if another safety-related element receives incorrectly coded data.

[TLSR-3] The SEooC shall safely execute vector operations according to the instructions and algorithms given by software. [ASIL C]
The CVflow processor shall be implemented to execute vector operations specified via a Directed Acyclic Graph (DAG) given by software. An error in CVflow processor may lead to a safety goal violation because results sent to the system cease to represent critical functions accurate (e.g. wrong objection detection, recognition, lane marking, etc).

[TLSR-4] The SEooC shall perform safe general purpose data processing, including the correct execution of the defined instruction set and correct handling of exceptions and interrupts. [ASIL C]
Cortex CPU cores and the corresponding interrupt handler will be used to run safety-related application and process safety related data, such as tasks that cannot be offloaded to CV flow processor or other computer vision co-processors. Thus, any fault in the CPU cores may lead to a safety goal violation.

[TLSR-5] For safety-related data, the SEooC shall perform on-chip data transfer and data storage between local memories without loss or corruption. [ASIL C]
Safety related data is transferred and stored via on-chip buses and storage. Any fault in the corresponding logic, such as cache, NIC, DRAM controller, DDRC and DMA engine, may cause data loss or corruption and hence lead to safety goal violation.

[TLSR-6] The SEooC shall safely configure and control the safety-related sub-elements without loss or corruption. [ASIL C]
The SEooC is operated based on the given configuration. If there is any fault in the storage of configurations or the corresponding interface logics, the sub-element may run in an unexpected behavior and thus lead to safety goal violation.

[TLSR-7] The SEooC shall support safe I/O communication with the safety-related external components. [ASIL C]
The SEooC may have safety-related data exchange with the following external components.

- NAND, NORSPI, or eMMC device,
- CAN, SPI, I2C interface,
- DRAM (DDR4 / LPDDR4 / LPDDR4x) interface,
- sensor(s) via MIPI interface,
- video output via MIPI-DSI
- GPIO interface.

Data integrity shall be ensured for these I/O communication interface. Otherwise, the safety-related data may be corrupted and thus lead to safety goal violation.

3.2 Top-Level Safe States

In addition to the state of normal and fault-free operation, the following top-level safe states (TLSS) and error indicators are defined on SEooC level.

[TLSS-1] The power-down state, hardware-reset and cold-reset state shall be a safe state for the SEooC. [ASIL C]

Motivation: All external SEooC interfaces in tri-state and inactive in these states, and hence the SEooC cannot interfere any other element in the system.

Note: The system can either power off the SEooC or keep the reset pin asserted to force the SEooC enter the safe state.

[TLSS-2] The SEooC shall have two dedicated hardware error output pins to indicate a safety error. [ASIL C]

Motivation: two copies of safety error pins ensures that the indicator itself doesn't inform the system that it's fault-free erroneously.

[TLSS-3] The SEooC shall have safety error interrupt to internal CPU to indicate an error. [ASIL C]

Motivation: TLSS-03 provides flexibility to the safety measures that may false-trigger the safety error indicators. Once the SEooC enters TLSS-03, SW shall check the reasoning of the safety error and take proper actions. Note that TLSS-03 is not yet a final safe state. Eventually SW shall set SEooC state to TLSS-02 or TLSS-04 to trigger a system-wide safe state if error reported by the interrupt is determined to compromise the safety.

[TLSS-4] The SEooC shall be able to deactivate by hardware an external watchdog interface, to indicate an error. [ASIL C]

Motivation: If the watchdog interface is deactivated by hardware means, the watchdog timer can no longer be refreshed anymore, independent of the software that is servicing the watchdog timer.

Any error indication at the hardware or software levels (for example, an error signal from a hardware module or failure of a software test) should be converted to one or several of the above defined SEooC top-level safe states.

The SEooC should consider this in the hardware design, and also, provide appropriate hooks for software developers. Also note that additional TLSS (e.g. CAN error message) can be defined on an application level.

3.3 Assumed Non-Functional Requirements

3.3.1 Environmental Conditions, Lifetime and Quality Requirements

The following lifetime requirements are assumed for the SEooC:

[NFR-1] The SEooC shall be designed for 10 years total lifetime and 8760 operating hours

Reasoning: Assumed passenger car usage: 10%

Note: The additional binning/screening can be run for extended lifetime guarantee.

[NFR-2] The SEooC shall be qualified according to AEC-Q100, Automotive Grade 2

Note: Ambient operating temperature range of AEC-Q100 Automotive Grade 2 is -40°C to +105°C, and 125°C junction temperature is considered in stress test environment.

[NFR-3] The SEooC shall be designed for the following temperature profile:

Junction Temperature	Operating Hours	Percentage
-40°C	100 h	1.14%
27°C	800 h	9.13%
85°C	1500 h	17.12%
105°C	4760 h	54.34%
115°C	1500 h	17.12%
125°C	100 h	1.14%
Effective Ø = **105.1°C**	**8760 h**	**100.0%**

Reasoning:The temperature profile is in line with AEC-Q100 Automotive Grade 2, assuming difference of 20°C between ambient and linear-average junction temperature. For base failure rate calculation, passenger compartment temperature profile is used. The junction/ambient temperature difference of 40°C , which is more pessimistic thus gives more margin in base failure rate calculation.

[NFR-4] The SEooC shall be designed for the following temperature cycles:

Category	Cycles per Year	Temperature Difference
Vehicle not used	30	10.0°C
4 days starts	1340	79.7°C
2 night starts	670	89.7°C

Reasoning: The temperature cycles are adopted from IEC TR 62380 (automotive mission profile, average world-wide climate conditions), with a linear-average junction temperature of 94.7°C.

3.3.2 Quantitative Analysis of Random Hardware Faults

A quantitative analysis of random hardware faults is performed according to ISO 26262-5, Chapter 9.4.2 (PMHF method). The FMEDA results are shown in section 7.1.3 .

3.4 Assumed Constraints

The following section lists the constraints and assumptions for the usage of CV2FS / CV22FS / CV2FSA SEooC in a safety-related application.

[CON-1] The SEooC shall be used within the operating conditions and limits as specified in the datasheet. [ASIL C]

[CON-2] The recommendations for system integration and configuration of the SEooC, as given in the datasheet, safety manual and other relevant application notes shall be followed. [ASIL C]

[CON-3] The SEooC shall be used in normal operating mode for safety-related operation. Test mode, debug mode, and other debug-related features must not be used or enabled at any time during a safety-related application. [ASIL C]

Reasoning: Activation of test mode or debug mode may have unpredictable effects on the application.

[CON-4] A diagnostic test time interval (DTTI) of 100ms shall be assumed for the SEooC. [ASIL C]

Reasoning: A minimum 200ms FTTI is assumed for the systems and use cases. It is further assumed that 50% of that FTTI can be used as SEooC DTTI, leaving 50% for safe state propagation and fault reaction.

[CON-5] The USB interface of the SEooC shall not be used in a safety-related application. [QM]

Reasoning: USB is not widely used in safety related to automotive systems.

[CON-6] The Ethernet interface of the SEooC shall not be used in a safety-related application. [QM]

Reasoning: Ethernet is not considered as safety related to automotive systems.

[CON-7] The I2S interface of the SEooC shall not be used in a safety-related application. [QM]

Reasoning: I2S interface is not considered as safety related to automotive systems.

[CON-8] The PWM interface of the SEooC shall not be used in a safety-related application. [QM]

Reasoning: PWM interface is not considered as safety related to automotive systems.

[CON-9] The SDIO interface of the SEooC shall not be used in a safety-related application. [QM]

Reasoning: SDIO interface is not considered as safety related to automotive systems.

[CON-10] The UART interface of the SEooC shall not be used in a safety-related application. [QM]

Reasoning: UART interface is not considered as safety related to automotive systems.

[CON-11] The SEooC shall be monitored by either an external MCU or an external watchdog timer with an independent clock source. [ASIL C]

Reasoning: The external MCU is used to turn TLSS-02 into a system safe state. Similarly, the external watchdog timer is used to turn TLSS-04 into a system safe state.

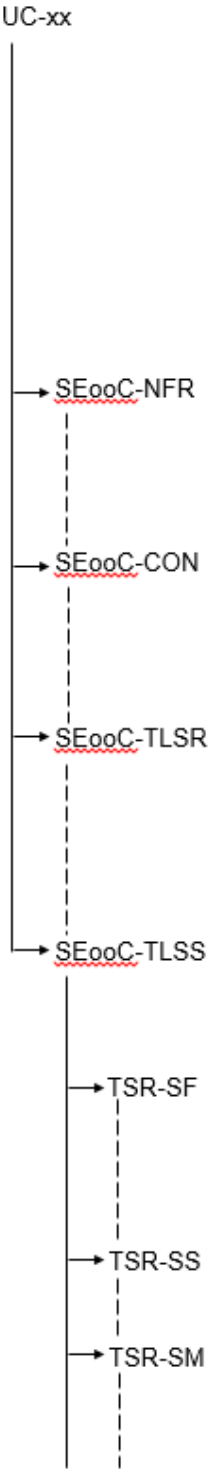
[CON-12] The temperature sensor in the SEooC shall not be used for safety-related temperature monitoring. [QM]

The temperature sensor is instantiated to detect whether the operating temperature of the SEooC is in the specified range. However, it shall not be used for safety-related temperature monitoring of the SEooC or the system. System developer must ensure proper thermal design on system level to stay within datasheet limits.

4 Technical Safety Concept Overview

4.1 Requirement Structure

This document defines the SEooC by requirements. The following hierarchical requirements structure is used throughout and assumed beyond the SEooC definition document:



Use Cases

A use case describes a possible application of a product or an item in which the product may be integrated. A clear understanding of the targeted use cases helps to derive functional and safety-related requirements for the product, including the SEooC top-level safety requirements.

Use cases are typically described in a product requirement document (PRD). However, it is recommended to include a summary (or reference) of the use cases in the SEooC definition document.

Non-Functional Requirements

Requirements that are safety-related and relevant for the SEooC, but that are not related to its functionality. This includes environmental conditions, lifetime, and quality requirements for the SEooC.

Constraints and Assumptions

Constraints and assumptions made during the SEooC definition phase, which become a requirement for the SEooC integrator / user, or limit the application of the SEooC.

Top-Level Safety Requirements

Highest-level safety-related requirements for the SEooC. TLSR are derived from the use cases and functional requirements and represent the main result from the SEooC definition phase. TLSR are assumed requirements and must be validated by the SEooC integrator / user.

Top-Level Safe State

Highest-level safe state for the SEooC. Any error indication on hardware or software-level should be converted to TLSS.

Safety Function

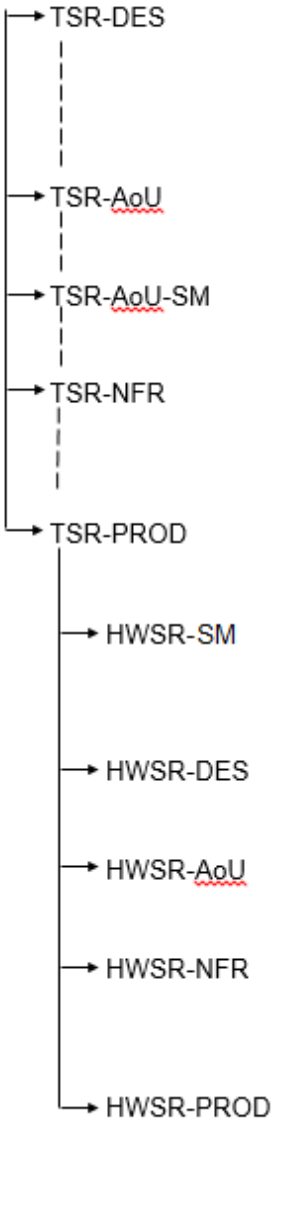
Intended function of the SEooC that is related to a safety requirement or safety goal. An error in the safety function may lead to a potential violation of a safety goal.

Safe State

Technical safety requirements for implementing a safe state.

Safety Mechanism

Technical solution implemented by either hardware or software to detect faults or control failures in order to achieve or maintain a safe state.



Design Measure
Technical solution to avoid or control systematic failures and to detect random hardware failures or control random hardware failures, or mitigate their harmful effects.

Assumption of Use
Requirement or constraint for the usage or integration.

Assumption of Use – Safety Mechanism
Safety mechanism which the customer or integrator must implement.

Non-Functional Requirement
Requirements that are safety-related and relevant for the TSR, but that are not related to its functionality; for example, FMEDA metrics.

Production Requirements
Requirements for production, including wafer, package, and testing.

Safety Mechanism
Refined safety mechanism in the TSR level for the hardware design block.

Design Measure
Refined design measure in the TSR-level for the hardware design block.

Assumption of Use
Refined assumption of use in the TSR- level for the hardware design block.

Non-Functional Requirement
Refined non-functional requirement in the TSR-level for the hardware design block.

Production Requirements
Refined production requirements in the TSR level for the hardware design block.

4.2 Safety Architecture

4.2.1 HW Safety Architecture

Figure 2 provides and overview of the technical safety concept for CV2A, showing the major safety mechanisms that are implemented in hardware, software or that are defined as assumptions-of-use (AoU) to detect and control internal errors. It also indicates the ASIL allocated to the different IP hardware blocks.

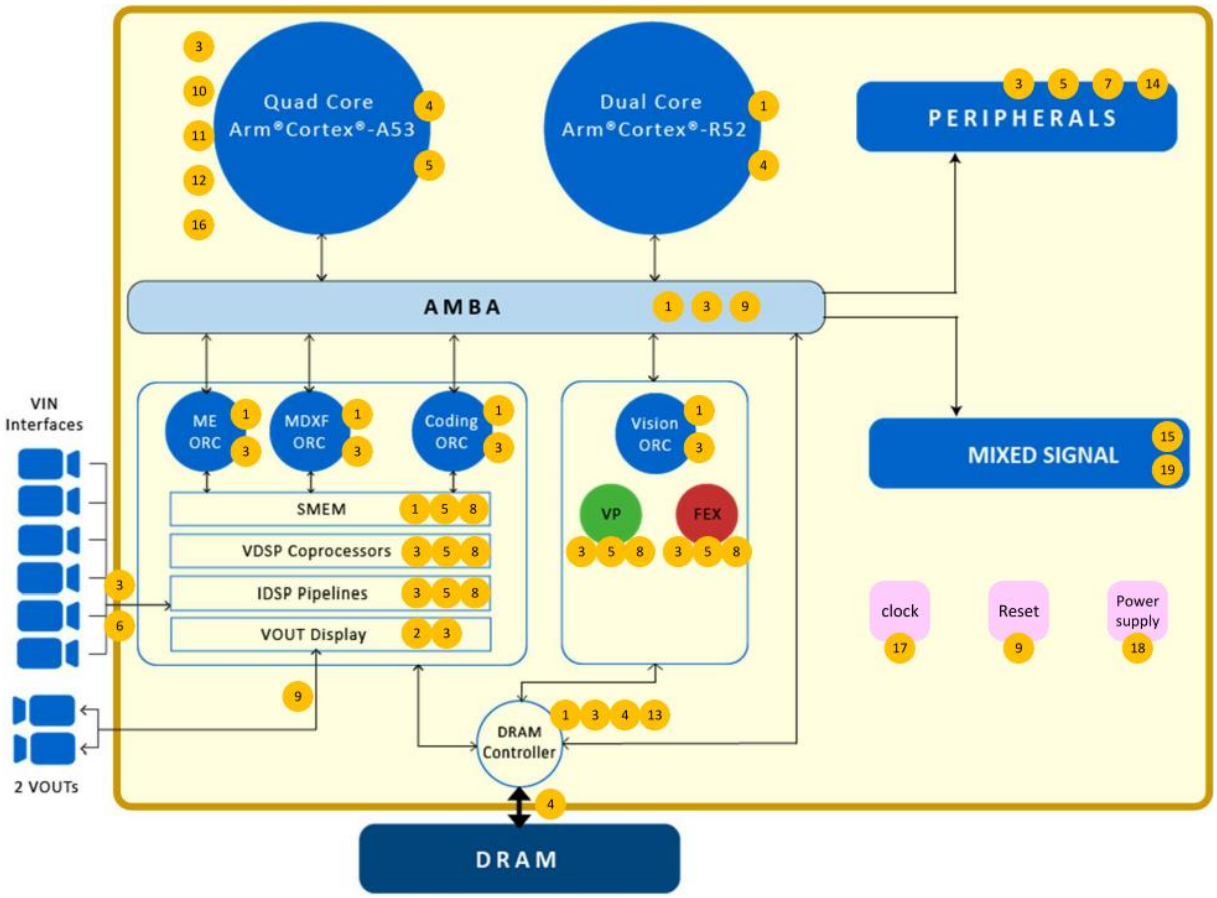


Figure 2 Microcontroller with safety mechanisms and ASIL allocation

The following safety mechanisms are shown in Figure 2:

①. Dual-core lockstep (DCLS)

②. Duplicated logic (Hardware redundancy)

③. Watchdog timer (or timeout mechanism)

④. ECC

⑤. Parity

⑥. CRC or signature

⑦. End-to-End (E2E) protection protocol

⑧. Software Test pattern

⑨. Software Register check

⑩. Software Data Integrity check

⑪. Software GIC checker (Interrupt monitor)

⑫. Software Test Library (STL)

⑬. Segmentation (Access protection)

⑭. Hardware loop-back

⑮. Built-in self-test pattern (BIST)

⑯. Performance monitor

⑰. Clock monitor

⑱. Power monitor (either internal or external)

⑲. Voltage detector

Note: Not all safety mechanisms are shown in Figure 2. Only major safety mechanisms, which provide the most significant diagnostic coverage contribution, are included, while smaller safety mechanisms are omitted for the sake of simplicity and to avoid overloading the picture.

CV2A contains a central error handling unit (CEHU), to consolidate all internal error signals, error interrupts and other error indicators from the hardware modules in one central location and to map these to one or several configurable error reactions. An error reaction can be any of the defined top-level safe states (TLSS). In addition, actions for internal error handling are also available as possible error reactions. These include triggering an interrupt request to a CPU core (i.e. to handle the error via an interrupt service routine), or issuing a programmable software reset (e.g. to mitigate the effect of a transient fault).

Besides consolidating hardware errors, errors detected by software can also be reported to CEHU by setting one of several dedicated SW error indication registers. This triggers the configured error reaction(s) in the same way as for hardware errors.

Most of the components within the chip are built to ASIL-C specification and include specific features that detect hardware and software faults (e.g. watchdog timers, duplicate logic, etc). When sub-components in the chip recognize a failures, it notifies the system through the CEHU, then sends a notification via error pin(s) if needed.

The customer is responsible for ensuring that software run on the Cortex A53 is safe. Special care must be taken when using the Cortex A53**.** Though the caches in the Cortex A53 are either ECC or parity protected, this Cortex A53 is considered a QM component, and thus does not automatically detect its own hardware failures. This software must not only be safe from software bugs and failures, but must also compensate for running on unreliable QM hardware.

The Cortex R52, an ASIL component, is provided to aid the customer in this effort. For example, cross-checking software may run on the Cortex R52 to verify that software running on the Cortex A53 remains safe. Customer determine what type of software needs to run on the Cortex R52 to meet their safety requirements of any software they write on the system.

The Cortex A53 is not the only QM component on the chip. A collection of peripherals used for debugging and development only, such as JTAG, are also QM but do not pose a risk to the safety of the chip because they are inactive during deployment.

4.2.2 SDK Safety Architecture

The SDK delivers the software-side safety architecture as a collection of driver libraries collectively called the IO Driver (Safety Domain). These libraries implement the software safety mechanisms defined in Sections 5.1 and 5.2 and provide a structured software interface between the system integrator's application and the safety hardware of the CV22FS.

For every safety mechanism that carries an SDK Usage row — whether a hardware SM or a software SM — the SDK provides a driver API. The system integrator is required to call those APIs as described in the SDK Usage instructions. The structurally distinct components are the SW timing and health monitors — pure-software mechanisms that continuously supervise the timing and health of DSP and VIN events — and the IPC Safety layer, which is cross-core communication infrastructure (A53 ↔ R52); neither is a per-SM driver.

CEHU configuration (error masks, safety modes, interrupt handling) and the integrator's fault response logic are outside the SDK scope and are performed by the system integrator directly.

4.2.2.1 SDK APIs for Safety Mechanisms

For each SM with an SDK Usage row, the SDK provides a driver API covering initialization, configuration, operation, and (where applicable) status polling or test execution. The integrator must use the API as described — this is the binding usage contract for each SM.

SM ID	SDK API Role
SM_HW_PSMON	Initialize power supply monitor and read status flags (AmbaPsMon.c)
SM_HW_CKMON	Configure per-domain clock frequency bounds and read status (AmbaClkMon.c)
SM_HW_WDT	Start, feed, stop, and query the hardware watchdog timer (AmbaDSP_SetSystemCfg)
SM_HW_ORC_TIMEOUT	Read the ORC wait-timeout status (SPR_TOR) and handle the timeout event
SM_HW_TESTFRAME	Configure and run the test frame, then verify the checksum (AmbaDSP_TestFrameConfig / AmbaDSP_TestFrameCtrl)

SM ID	SDK API Role
SM_HW_VIN_STATUS	Read the VIN interface error status register (AmbaDiag_VinGetStatus)
SM_HW_ERR_COUNTER_THR	Set the pixel-data SRAM error-count threshold (via AmbaRTSL_CEHUInit)
SM_HW_LBIST	Read the boot-time Logic BIST result (triggered in AmbaBST.asm)
SM_HW_TESTDAG	Run the test DAG and read the global checksum register (DCB/VMB accumulation)
SM_HW_DRAM_SEGMENTATION	Configure DRAM region-access protection (AmbaMemProt_Config)
SM_HW_DRAM_BW_MON	Enable and read DRAM bandwidth statistic counters (AmbaDRAMC_GetStatisInfo)
SM_HW_ASTUCK_TEST	Execute the address stuck test and read DTTE result registers
SM_HW_DRAM_ECC_CORR_CNT	Set the DRAM ECC corrected-error threshold and poll the counter (AmbaDRAMC_GetEccCount / AmbaDRAMC_ResetEccCount)
SM_HW_BM_ID_CHK	Configure AHB decoder master-ID access checking (AmbaRTSL_AxiSetClientAccess)
SM_HW_CAN_RE	Configure CAN RX error detection and read ECRU status
SM_HW_VOUT_CS	Read the VOUT frame checksum for frame-freeze detection (AmbaVout_GetDisp0Status / AmbaVout_GetDisp1Status)
SM_HW_ECC_CORR_CNT	Set the ECC corrected-error threshold and poll the counter
SM_SRAM_SECDED_ADDR_DATA	Read SRAM address/data SECDED ECC status
SM_HW_PO_MBIST	Read the power-on MBIST result (triggered in AmbaBST_BIST.asm)
SM_SW_WDT_ORC	Monitor ORC liveness from the R52 (AmbaDSP_MonitorInit / AmbaDSP_MonitorConfig)
SM_SW_CHK_ORC_INTF	Configure the timestamp-interface check (AmbaDSP_MainSafetyCfg)

SM ID	SDK API Role
SM_SW_WDT_VIN	Configure the VIN first-frame timeout (AmbaDSP_ResourceLimitConfig)
SM_SW_CODE_CRC	Generate and compare the coding-stream checksum (AmbaDSP_TestFrameConfig / AmbaDSP_TestFrameCtrl)
SM_SW_SD_ERR_CHK	Monitor SD CRC / error results (AmbaSD_ReadSector return value)

4.2.2.2 SW Timing and Health Monitors

Beyond the per-SM driver APIs, the SDK provides pure-software safety mechanisms that give temporal supervision and health monitoring of the periodic events produced by the DSP and the video input (VIN), ensuring both timing correctness and data plausibility before the data is consumed by safety-critical logic. These monitors run as continuous tasks across the QM Cortex-A53 and the ASIL Cortex-R52 and surface detected faults to the R52 and/or the CEHU; like the IPC Safety layer, they are structurally distinct from the per-SM driver APIs.

DSP Health and Data Plausibility Monitor (SM_SW_DSP_HEALTH_MON): The Cortex-A53 acts as a data aggregator and checker — it consumes DSP messages through the Message Dispatcher, performs application-defined plausibility checks (e.g. frame size, timestamp and frame-number consistency), and emits a periodic heartbeat only while the DSP data is plausible. The Cortex-R52 acts as the safety gatekeeper — a windowed watchdog verifies that the A53 heartbeat arrives within its time window and inspects the heartbeat payload for error flags. A missing heartbeat, an implausible or error status, or an invalid sequence counter is reported to the CEHU, and the R52 forces a safe state within the FTTI.

Software Frame Monitor (SM_SW_FRAME_MON): A software module provides temporal supervision of periodic frame streams from the DSP and VIN, detecting frame timeout, temporal jitter, excessive frame rate and inconsistent periodicity. Because it does not trigger the CEHU directly, it routes timing-error packets through the Message Dispatcher to the Cortex-R52, which is the decision maker: the R52 invalidates the affected frame, executes the configured counter-measure (re-synchronise, re-transmit, or enter a local safe state), and escalates to a safe state within the FTTI if the errors persist.

4.2.2.3 IPC — Inter-Processor Communication Safety Layer

The IPC Safety layer (implemented as a Complex Device Driver, CDD) is the software bridge between the QM Cortex-A53 and the ASIL Cortex-R52 cores. It manages a shared memory region (software shadow) carrying two categories of cross-core safety data:

- Safe state coordination — the A53 signals its operational state to the R52 safety monitor; the R52 can initiate a safe state transition if the A53 fails to update within the defined window.
- Heartbeat monitoring — periodic tokens are exchanged across cores to detect execution hangs or communication path failures.

Additional safety monitoring features integrated into the IPC layer include IRQ flooding detection and an ASIL-rated periodic safety checker executed in the R52 context. Key APIs: AmbaIPC_Init, AmbaIPC_Alloc, AmbaIPC_RegisterChannel, AmbaIPC_Send.

4.2.2.4 Architectural Relationships

The SDK provides APIs for all SMs with SDK Usage rows. The system integrator calls those APIs to initialize, configure, and operate each SM. For hardware SMs, when a fault is detected the hardware asserts a safety error signal directly to the CEHU hardware — this is a hardware-routed path independent of the SDK API layer. The SDK API covers the operational software interface; the hardware routes the resulting error signal to CEHU independently. CEHU configuration is outside the SDK scope and is performed by the integrator directly.

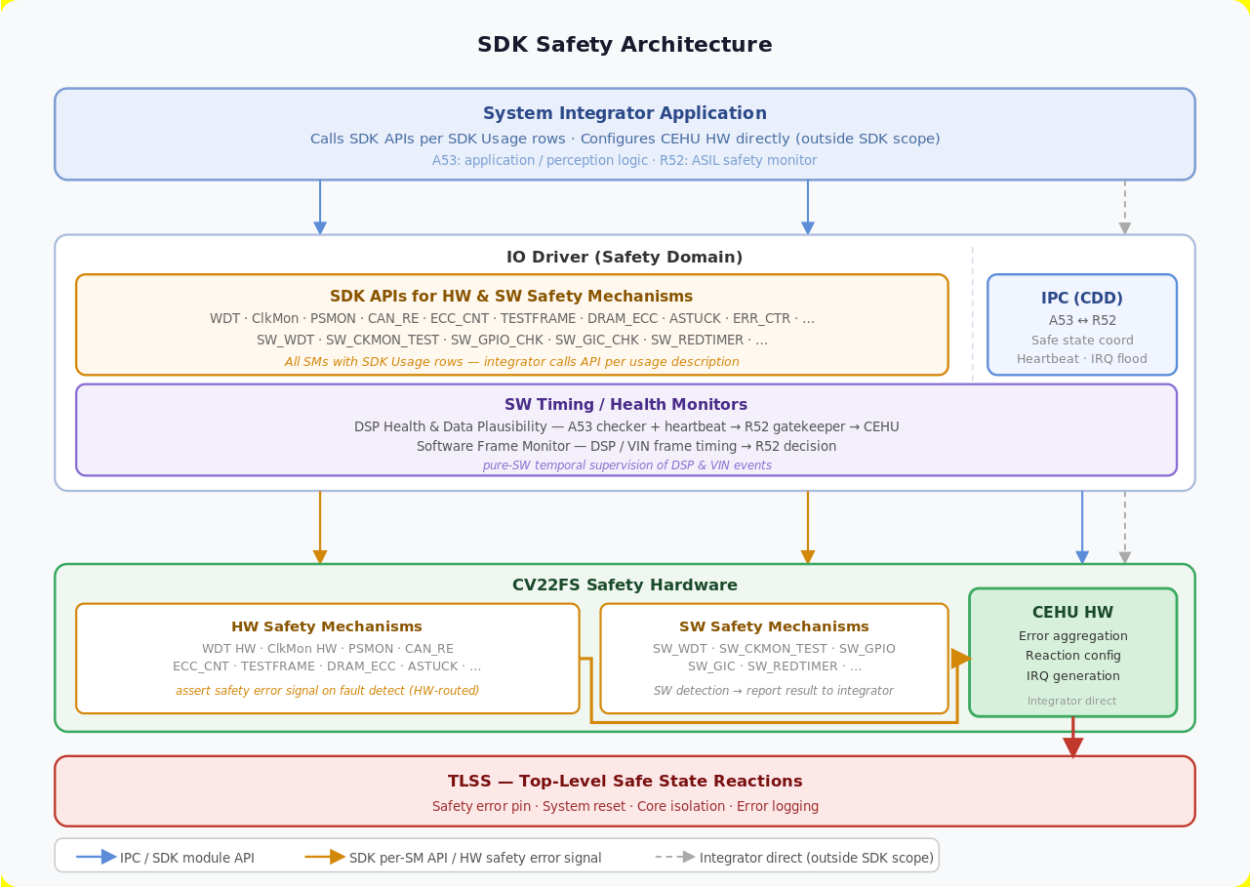


Figure: SDK Safety Architecture — IO Driver (Safety Domain)

4.2.2.5 Relationship to Section 4.2.1 (HW Safety Architecture)

Section 4.2.1 describes the hardware safety mechanisms implemented on the CV22FS. The SDK Safety Architecture provides the software interface layer required to:

- Configure and operate each safety mechanism with an SDK Usage row — the system integrator calls the SDK-provided API as described in the usage instructions.
- Detect software-visible fault conditions — the SW timing and health monitors surface DSP and VIN timing and data-plausibility faults, and the IPC layer surfaces execution failures (hangs, heartbeat loss, IRQ flooding), to the integrator's safety application.
- Coordinate cross-core safety state between the QM A53 and ASIL R52 (via IPC Safety layer).

CEHU error mask configuration, interrupt callback registration, and safe state reaction logic are the system integrator's responsibility and fall outside the SDK driver scope.

4.3 Error Handling

Error handling is served by the central error handling unit (CEHU). The purpose of the CEHU is to do the following:

1) Alert the external system that the chip is in error (go to the safe state) via safety pins.

2) Record the ID of the device that first reported an error for debugging purposes.

As the chip comes out of reset, the external safety pin is active-low, indicating error; until the boot process completes, software completes all safety checks, including checking safety monitors for latent faults, and determines that CEHU is operating properly, and the chip is otherwise healthy.

When the boot software completes safety checks on the CEHU, it resets any error signal set during testing and configures the CEHU for normal operation.

The CEHU may handle an error in the following ways:

a) 0: Propagate the error to the safety pin (default)

b) 1: Generate an interrupt to Arm's generic interrupt controller (GIC)

c) 2: Both a) and b)

d) 3: Disabled

The behavior of each error signal is controlled by a 2-bit SAFETY_MODE field.

Error signals that propagate to the safety pin are OR'd together to generate a SAFETY_ERROR signal that goes out to the safety pin. SAFETY_ERROR itself is sticky.

Error signals that generate an interrupt to the GIC are level sensitive.

The ERROR_ID provides the component that first encountered an error. A value of '0' indicates that the chip is executing safely. This status register only serves for debugging purposes when an error is detected, and the system has already entered a safe state. The register is visible on the AMBA bus and may be cleared by software. This register, however, cannot be relied on to classify errors as the register is programmed to classify only certain errors; for example, if the reference clock is in error, then nothing is recorded in the register.

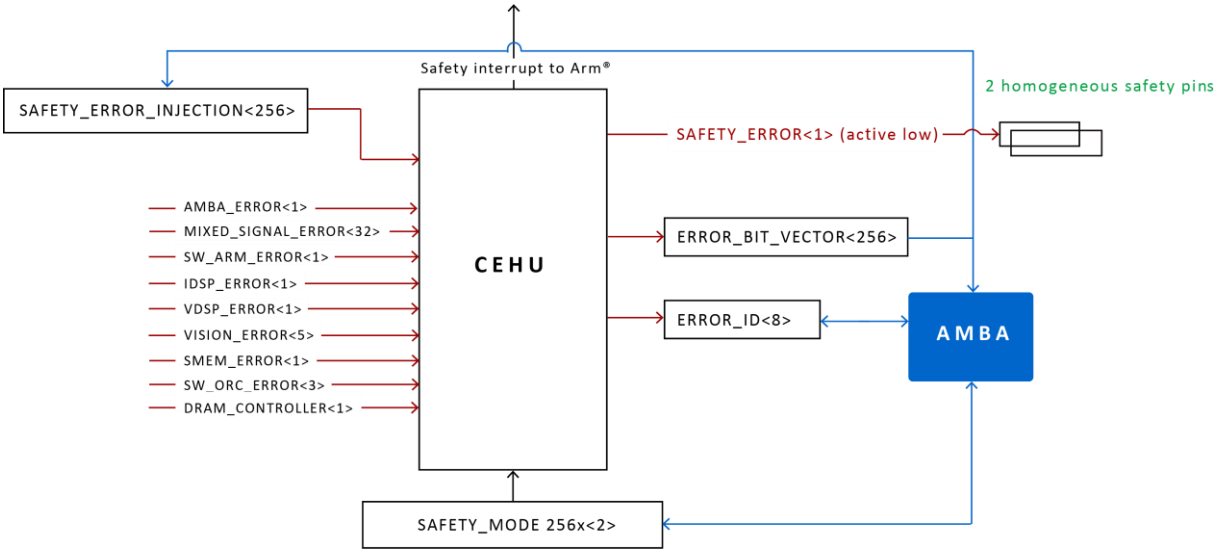


Figure 3 CEHU Internals

Notes:

1) There are two CEHUs in CV2FS/CV22FS/CV2FSA

2) Figure 3 is just a conceptual view

The ERROR_BIT_VECTOR holds the state of the error signals at input to the CEHU and is set independent of the SAFETY_ERROR_MODE.

The ERROR_ID and ERROR_BIT_VECTOR values are visible via the AMBA bus for debugging purposes. Reading the ERROR_BIT_VECTOR requires multiple cycles to read out the entire bit vector.

The SAFETY_ERROR_INJECTION register holds 1 bit per error signal. Setting a bit in this register to '1' forces the corresponding input error signals to active-low prior to error propagation or interrupt generation.

SAFETY_ERROR_INJECTION and SAFETY_MODE registers are used to determine that the CEHU itself is free of latent faults prior to normal operation. Once software is done inspecting the CEHU, it resets any error signal set during testing and configures the CEHU for normal operation.

Some sub blocks may generate many internal error signals and aggregate to a single level sent to the master CEHU.

4.3.1 SDK Error Handling

[DRAFT]

The SDK is provided in the form of a driver. Users of the driver shall follow the safety manual and the associated software documents to handle the error signals from software modules.

Users shall design their own software architecture to ensure the proper execution of error handling. The system integrator is responsible for defining and implementing the safety reactions appropriate to their application context.

5 Description of Safety Mechanisms

This chapter describes the safety mechanisms and assumption-of-use that are considered during the development of the safety requirements. Since the SEooC was developed based on the three use cases all together, system integrator is supposed to discard the ones that are not applicable to the system. The following table shows the template used to describe each safety mechanism in the sub-sections 5.1 and 5.2 .

Name	Description
ID	*Unique ID used for this safety mechanism*
Description	*The description for this safety mechanism*
Ownership	***Ambarella**: Ambarella owns the safety mechanism implemented on silicon, and the binary library of the SDK if applicable.* ***System Integrator**: the System Integrator must analyze, implement, integrate and verify the software used to initialize or operate this safety mechanism.*
SM is to against	*The description of the failure mode(s) that is covered by this safety mechanism*
Software Related	***Yes** or **No**, to mention whether this safety mechanism needs software to initialize or operate*
Failure mode coverage	*The level of failure mode this safety mechanism can cover*
Initialization	*The procedure to initialize this safety mechanism*
Operation	*The procedure to operate this safety mechanism*
Error indication	*The description of how a detected error is indicated to the system*
Error detect time	*Maximum time to detect and indicate an error*
Measure to prevent latent fault	*The measure to prevent latent fault of this safety mechanism*

5.1 Hardware Safety Mechanisms

A HW-SM (hardware safety mechanism) is a safety mechanism implemented by Ambarella on silicon to catch the faults that either possibly violate safety goal, or are latent. It may need software to enable diagnostic and/or take action for fault detection.

5.1.1 Power Monitor

Name	Description
ID	SM_HW_PSMON
Description	Power supply monitor (PSMON) is a safety detection functional circuit. It supervises all the power supplies and outputs the status flags to indicate the power supply operation conditions. System Integrator is responsible for analyzing the system configuration and defining the target power rail to monitor, as well as safety threshold criteria.
Ownership	Ambarella and System Integrator
SM is to against	“under and over voltage”, and “drift” in power supply
Software Related	Yes
Failure mode coverage	99%

Initialization	– Setup the the related power monitor power down and disable configuring register. - Pd_declo (pm_ctrl2[12])=1'b0 (low gear detector power down control) - Pd_dechi1 (pm_ctrl2[13])=1'b0 (high gear detector 1 power down control) - Pd_declo1 (pm_ctrl2[13])=1'b0 (low gear detector 1 power down control psmon_lv version) - Pd_dechi2 (pm_ctrl2[14])=1'b0 (high gear detector 2 power down control) - Pd_declo2 (pm_ctrl2[14])=1'b0 (low gear detector 2 power down control psmon_lv version) - Pm_disabble(pm_ctrl2[23]) =1'b0 (disable control) – Setup the input level select configures according to power monitor distribution doc. - vsensesello(pm_ctrl2[3:0]) (low gear detector sense level select) - vsenseselhi1(pm_ctrl2[7:4]) (high gear detector 1 sense level select) - vsensesello1(pm_ctrl2[7:4]) (low gear detector 1 sense level select psmon_lv version) - vsenseselhi2(pm_ctrl2[11:8]) (high gear detector 2 sense level select) - vsensesello2(pm_ctrl2[11:8]) (low gear detector 2 sense level select psmon_lv version) – Setup the under voltage and over voltage comparison reference level - Vuvsello (pm_ctrl1[3:0]) (Default=3'1h3: low gear monitor under voltage reference) - Vovsello (pm_ctrl1[7:4]) (Default=3'h3: low gear monitor over voltage reference) - Vuvselhi1 (pm_ctrl1[11:8]) (Default=3'h3: high gear monitor 1 under voltage reference) - Vuvsello1 (pm_ctrl1[11:8]) (Default=3'h3: low gear monitor 1 under voltage reference psmon_lv version) - Vovselhi1 (pm_ctrl1[15:12]) (Default=3'h3: high gear monitor 1 over voltage reference) - Vovsello1 (pm_ctrl1[15:12]) (Default=3'h3: low gear monitor 1 over voltage reference psmon_lv version) - Vuvselhi2 (pm_ctrl1[19:16]) (Default=3'h3: high gear monitor 2 under voltage reference) - Vuvsello2 (pm_ctrl1[19:16]) (Default=3'h3: low gear monitor 2 under voltage reference psmon_lv version) - Vovselhi2 (pm_ctrl1[23:20]) (Default=3'h3: high gear monitor 2 over voltage reference)

	- Vovsello2 (pm_ctrl1[23:20]) (Default=3'h3: low gear monitor 2 over voltage reference psmon_lv version) – Clear old result by toggling restn signal - Pm_rstn(pm_ctrl2[25]) – Enable sticky function - Sf_enable (pm_ctrl2[29]) = 1'b1 (sticky function enable control) – Observe the error report signal - pm_errorb, 1'b1: safe; 1'b0: PSMON safety error - Note: pm_errorb=1'b1 for reset and pm_errorb=1'b0 for disable stat
Operation	N/A
Error indication	Assert safety error signal
Error detect time	100 us
Measure to prevent latent fault	SM_SW_PSMON_TEST
SDK Usage	The Power Supply Monitor (PSMON) is a dedicated hardware safety circuit that continuously supervises all power rails on the device and outputs status flags to indicate operating conditions. This mechanism ensures that all supply voltages remain within safe operating thresholds, enabling early detection of power-related faults before they propagate to functional safety logic. **Detected Error Types:** **Over-voltage Condition:** Detection of a power rail exceeding the programmed upper safety threshold. **Under-voltage Condition:** Detection of a power rail dropping below the minimum required level, which could compromise logic stability.

	Power Rail Failure: Complete loss of a monitored supply, indicating a catastrophic power fault. **Threshold Crossing:** Any deviation outside the configured window for safety-critical rails. **Assumptions of Use:** **1. Safety Requirement Analysis and Rail Selection:** The System Integrator shall perform a safety analysis to identify all power rails critical to the system's Safety Goals. Based on this analysis, the integrator shall determine which rails require monitoring and define appropriate safety thresholds (upper and lower voltage limits) for each rail. **2. SDK-Based Configuration:** The System Integrator shall utilize the provided SDK APIs (AmbaPsMon.c) to configure the PSMON during system boot. The reference sample code (SvcSafeState_ConfigPwrMon() in SvcSafeStateMgr.c) shall be used as the validated starting point. **3. CEHU Integration and Error Handling:** PSMON fault signals are routed to the Central Error Handling Unit (CEHU). The System Integrator shall implement an error-handling routine on the Cortex-R52 to monitor CEHU status registers and define the specific safety reaction for each power fault (e.g., immediate system reset, transition to Safe State, or driver notification). **4. Safe State Transition:** The System Integrator shall ensure that upon detection of a power supply fault, the system transitions to a Safe State within the Fault Tolerant Time Interval (FTTI). The safe state reaction must not depend on subsystems powered by the failing rail. **SDK API and integration flow:** The PSMON driver API is provided in AmbaPsMon.c; the reference configuration flow is SvcSafeState_ConfigPwrMon() in SvcSafeStateMgr.c, used to configure PSMON during boot. *Note: The reference code / sample code is provided to demonstrate usage only and shall not be used as production code.*

5.1.2 Duplicated Logic

Name	Description
ID	SM_HW_DUP
Description	The functional logic (a.k.a. mission logic) is replicated as a shadow logic and runs the same operation at the same time in parallel. The outputs of mission logic and the shadow logic are compared to determine if there's a fault.
Ownership	Ambarella

SM is to against	Any random hardware fault.
Software Related	No
Failure mode coverage	99%
Initialization	N/A
Operation	N/A
Error indication	Assert safety error signal
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_LBIST (if applicable)

5.1.3 Software Check Dummy Resettable/Writeable Registers

Name	Description
ID	SM_HW_DUMMY_REG
Description	Implement a dummy readable / writeable register for each reset domain which is not covered by other safety measures. System integrator is responsible for implementing and integrating this safety mechanism software with Ambarella's reference code.
Ownership	Ambarella and System Integrator
SM is to against	stuck-at fault on reset signal
Software Related	Yes
Failure mode coverage	99%
Initialization	N/A

Operation	Software shall check the state of these dummy registers right after the reset procedure is over. Correct register default values implies reset was asserted. Once the check is complete, software shall then write non-default values to these dummy registers. Refer to Ambarella's reference code for the list of dummy registers.
Error indication	Assert safety error signal by software
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_DUMMY_REG

5.1.4 Clock Monitor

Name	***Description***
ID	SM_HW_CKMON
Description	The clock monitor can check the target clock frequency and see if the target clock is stuck. 24 MHz reference clock is cross verified with a 32 KHz RTC clock. All other clock monitor's reference clock is the 24 MHz reference clock. These clock monitors are placed at the following nodes. - PLL outputs - Clock divider outputs - Cluster level clock gating cell output - Right before Cortex-R52 CLKIN input - Clock to main logic of lockstep / duplicate function System integrator is responsible for analyzing the system configuration and defining the target clocks to monitor, as well as the safety threshold criteria.
Ownership	Ambarella and System Integrator
SM is to against	Incorrect frequency or no clock
Software Related	Yes
Failure mode coverage	99%

Initialization	• Clock monitors assert error flag to CEHU during power on reset, SW should check that corresponding error bits in CEHU are set to make sure clock monitors are well reset. Clock monitors for 24 MHz system clock and 32 KHz RTC clock also wired-out to CEHU safety error IO pins in case clock to CEHU not available. • Clear clock monitor CEHU error bits. • Program PLLs and clock dividers for preferred clock frequencies. • Program upper_bound and low_bound of clock monitors. • Program enable input of clock monitors to start checking clock frequencies. • Program related settings in CEHU. Please refer to Programming Reference Manual for the details of the configurations.
Operation	N/A
Error indication	Assert safety error signal
Error detect time	99 ms
Measure to prevent latent fault	SM_SW_CKMON_TEST
SDK Usage	The device features a comprehensive hardware-based clock monitoring infrastructure designed to ensure the temporal integrity of the system's timebase. This mechanism employs a "Root of Trust" hierarchy where a 24 MHz reference clock is cross-verified against a 32 KHz RTC clock. Once validated, the 24 MHz clock serves as the reference for distributed monitors located at safety-critical nodes throughout the clock tree. **Monitoring Locations:** Source Level: PLL outputs and Clock Divider outputs. **Distribution Level:** Cluster-level clock gating cell outputs. **Terminal Level:** Cortex-R52 CLKIN input and clocks driving main logic for lockstep/duplicated functions. **Detected Error Types:** **Frequency Drift:** Detection of target clocks running outside the programmed frequency range (Over-frequency or Under-frequency). **Clock Stuck-at:** Detection of a complete loss of clock pulses (stuck-at-high or stuck-at-low). **Reference Clock Failure:** Inconsistency between the 24 MHz reference and the 32 KHz RTC clock, indicating a failure in the primary timebase. **Gating Malfunctions:** Presence of a clock signal when a cluster-level gate is commanded to be disabled, or absence when enabled. **Assumptions of Use**

	1. Safety Requirement Analysis and Configuration The System Integrator shall perform a safety analysis to determine which clock nodes are critical to the system's Safety Goals. Based on this analysis, the integrator shall define the safety thresholds (e.g., maximum allowable frequency deviation) for each monitored node. The integrator must ensure these thresholds are aligned with the system's hardware-software interface (HSI) and the timing requirements of safety-critical tasks. 2. SDK-Based Monitor Initialization The System Integrator shall utilize the provided SDK APIs to configure the clock monitors during the system boot sequence. This includes programming the target frequency, window tolerances, and enabling the monitors before safety-critical applications are allowed to execute. The sample code provided in the SDK should be used as a reference for validated register settings. 3. Reference Clock Cross-Verification The System Integrator shall ensure the 24 MHz vs. 32 KHz cross-verification is successfully completed and validated before the 24 MHz clock is used as a reference for any other system monitors. Any failure at this root level must prevent the system from entering "Normal Operation." 4. CEHU Integration and Error Handling The hardware clock monitors are hardwired to signal failures to the Central Error Handling Unit (CEHU). The System Integrator shall: Implement an error-handling routine (typically on the Cortex-R52) to monitor the CEHU status registers. Define the specific safety reaction for each clock failure (e.g., system reset for a PLL failure, or transitioning a specific cluster to a Safe State for a local gating failure). Ensure the reaction time from the CEHU error signal to the achievement of a Safe State is within the defined Fault Tolerant Time Interval (FTTI). **SDK API and integration flow:** Configure each clock monitor via AmbaClkMon_Config(UINT32 ClkMonID, UINT8 Enable, UINT8 UpBound, UINT8 LowBound); UpBound and LowBound set the allowed frequency deviation in percent above and below nominal (valid range 0-100; returns 0U on success or CLKMON_ERR_ARG on invalid argument). The driver API is in AmbaClkMon.c; the reference configuration flow is SetClkMon() in SvcSafeStateMgr.c. *Note: The reference code / sample code is provided to demonstrate usage only and shall not be used as production code.*

5.1.5 Monitor xx_test_mode PAD VIA a Combinational path to CEHU

Name	*Description*

ID	SM_HW_TEST_MODE_MON
Description	The xx_test_mode pad is monitored via a combinational path to CEHU.
Ownership	Ambarella
SM is to against	Faults in package or xx_test_mode pad
Software Related	No
Failure mode coverage	99%
Initialization	N/A
Operation	N/A
Error indication	Assert safety error signal
Error detect time	99 ms
Measure to prevent latent fault	None

5.1.6 Duplicate Mode Decoding Logic

Name	*Description*
ID	SM_HW_TMODE_DEC_DUP
Description	Duplicating the mode decoding logic provides protection against faults in the test mode decoding logic.
Ownership	Ambarella
SM is to against	Faults in test mode decoding pads or logic
Software Related	No

Failure mode coverage	60%
Initialization	N/A
Operation	N/A
Error indication	Assert safety error signal
Error detect time	99 ms
Measure to prevent latent fault	None

5.1.7 Monitor ijtag_reset from Different Clusters via a Combinational Path to CEHU

Name	*Description*
ID	SM_HW_DFT_RST_MON
Description	The ijtag_reset from different clusters is monitored via combination path to CEHU
Ownership	Ambarella
SM is to against	Any fault in DFT asynchronous reset signals
Software Related	No
Failure mode coverage	60%
Initialization	N/A
Operation	N/A
Error indication	Assert safety error signal
Error detect time	99 ms

Measure to prevent latent fault	None

5.1.8 CRC for Address and Data

Name	*Description*
ID	SM_HW_OTP_CRC
Description	OTP controller checks the CRC checksum which is encoded with address and data for the protected entries.
Ownership	Ambarella
SM is to against	Faults in OTP memory or data read paths
Software Related	No
Failure mode coverage	99%
Initialization	N/A
Operation	N/A
Error indication	Assert safety error signal
Error detect time	99 ms
Measure to prevent latent fault	None

5.1.9 Parity

Name	*Description*

ID	SM_HW_PAR
Description	In a parity safety mechanism, a parity bit is added to one or more data bits to ensure the total number of 1s is even or odd. In the SEooC, even parity is used unless otherwise specified.
Ownership	Ambarella
SM is to against	Any transient fault in flops.
Software Related	No
Failure mode coverage	99%
Initialization	N/A
Operation	N/A
Error indication	Assert safety error signal
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_LBIST (if applicable)

5.1.10 Hardware Watchdog Timer

Name	*Description*
ID	SM_HW_WDT
Description	In the safety measure of hardware watchdog timer, it implements watchdog timers to detect hang for a specific functional block. The hardware watchdog timer shall be enabled with proper timeout threshold by either Cortex software via debug port, or ORC software via special IDSP command or VP poke command. System integrator is responsible for analyzing the system configuration and configuring the required timeout threshold to meet the safety requirement.

Ownership	Ambarella and System integrator
SM is to against	Any fault that causes hang in the IDSP pipeline or in the ORC-IDSP command transfer, or in the VP module, or in display sections in VOUT pipeline.
Software Related	Yes
Failure mode coverage	99%
Initialization	Program the watchdog timer with appropriate threshold cycles
Operation	N/A
Error indication	Assert safety error signal
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_LBIST (if applicable)
SDK Usage	This safety mechanism provides a multi-layered approach to detect execution "hangs" and "stalls" within specialized functional blocks (such as the IDSP and Vector Processor). It combines software-configurable timeouts managed by the ORC with independent hardware watchdog timers (WDT) to ensure that no safety-critical functional block remains in an unresponsive state indefinitely. **Detected Error Types:** **Functional Block Hangs:** Complete cessation of processing within a specific hardware block (e.g., IDSP or VP) due to internal logic errors or resource conflicts. **Infinite Wait States:** Situations where the ORC or IDSP waits for a resource or event that never occurs. **Software/Firmware Deadlocks:** Logical deadlocks within the ORC software that prevent the completion of IDSP commands. **Processing Overruns:** Scenarios where a functional block is operational but fails to complete a task within the safety-critical time limit. **Assumptions of Use** 1. IDSP Timeout Configuration via SDK API The System Integrator shall ensure the IDSP timeout duration is correctly configured during system initialization. The integrator can reuse default settings or utilize the AmbaDSP_SetSystemCfg(31, msec)

	API to define a custom timeout based on the system's worst-case execution time (WCET) for vision or image processing tasks. 2. Hardware Watchdog Enablement and Calibration The System Integrator is responsible for enabling the hardware watchdog timers for all safety-related functional blocks. This must be performed via one of the supported paths: **Cortex Software:** Configuration via the debug port. **ORC Software:** Direct configuration via special IDSP commands or VP "poke" commands. The watchdog must be enabled before the functional block begins processing safety-critical data. 3. Safety Analysis of Timeout Thresholds The System Integrator shall perform a detailed safety analysis to determine the appropriate timeout thresholds for both the ORC software timers and the hardware WDTs. These thresholds must be: **Robust:** Long enough to avoid "False Positives" caused by transient system jitter or heavy bus traffic. **Safe:** Short enough to ensure that the detection and subsequent system reaction occur within the Fault Tolerant Time Interval (FTTI) of the relevant Safety Goal. 4. Error Handling Upon detection of a watchdog reset or an ORC-reported timeout, the system must notify the safety-critical software (typically running on the Cortex-R52). The System Integrator shall implement a handler to: Log the specific block ID that timed out. Invalidate any data outputs originating from the failed block. Command a reset of the affected hardware cluster if necessary to restore operation. 5. Safe State Transition The System Integrator shall ensure that if a functional block hang is detected, the system transitions to a Safe State (e.g., switching to a backup sensor, notifying the driver, or initiating an emergency stop). The integrator must verify that the transition to the safe state is not dependent on the hardware block that has just timed out. **SDK API and integration flow:** Configure the IDSP timeout from the ARM side via AmbaDSP_SetSystemCfg(31, msec); the ORC applies this setting on IDSP timeout.

5.1.11 ORC Wait Timeout Mechanism

Name	*Description*
ID	SM_HW_ORC_TIMEOUT

Description	The wait instruction of the ORC will timeout after waiting for a specific number of cycles programmed by software.
Ownership	Ambarella
SM is to against	Any fault that causes hang in IDSP, VDSP clients, or FEX.
Software Related	Yes
Failure mode coverage	99%
Initialization	N/A
Operation	Software can read from special purpose register timeout reason (SPR_TOR) after a wait instruction to determine if a timeout occurred.
Error indication	Assert safety error signal by software
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_LBIST
SDK Usage	The ORC (Output/Result Computer) Wait Timeout Mechanism detects execution hangs in the ORC processors embedded in the IDSP and CVflow engine. Each ORC has a software-configurable timeout counter: if a wait instruction does not complete within the programmed number of cycles, the hardware fires a timeout interrupt and asserts a safety error signal. Software reads the timeout reason from the Special Purpose Register SPR_TOR to identify the cause. **Detected Error Types:** **Functional Block Hang:** The ORC wait instruction did not complete within the timeout threshold, indicating that the downstream functional block (IDSP or VP) has stopped responding. **ORC Execution Stall:** The ORC itself has entered an infinite or excessively long wait, suggesting an internal logic error or deadlock condition. **Assumptions of Use** **1. Timeout Threshold Configuration:** The System Integrator shall program an appropriate timeout threshold in cycles for each ORC instance, derived from the worst-case legitimate execution time of the functional block. **2. Interrupt Handler and SPR_TOR:** The System Integrator shall implement an interrupt handler that reads SPR_TOR from the

	affected ORC upon a timeout interrupt to determine the timeout reason, then logs the result and determines whether a safe state transition is required. **3. CEHU Integration:** Upon confirming a timeout, the safety application shall assert a safety error signal. Note: the "Assert safety error signal by software" error indication means software must explicitly write to CEHU — it is not automatically asserted by hardware. **4. Relationship to SM_HW_WDT:** SM_HW_ORC_TIMEOUT provides per-instruction timeout detection within the ORC; SM_HW_WDT provides a coarser watchdog for the entire functional block. Both are complementary and should be configured together for complete coverage. **SDK API and integration flow:** This safety mechanism is implemented inside the microcode (ucode); the System Integrator shall check the error reported from the DSP driver or the CEHU.

5.1.12 Test Frame

Name	*Description*
ID	SM_HW_TESTFRAME
Description	In the safety measure of test frame, a checksum calculator is implemented. IDSP filters are first programmed with a pre-calculated golden checksum. Each IDSP filter then calculates and accumulates a checksum over the span of the test frame, and compares the generated checksum against the golden checksum at the end of frame. In VDSP and FEX, the datapaths calculate a checksum over the test frame, software running on the ORC then reads the checksum and compares it against the golden checksum. The ORC library binary (ucode) provided by Ambarella is in charge of executing the Test Frame safety measure. System Integrator shall integrate the binary in the system software and trigger the safety mechanism per the requirement.
Ownership	Ambarella and System integrator
SM is to against	A majority of permanent errors in the data-path flops and combinational logic of IDSP filters, VDSP clients, and FEX that do not use duplicated logic as a safety measure.
Software Related	Yes

Failure mode coverage	60 to 90%
Initialization	N/A
Operation	SW runs a pre-determined frame by issuing a test frame command to IDSP. IDSP programs all IDSP filters which are active in that mode with the golden checksum values. IDSP filters then compute and accumulate checksum during the execution of the test frame. The collective checksum is compared against the programmed checksum at the end of the frame. A mismatch indicates a safety error. For VDSP and FEX, the datapaths calculate a checksum over the test frame, the software running on the ORC then reads the checksum and compares it against the golden checksum. If there is a mismatch, the software will write to a special purpose register to signal an error.
Error indication	Assert safety error signal
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_LBIST
SDK Usage	The DSP engine includes a "Test Frame" diagnostic function provided via the SDK. This mechanism performs periodic online hardware latent fault detection and logical execution monitoring of the DSP pipeline. It ensures that the DSP hardware and associated data paths are functioning within specified operational limits. **Assumptions of Use** 1. Configuration of Diagnostic Frequency The System Integrator shall configure the execution period of the DSP Test Frames. This period must be defined such that the sum of the Test Frame execution time and the software overhead remains below the Fault Tolerant Time Interval (FTTI) of the relevant system-level Safety Goal. 2. Error Handler Registration The System Integrator shall implement and register a high-priority callback function (Error Handler) with the SDK. This handler is responsible for capturing asynchronous error notifications generated by a Test Frame failure in the DSP engine. The Test Frame may also trigger CEHU error which system integrator shall handle. 3. Safety Response and Safe State The System Integrator shall define a diagnostic monitoring strategy to evaluate the results of the Test Frame execution. Upon detection of a "Test Frame Failure" the system shall transition to a Safe State (e.g., notifying the driver, disabling automated features, or triggering a system reset) within the time remaining in the FTTI.

	SDK API and integration flow: IDSP/VDSP test frame APIs: AmbaDSP_TestFrameConfig() and AmbaDSP_TestFrameCtrl().

5.1.13 VIN ECC Decoder

Name	*Description*
ID	SM_HW_VIN_ECC
Description	Per MIPI-DPHY specification 1.1, Hamming modified code (30, 24) shall be implemented to detect or correct the error in data packet header. Ambarella reuses the protocol requirement as the safety measure to detect fault in the data path of the MIPI-DPHY module. Ambarella is responsible for the CRC checker implementation in silicon logic. The System Integrator is responsible for the Image sensor device configuration and control, as well as the signal integrity on board level which all are the impact factors for the sensor data integrity.
Ownership	Ambarella and System integrator
SM is to against	Any fault in MIPI CSI-2 interface with external sensor, MIPI-DPHY and front end part of VIN datapath
Software Related	Yes
Failure mode coverage	50% 99% for continuous multiple-bit error (CMBE)
Initialization	SW shall enable this feature by setting MIPI_ECC_ENABLE in the VIN register. Refer to Programming Reference Manual for the details of the configuration.
Operation	N/A
Error indication	Assert safety error signal
Error detect time	99 ms

Measure to prevent latent fault	SM_HW_LBIST

5.1.14 VIN CRC Checker

Name	*Description*
ID	SM_HW_VIN_CRC
Description	Per the MIPI-DPHY specification 1.1, the CRC checker shall be implemented to check the data payload of packets. Ambarella reuses the protocol requirement as the safety measure to detect fault in the data path of the MIPI-DPHY module. The CRC checker on silicon is owned by Ambarella. System Integrator is responsible for configuring the CRC error threshold for this safety mechanism.
Ownership	Ambarella and System integrator
SM is to against	Any fault in MIPI CSI-2 interface with external video sensor
Software Related	Yes
Failure mode coverage	99%
Initialization	SW shall set proper CRC_ERR_THRESHOLD in the VIN register. Refer to Programming Reference Manual for the details of the configuration.
Operation	N/A
Error indication	Assert safety error signal
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_LBIST

5.1.15 VIN ORC SW Readable Status Register

Name	*Description*
ID	SM_HW_VIN_STATUS
Description	This status register records any communication error detected on the video sensor interface, like, ECC and CRC errors in control and data packets from video sensor, short line and short frame input from video sensor, FIFO overflows, MIPI control packet sequence error, etc. System integrator shall analyze the system safety requirement and implement the corresponding software to read out the VIN status for safety application.
Ownership	Ambarella and System integrator
SM is to against	Corrupted control or data packets on video sensor input interface
Software Related	Yes
Failure mode coverage	99%
Initialization	N/A
Operation	Software can poll this status register either periodically or upon interrupt to determine if any safety critical error occurred on the video input interface and take either corrective measures or report to CEHU, depending of the type of error.
Error indication	Assert safety error signal by software
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_LBIST
SDK Usage	The Video Input (VIN) module incorporates a hardware-based monitoring unit that detects transmission and protocol violations on the video sensor interface (e.g., MIPI CSI-2). This mechanism ensures the integrity of the raw image data. **Detected Error Types:** **Data Integrity Errors:** ECC and CRC failures in both control and data packets.

	Timing & Frame Errors: Detection of "Short Line" or "Short Frame" conditions, indicating synchronization loss or sensor malfunctions. **Protocol Violations:** MIPI control packet sequence errors and invalid start/end-of-transmission sequences. **Resource Management Errors:** FIFO overflow conditions, indicating data rate mismatches or internal bus congestion. **Assumptions of Use** 1. Software Diagnostic Integration The System Integrator shall implement a diagnostic software routine to periodically monitor the error reported from SDK VIN module. This routine must be integrated into the system's safety-critical execution path. 2. Error Threshold and Safety Analysis The System Integrator shall analyze the impact of VIN errors (e.g., a single CRC error vs. a persistent FIFO overflow) on the system-level Safety Goal. Based on this analysis, the integrator shall define which error types or frequencies constitute a failure. 3. Monitoring and Safe State Transition The System Integrator shall utilize the VIN API to monitor the health status of the video stream. Upon detection of a defined failure condition, the system shall transition to a Safe State (e.g., "Vision Data Invalid" flag, system fallback, or emergency stop) within the Fault Tolerant Time Interval (FTTI). **SDK API and integration flow:** The application reads VIN status via AmbaDiag_VinGetStatus(). Reference code: IoDiag_CheckVinStatus() (cortex_a/svc/apps/iout/src/diag/cv22fs). *Note: The reference code / sample code is provided to demonstrate usage only and shall not be used as production code.*

5.1.16 Error Counter and Threshold to Detect Permanent Fault in Pixel Data SRAM

Name	*Description*
ID	SM_HW_ERR_COUNTER_THR
Description	Transient errors in IDSP pixel data are considered not safety critical. However, permanent errors could be safety critical. Test-frames are used to detect permanent faults is IDSP data-paths. To detect permanent faults in the line buffer pixel data SRAMs, the test frame needs to be as wide as the maximum supported frame width by IDSP pipelines. In order to eliminate this requirement and reduce the test-frame overhead, the IDSP filters implement address+data parity on the line buffer SRAMs, an error counter, and a software programmable error threshold registers. Irrespective of whether a frame is test-frame or normal video frame, the parity error is reported to the error counter. The

	error counter then compares the accumulated error count against the programmed threshold and asserts the safety error output to CEHU when the error count exceeds the programmed threshold value. The same measure is applied to the SRAMs in VDSP, FEX, and VP functions.
Ownership	Ambarella
SM is to against	Permanent / persistent errors in the pixel data line buffers
Software Related	Yes
Failure mode coverage	99%
Initialization	Software may program the appropriate error threshold in each sub-block. If the threshold is set to 0, any fault in SRAMs is considered a critical error.
Operation	N/A
Error indication	Assert safety error signal
Error detect time	99 ms
Measure to prevent latent fault	N/A (this safety mechanism is to detect latent fault)
SDK Usage	The SDK provides a hardware-based diagnostic mechanism to detect permanent faults (e.g., "stuck-at" high or low) within the pixel data path of the CVflow and DSP engine. This mechanism utilizes an error counter to track the occurrence of pixel-level corruption during frame processing. If the number of detected pixel faults exceeds a user-defined threshold, a safety error signal will be asserted. **Assumptions of Use** 1. Configuration of Error Threshold The System Integrator shall determine and configure the maximum allowable pixel error threshold using the provided SDK API. This threshold shall be derived from a safety analysis (e.g., FMEA or Simulation) of the vision algorithm's robustness. The threshold must be set such that the density and distribution of permanent pixel faults do not result in a violation of the Safety Goal (e.g., "Failure to detect an object in the path").

	2. Fault Tolerance Evaluation The System Integrator shall justify the chosen threshold based on the system's operational design domain (ODD) and the specific requirements of the perception stack. If the vision algorithm cannot tolerate any pixel-level degradation, the threshold shall be set to the minimum detectable value. 3. Error Monitoring and Safe State Transition Upon reaching the configured threshold, the CEHU will report a error. The System Integrator shall monitor this error and implement a safety response that transitions the system to a Safe State within the FTTI. **SDK API and integration flow:** Thresholds are applied via AmbaRTSL_CEHUInit() (reference values: FEX threshold = 4; IDSP threshold = default 127). The System Integrator shall adjust these threshold values according to the product need.

5.1.17 Logic BIST

Name	*Description*
ID	SM_HW_LBIST
Description	During power-on, software executes LBIST to verify that the logic is free of hard defects before normal operation begins. As part of the boot process, data from the off-chip NAND flash is copied into an internal SRAM. The Arm boots from the program starting PC in the internal SRAM. The boot code copies LBIST vectors into the IST (In-system-test) SRAM and writes to a separate CONTROL register to kick off IST. An in-system test controller executes commands in LBIST SRAM to verify block under test. Once one LBIST operation is complete, the done/fail status is written to the STATUS register. Software on the Arm reads this status register via AMBA bus and generates error if the test fails. The chip has multiple LBIST controllers, and this process needs to be repeated for all of them. It is possible to run some LBIST controllers at chip shutdown to reduce boot time. If any LBIST controller is run at shutdown, the status information needs to be stored in RTC and read back and checked during boot process. If the IST controller fails to complete due to latent, transient, or permanent failures in the LBIST SRAM, the IST controller, or the status register itself; Arm software can still detect the problem and notify the CEHU via a write to the AMBA bus. System integrator is responsible for integrating the LBIST software from Ambarella into the start-up sequence.
Ownership	Ambarella and System integrator

SM is to against	Any latent fault
Software Related	Yes
Failure mode coverage	100% for D flip-flop 70% for combinational logic and ICG cells
Initialization	LBIST vector data needs to be transferred from flash to IST SRAMs and LBIST controllers need to be enabled.
Operation	N/A
Error indication	SW determines to remain in safe state by reading error bits in IST STATUS registers
Error detect time	Power-on test
Measure to prevent latent fault	N/A (this safety mechanism is to detect latent fault)
SDK Usage	The device incorporates multiple hardware LBIST controllers and an In-System-Test (IST) infrastructure to detect permanent (hard) defects in the logic design. This safety mechanism is designed to satisfy the diagnostic coverage requirements for the hardware during the Boot-up or Shutdown phases (latent fault detection). **Detected Error Types:** **Logic Hard Defects:** Detection of permanent faults within the "Block Under Test" through deterministic LBIST vector execution. **Control Path Failures:** Failures in the IST controller's ability to execute commands or manage the test sequence. **Infrastructure Failures:** Latent or permanent failures within the dedicated IST SRAM, the CONTROL register, or the STATUS register. **Sequence/Timeout Errors:** Situations where the IST operation fails to complete (hangs) or fails to report a "Done" status within the expected timeframe. **Assumptions of Use** 1. Software Test Management and Sequencing The System Integrator shall implement a boot-time software handler to orchestrate the IST flow. This includes: Copying valid LBIST vectors from non-volatile memory (NAND Flash) to the internal IST SRAM. Sequentially triggering all available LBIST controllers via the CONTROL register. Verifying the integrity of the status read-back via the AMBA bus before proceeding to normal operation.

	2. Persistent Storage of Shutdown Test Results If the System Integrator elects to run LBIST during the chip shutdown phase to optimize boot time, the software shall ensure the PASS/FAIL status is successfully committed to the Real-Time Clock (RTC) domain or other persistent, safety-related registers. During the subsequent boot process, the software must read, validate, and clear these stored results. 3. Redundant Error Reporting (CEHU Integration) The System Integrator shall configure the Arm-based software to act as a secondary monitor for the IST infrastructure. In the event of a test failure, or if the IST controller itself becomes--- unresponsive (indicating a failure in the SRAM or registers), the software must explicitly notify the Central Error Handling Unit (CEHU) via the AMBA bus. 4. Handling of IST Completion Failures The System Integrator shall define a timeout period for each IST operation. If the "Done" bit is not set within this interval, the system shall treat this as a hardware failure. The integrator must ensure that any detected LBIST failure or timeout prevents the system from exiting the "Start-up" state and entering "Normal Operation," effectively maintaining a Safe State. **SDK API and integration flow:** LBIST runs from bootstrap code (AmbaBST.asm); the safety application reads the result via API AmbaCEHU_BstBistResultGet(). *Note: The reference code / sample code is provided to demonstrate usage only and shall not be used as production code.*

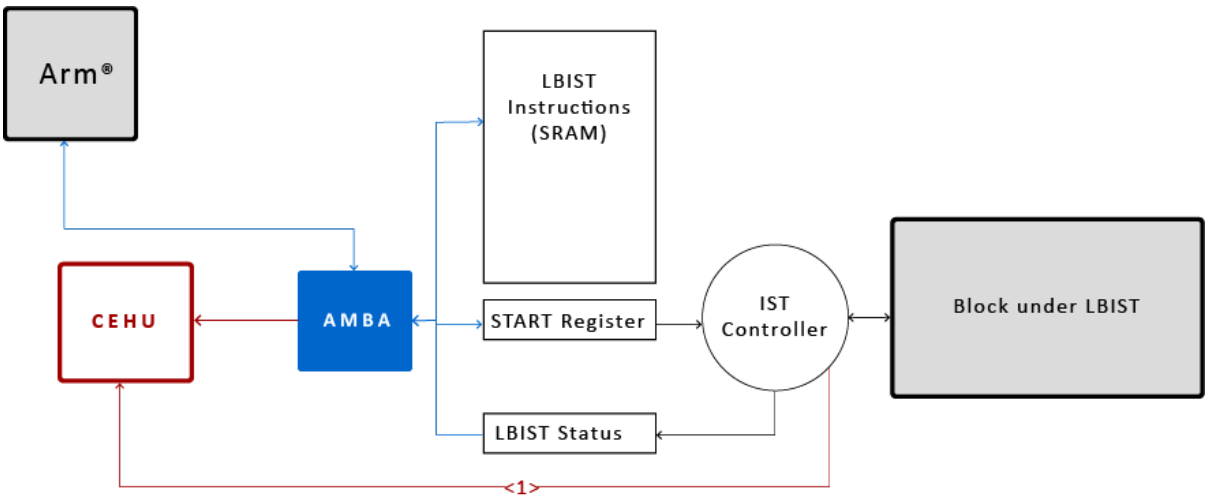


Figure 4 Major Components of a Block Under LBIST

5.1.18 Delayed Lockstep

Name	*Description*

ID	SM_HW_DCLS
Description	Functional logic is duplicated and operated in delayed lockstep mode. A mismatch indicates an error. Memories are protected via another safety measure and are excluded from duplication. Only associated read / write control and parity logic is duplicated. Synchronizers are protected via another safety measure and are excluded from duplication.
Ownership	Ambarella
SM is to against	Any random hardware fault
Software Related	No
Failure mode coverage	99%
Initialization	N/A
Operation	N/A
Error indication	Assert safety error signal
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_LBIST (if applicable)

5.1.19 Delayed Hashed Lockstep

Name	*Description*
ID	SM_HW_HASH_DCLS
Description	Delayed hashed lockstep is implemented to detect any random hardware fault in datapath logics in a lockstep module without fully duplicating the datapath. - For use in blocks that move data around without changing the data itself. - Each byte of incoming data is converted to a 2-bit

	hash value for the duplicate datapath to transfer. - Output data from the main is hashed and compared with the hash values from the duplicate datapath.
Ownership	Ambarella
SM is to against	Any random hardware fault
Software Related	No
Failure mode coverage	99%
Initialization	N/A
Operation	N/A
Error indication	Assert safety error signal
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_LBIST

5.1.20 Modulo3

Name	*Description*
ID	SM_HW_MOD3
Description	For the Modulo3 safety mechanism, the modulo 3 value is added as a local checksum for a data work. It can be seen as an extension of parity where not 1 but 2 extra bits are added to the data bits. To check if the data bits are correct, the modulo3 value of the data bits is calculated and then compared against the stored modulo3 value. The advantage of modulo3 is that it can be used for end-to-end protection of combinational logic if that logic is of arithmetic nature, since addition, subtraction, and multiplication are compatible with the congruence relation of integers that is introduced by the modulo N classification.
Ownership	Ambarella

SM is to against	- permanent fault or transient fault in memory bit-cell, latch, and flops - permanent fault on combinational logic via end-to-end protection
Software Related	No
Failure mode coverage	99% - detect permanent and transient fault in memory bit-cell, latch, and flops 90% - detect permanent faults on the combinational logic via end-to-end protection
Initialization	N/A
Operation	N/A
Error indication	Assert safety error signal
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_LBIST

5.1.21 Test DAG

Name	*Description*
ID	SM_HW_TESTDAG
Description	In the safety measure of test DAG, a checksum calculator is implemented for VP function. During DAG execution, for all output data that is stored to DCB and VMB buffers via XMEM, a 32-bit checksum is accumulated for each DCB and VMB separately. At the end of the test DAG, a collective 32-bit checksum is calculated and made available for comparison against a golden checksum value by SW. System Integrator is responsible for implementing the safety application to configure and enable the test DAG execution with the silicon support from Ambarella.
Ownership	Ambarella and System integrator
SM is to against	Majority of permanent errors in the combinational logic

Software Related	Yes
Failure mode coverage	60% - A test DAG safety mechanism can be used to detect majority of permanent errors in the combinational logic of VP-Top, XMEM, VMEM, and PUs that do not use lockstep operation as a safety measure. 90% - A test DAG safety mechanism can also be used to detect any random hardware fault in SECDED ECC engine that causes PVSG faults by actively corrupting SRAM data.
Initialization	N/A
Operation	SW runs a pre-packaged test DAG. The checksum registers for DCBs and VMBs are calculated during the DAG execution and after the run completes, the collective checksum can be read by peeking at the global checksum register. The collective checksum should be compared against the golden value by SW, and in case of mismatch a safety error shall be asserted by SW.
Error indication	Assert safety error signal by software
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_LBIST
SDK Usage	The CVflow VP (Vector Processor) engine includes a "Test DAG" diagnostic function provided via the SDK. A test DAG is a pre-packaged computational graph (Directed Acyclic Graph) specifically designed for online hardware latent fault detection within the VP data path. During test DAG execution, a 32-bit checksum is accumulated internally for each DCB (Data Control Block) and VMB (Vector Memory Block) buffer written to XMEM. The checksum comparison against the golden value is performed internally by the Ambarella SDK and hardware modules — the system integrator does not need to read or compare checksum registers directly. Faults are surfaced via DSP driver error callbacks or CEHU error signals. **Detected Error Types:** **VP Data Path Fault:** A mismatch between the accumulated DCB/VMB checksum and the golden checksum, indicating a permanent fault in the CVflow VP processing logic. Reported to the system integrator via DSP driver error notification or CEHU interrupt. **Checksum Mechanism Fault:** Failure in the internal checksum accumulation or comparison infrastructure, resulting in an error flag from the SDK or hardware.

	Assumptions of Use **1. Diagnostic Frequency:** The System Integrator shall configure the test DAG execution period such that it runs within the FTTI for latent faults in the VP data path. The SDK provides the API to set this period. **2. Error Monitoring:** The System Integrator shall register an error callback with the DSP driver and/or configure the CEHU interrupt handler to receive fault notifications from the test DAG mechanism. No manual checksum reading or comparison is required. **3. Safe State Response:** Upon receiving a test DAG error notification (from the DSP driver callback or CEHU interrupt), the safety application shall execute the configured safe state reaction within the FTTI. **4. Scope:** The test DAG covers the VP data path (DCB/VMB outputs via XMEM). For IDSP/DSP pipeline coverage use SM_HW_TESTFRAME. Both mechanisms should be used together for complete DSP+VP coverage. **SDK API and integration flow:** Trigger the test DAG via API AmbaCV_SchdrAddCvtable() with the prebuilt tbar provided by Ambarella (or the "cv flexidag schdr start ..." command). *Note: The reference code / sample code is provided to demonstrate usage only and shall not be used as production code.*

5.1.22 Hardware Watchdog Timer on DRAM

Name	*Description*
ID	SM_HW_DRAM_WDT
Description	Several hardware watchdog timers are implemented to detect hang of specific functional blocks in the DRAM sub-system. System Integrator is responsible for implementing the safety software to configure the related settings.
Ownership	Ambarella and System Integrator
SM is to against	Hang of specific functional blocks in DRAM sub-system.
Software Related	Yes
Failure mode coverage	99%

Initialization	Program WATCH DOG_TIMER_0_CONFIG and WATCH DOG_TIMER_1_CONFIG registers with number of cycles to expire/16
Operation	N/A
Error indication	Assert safety error signal
Error detect time	3 ms
Measure to prevent latent fault	SM_HW_LBIST (if applicable)

5.1.23 Hamming ECC

Name	*Description*
ID	SM_HW_HECC
Description	– SM_HW_HECC is implemented with Hamming (298, 16). - Each codeword(32B) is protected by a single error correction and dual error detection (SECDED) - Each codeword consists of a 32B message, 26-bit physical address, and 16-bit parities. - The message part is stored in main DDR device and the parity part is stored in ECC-DDR device. System Integrator is responsible for implementing the software to enable the hardware ECC function in start-up flow.
Ownership	Ambarella and System Integrator
SM is to against	any random hardware fault of external DDR device, and datapath from ECC engine to the interfaces with DDR devices.
Software Related	Yes
Failure mode coverage	100% - for single-bit error 100% - for double-bit error 98% - for multiple-bit error with less that 2% false correction probability
Initialization	Software shall enable ECC enabled bit in REG_ECCH_MODE.

Operation	N/A
Error indication	Assert safety error signal on uncorrectable error detection, increment MAIN_CORRECTABLE_COUNTER or ECC_CORRECTABLE_COUNTER on a single bit error correction
Error detect time	3 ms
Measure to prevent latent fault	SM_HW_LBIST

5.1.24 AHB Rule Checker

Name	*Description*
ID	SM_HW_AHB_CHECKER
Description	SM_HW_AHB_CHECKER consists of the following two rule checkers: – The address provided by the client in each beat must be aligned to the beat size. In particular, (hsize == 1) -> (haddr[0] == 0), and (hsize == 2) -> (haddr[1:0] == 0) – Early burst termination is not allowed. For example, the client can't start a INCR8 write transaction and transfer just 1 beat of write data before starting another completely different transaction. A similar read example would be a INCR8 read transaction that accepts the first data beat, but provides a completely different transaction (combination of haddr / htrans / hburst / hsize) while accepting the beat.
Ownership	Ambarella
SM is to against	illegal activities from QM clients
Software Related	No
Failure mode coverage	99%
Initialization	N/A

Operation	N/A
Error indication	Assert safety error signal
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_LBIST

5.1.25 SEGMENTATION

Name	*Description*
ID	SM_HW_DRAM_SEGMENTATION
Description	SM_HW_DRAM_SEGMENTATION is implemented to detect a DRAM client to access illegal DRAM region and a specific client to access secure region. System Integrator is responsible for analyzing the DRAM region/clients configuration and implementing the software to configure the DRAM segmentations settings to avoid interference from QM clients.
Ownership	Ambarella and System integrator
SM is to against	Illegal activities from QM clients
Software Related	Yes
Failure mode coverage	99%
Initialization	The unit of segment is 256 KB. Software shall ensure a minimum 256 KB non-overlapped memory region allocated for each DRAM client.
Operation	N/A
Error indication	Assert corresponding bit in REG_DRAM_SEG_FAULT and an interrupt. Software determines to go to safe state if needed.
Error detect time	99 ms

Measure to prevent latent fault	SM_HW_LBIST
SDK Usage	The DRAM Segmentation mechanism implements hardware-enforced memory region protection within the DRAM Controller. It detects unauthorized access when a DRAM client attempts to read from or write to a memory region outside its permitted boundaries, or when a Quality Management (QM) client attempts to access a safety-critical or secure memory region. This mechanism ensures "Freedom from Interference" (FFI) between safety and non-safety software components sharing the DRAM subsystem. **Detected Error Types:** **Illegal Region Access:** A DRAM client accesses a memory address outside its configured permitted range. **Secure Region Violation:** A non-privileged or QM client attempts to access a memory segment designated as safety-critical or secure. **Misconfigured Boundary Fault:** A software error causes a client to address memory beyond its allocated segment, indicating a potential stack overflow, pointer corruption, or DMA misconfiguration. **Inter-partition Interference:** Unauthorized access from a lower-integrity partition into a higher-integrity memory zone, violating FFI requirements. **Assumptions of Use** 1. System Memory Map Analysis The System Integrator shall define and document the complete DRAM memory map for the target system, explicitly identifying all memory regions allocated to safety-critical (ASIL) and non-safety (QM) software components. This analysis shall serve as the input for the segmentation configuration. 2. SDK-Based Segmentation Configuration The System Integrator shall implement the DRAM segmentation configuration using the provided SDK reference code (AmbaUserMemProt.c / AmbaMemProt.c). The entry function AmbaMemProt_Config shall be invoked during the safety application startup (as demonstrated in SvcSysEntry.c) to enforce memory boundaries before any application-level data processing begins. 3. QM Client Isolation The System Integrator shall configure segmentation rules to explicitly prevent all QM DRAM clients from accessing safety-critical memory regions. The segmentation boundaries must be aligned to the physical memory allocation of all active hardware accelerators and processors. 4. CEHU Integration and Error Handling Segmentation violations are reported to the CEHU. The System Integrator shall implement a fault handler on the Cortex-R52 to receive, log, and respond to segmentation faults. The handler shall treat any boundary violation as a critical safety fault requiring immediate transition to a Safe State.

	5. Validation at Integration The System Integrator shall verify the correctness of the segmentation configuration through integration testing, confirming that all safety-critical regions are protected and that QM clients cannot access them under any foreseeable operating condition. Note: Per the SDK architecture, segmentation fault handling must be implemented on the Cortex-R52. The CEHU interrupt for segmentation violations is routed to R52. System Integrators who implement the fault handler on A53 will not receive CEHU interrupts for this error source. **SDK API and integration flow:** The entry API is AmbaMemProt_Config (reference flow in AmbaUserMemProt.c / AmbaMemProt.c), called from svc/apps/isafety/SvcSysEntry.c. *Note: The reference code / sample code is provided to demonstrate usage only and shall not be used as production code.*

5.1.26 DRAM Bandwidth Monitor

Name	*Description*
ID	SM_HW_DRAM_BW_MON
Description	The statistic counters are implemented to monitor the traffic of DRAM clients. System Integrator is responsible for implementing the safety software to enable and read out the DRAM statistic counter for the safety application usage.
Ownership	Ambarella and System Integrator
SM is to against	Illegal activities from QM clients
Software Related	Yes
Failure mode coverage	90%
Initialization	Software shall enable STAT_CONTROL bit.
Operation	Software shall disable STAT_CONTROL bit, check the statistic counters, and then re-enable STAT_CONTROL bit every 99 ms.

Error indication	Assert safety error signal by software
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_LBIST
SDK Usage	The DRAM Controller (DRAMC) incorporates hardware-based statistic counters designed to monitor the data traffic of individual DRAM clients. This safety mechanism allows for the detection of "Freedom from Interference" (FFI) violations and hardware-level communication failures within the memory subsystem. **Detected Error Types:** **Memory Bandwidth Overrun:** A non-safety-critical client (e.g., A53) consuming excessive bandwidth, potentially starving safety-critical consumers (e.g., R52 or DSP). **Client Communication Stalling:** Detection of zero or near-zero traffic for a specific active client, indicating a hang in the bus interface or the client's memory port. **Abnormal Traffic Patterns:** Significant deviations from the expected memory access profile, which may indicate software-level malfunctions or unintended DMA (Direct Memory Access) transfers. **Resource Contention:** Indirect detection of interconnect congestion that impacts the determinism of safety-critical tasks. **Assumptions of Use** 1. Software Implementation of Traffic Checker The System Integrator shall implement a safety-related software routine (typically hosted on the Cortex-R52) to periodically poll the DRAM statistic counters. The integrator must utilize the provided SDK API AmbaDRAMC_GetStatisInfo (as demonstrated in the AmbaDiag_Dram.c sample code) to retrieve real-time traffic data for all safety-relevant memory ports. 2. Baseline Analysis and Threshold Definition The System Integrator is responsible for performing a worst-case timing and memory traffic analysis for the specific system configuration. Based on this analysis, the integrator shall define: Upper Thresholds: Maximum allowable bandwidth per client to prevent interference. Lower Thresholds: Minimum expected activity for clients that must remain "alive." These thresholds must be integrated into the safety application logic to trigger an error state. 3. Safety-Critical Interference Management The System Integrator shall use the gathered statistics to verify that non-safety-related software (running on A53 or other accelerators) does not interfere with the safety-critical memory path. If a violation is detected,

	the safety application on the R52 must take priority action to preserve the system's Safety Goal. 4. Error Handling and System Reaction Because these counters are read-only monitors, the System Integrator must define the software-led reaction upon a threshold violation. This may include: Invalidating data outputs that may have been compromised by memory latency. Throttling or resetting the offending DRAM client. Reporting a "Memory Subsystem Fault" to the system-level safety manager to initiate a transition to a Safe State. 5. Monitoring Frequency and FTTI The System Integrator must ensure that the polling interval for the statistic counters is fast enough to detect a bandwidth violation and execute a safety response within the Fault Tolerant Time Interval (FTTI). The integrator should refer to the AmbaDiag_DramShowStatisInfo sample application to ensure correct implementation of the read-out and data processing sequence. Note on Polling Interval: Per the HW specification, the polling cycle must complete every 99 ms — the integrator shall disable STAT_CONTROL, read all statistic counters, compare against expected thresholds, re-enable STAT_CONTROL, and finish this entire sequence within 99 ms to avoid missing a monitoring window. **SDK API and integration flow:** Read DRAM traffic statistics via AmbaDRAMC_GetStatisInfo() (in AmbaDiag_Dram.c); reference application AmbaDiag_DramShowStatisInfo. *Note: The reference code / sample code is provided to demonstrate usage only and shall not be used as production code.*

5.1.27 Address Stuck Test

Name	*Description*
ID	SM_HW_ASTUCK_TEST
Description	– Address stuck test is to periodically measure any stuck-at faults on DDRIO pins, including DDR-IO command, address and data interfaces, when using LPDDR4 DRAM memory. SM_HW_ASTUCK_TEST will write the pre-defined pattern bits to Mode Register 15 and 20 on an external DDR device, read out the Multi-Purpose-Command Read data from it and compare the read data with the expected patterns. Any mismatch on this read data will assert safety error. – The SM_HW_ASTUCK_TEST shall be run at least once every DTTI. – System Integrator is responsible for implementing the safety software

	to configure the ASTUCK engine test to meet the system safety requirement.
Ownership	Ambarella and System Integrator
SM is to against	Stuck-at fault on DDR-IO command, address, and data interfaces
Software Related	Yes
Failure mode coverage	90%
Initialization	At DDRC, DRAM_TYPE has to be 6 (LPDDR4). At DTTE, Configure DTTE_DUAL_DIE, DTTE_DUAL_CHANNEL, DTTE_MR15, DTTE_MR20, DTTE_EXPECTED_PATTERN, DTTE_ASTUCK_PERIOD and DTTE_ASUTCK_PERIOD_MSB, and at last set DTTE_ASTUCK_TEST_EN to 1 to enable SM_HW_ASTUCK_TEST. For more information about configuration, please refer to the DDRC/DTTE Programming Reference Manual.
Operation	N/A
Error indication	HW asserts safety error signal and sets DTTE_ERR_VLD and DTTE_ASTUCK_ERR_VLD in DTTE Software Programmable Registers.
Error detect time	99 ms
Measure to prevent latent fault	None
SDK Usage	The Address Stuck Test (SM_HW_ASTUCK_TEST) is a periodic online diagnostic mechanism that detects stuck-at faults on LPDDR4 DDRIO interface pins, including command, address, and data lines. The test writes a predefined bit pattern to Mode Register 15 (MR15) and Mode Register 20 (MR20) of the external DDR device, then reads back the Multi-Purpose Command (MPC) result and compares it against the expected pattern. Any mismatch indicates a permanent stuck-at fault on the DDR IO interface. **Detected Error Types:** **Address Pin Stuck-at Fault:** One or more address or command pins are permanently fixed at logic high or logic low, preventing correct memory addressing. **Data IO Stuck-at Fault:** DDRIO data pins that are permanently stuck at a fixed logic level, causing systematic data corruption.

	Mode Register Read-Back Mismatch: Discrepancy between the written MR pattern and the MPC read-back result, indicating an IO integrity failure. **Interface Degradation:** Partial or intermittent pin failures that result in unreliable DRAM communication under operational conditions. **Assumptions of Use** 1. Periodic Execution Within DTTI The System Integrator shall schedule the SM_HW_ASTUCK_TEST to execute at least once per Diagnostic Test Time Interval (DTTI) as required by the safety analysis. The scheduling must account for the test's impact on DRAM availability and system performance. 2. SDK API Integration The System Integrator shall utilize the provided SDK APIs and reference code to configure and trigger the ASTUCK engine test. The implementation must follow the validated reference sequence for MR15/MR20 write and MPC command read-back. 3. Performance Impact Management The System Integrator shall analyze and quantify the DRAM performance impact of running the periodic ASTUCK test. The test scheduling must ensure that execution does not violate the worst-case memory access latency requirements of safety-critical DRAM clients. 4. Error Handling and Safe State Upon detection of a pattern mismatch, the safety application on the Cortex-R52 shall treat the result as a permanent hardware fault. The system shall invalidate any data transacted over the affected DRAM interface and transition to a Safe State within the FTTI. **SDK API and integration flow:** The Address Stuck test is configured and enabled by the bootloader (BootloaderSH) as part of the DRAM safety functions; a detected stuck-at fault is reported to the CEHU. The user application handles the fault through the CEHU error-handling flow (see AmbaCEHU_ErrorGet). *Note: The reference code / sample code is provided to demonstrate usage only and shall not be used as production code.*

5.1.28 DRAM ECC Corrected-Error Counter

Name	*Description*
ID	SM_HW_DRAM_ECC_CORR_CNT
Description	- In the SEooC, external DRAM is SECDED ECC protected. - The ECC has: - a counter keeping track of how many single bit errors have been corrected. For every cycle, this counter only counts up to one corrected error;

	- a configurable threshold indicating at what count the monitor logic will report an error. That is, if the corrected count exceeds the threshold, an error will be reported to CEHU. This counter is 16-bits. The threshold's default value is its maximum value (all 1s). SW can read current corrected error count via debug port. - a mechanism that would prevent an error from being sent to CEHU for corrected single-bit errors, regardless of the count. This is achieved by a disable / mask bit. System Integrator is responsible for implementing the safety software to enable the DRAM ECC function and setup the proper threshold for the safety application usage.
Ownership	Ambarella and System Integrator
SM is to against	single or dual bit(s) permanent fault in memory data array
Software Related	Yes
Failure mode coverage	99%
Initialization	Software shall set the threshold on ERROR_STATUS_REG_2/3_Threshold, which 2 for Main-DDR and 3 for ECC-DDR
Operation	N/A
Error indication	Assert safety error signal on exceeding the threshold
Error detect time	99 ms
Measure to prevent latent fault	N/A (this safety mechanism is to detect latent fault)
SDK Usage	The external DRAM is protected by Single-Error-Correction Double-Error-Detection (SECDED) ECC logic within the DRAM Controller. This safety mechanism provides a hardware-based correctable single-bit error (SBE) counter with a software-configurable threshold. If the number of corrected single-bit errors exceeds the programmed threshold within a monitoring window, a safety error signal is asserted, allowing software to distinguish between rare transient upsets and the onset of a permanent DRAM fault. **Detected Error Types:** **Correctable Single-Bit Error (SBE) Accumulation:** A pattern of recurring single-bit corrections that exceeds the configured

	threshold, indicating DRAM degradation or a developing permanent fault. **Threshold Violation:** The SBE counter saturates or exceeds the programmed limit, triggering a CEHU error to signal that the DRAM reliability margin has been exhausted. **Uncorrectable Double-Bit Error (DBE):** Detection of two simultaneous bit errors within an ECC word, which cannot be corrected and represents an immediate data integrity failure (reported separately by the ECC hardware). **Assumptions of Use** 1. ECC Enable and Threshold Configuration The System Integrator shall enable the DRAM ECC function during system initialization and configure the correctable error threshold to a value appropriate for the system's safety analysis. The threshold shall be set conservatively enough to detect DRAM degradation before it results in an uncorrectable error, but high enough to avoid nuisance trips from isolated transient events. 2. SDK API Usage The System Integrator shall utilize the provided APIs (AmbaDRAMC_GetEccCount() and AmbaDRAMC_ResetEccCount()) to periodically read and reset the ECC error counter as part of the safety monitoring routine. The polling interval must be shorter than the DTTI. 3. Baseline Characterization The System Integrator should characterize the expected SBE rate for the target DRAM device under normal operating conditions (temperature, voltage, aging) to establish a realistic threshold. The threshold configuration should be made accessible via menuconfig with the maximum value (all 1s) as the default. 4. Error Handling Upon a threshold violation notified through CEHU, the safety application on the R52 shall log the event, attempt a memory scrubbing cycle if supported, and if faults persist, transition the system to a Safe State to prevent a future uncorrectable error from causing a safety violation. **SDK API and integration flow:** Read and reset the DRAM ECC correctable-error count via AmbaDRAMC_GetEccCount() and AmbaDRAMC_ResetEccCount(); the count threshold before a CEHU error is triggered is configurable.

5.1.29 Receive Panel Error Status

Name	*Description*
ID	SM_HW_ERPT

Description	– In error status report safety mechanism - [AR-12] is presumed valid. See 5.3.12 . - An analog low power receiver and a digital block for decoding and storing status bits are implemented. - VOUT will request MIPI DSI to send a DCS read/write command to external panel, followed by a BTA request. MIPI DSI shall receive panel status bits and store them onto readable registers in RCT. - VOUT shall dictate the timing of BTA request and SW shall decode the received status, including ECC/CRC error, protocol violation error, and BTA timeout error, to take proper actions. Since BTA procedure is done via MIPI LP signaling, it's recommended to issue BTA during vertical blanking period. System Integrator is responsible for the safety software implementation to use the BTA procedure to meet the system safety requirement.
Ownership	Ambarella and System Integrator
SM is to against	Any errors in transmitted packets and protocol violation
Software Related	Yes
Failure mode coverage	99%
Initialization	Link layer should stop high-speed data transmitting and wait for MIPI DSI to be under Control mode (STOP state).
Operation	1. Under control mode link layer shall start Escape Mode request 2. Link layer write necessary configuration code through Escape Mode to notify the panel to prepare the data. 3. Link Layer trigger BTA request, and wait DSI PHY LPRX done. 4. Configure rct_dsi_obsv_sel[2:0] properly to read out received panel values on rct. 5. Readout information (A and B are for DSI0 and DSI1) a. Set MIPI_DSI_A(B)_AUX0_REG[26:24]=5 i. MIPI_DSI_OBSV0_A(B)_REG[31:0]=rx_lp_out0[31:0] ii. MIPI_DSI_OBSV1_A(B)_REG[31:0]=rx_lp_out1[31:0] b. Set MIPI_DSI_A(B)_AUX0_REG[26:24]=6 i. MIPI_DSI_OBSV0_A(B)_REG[31:0]=rx_lp_out2[31:0] c. {rx_lp_out2,1,0} are the received bytes from panels, where rx_lp_out0 received first, then 1 and then 2. 6. Refer to panel's datasheet for the detailed operation address and coding style. One example can be found in ILI9488 datasheet (section 4.6.3.2.1.5, Table 32 and Table 33 of ILI9488_IDT_V100_20121128).

	7. Take proper action, such as assert safety error signal, based on the received panel status.
Error indication	Assert safety error signal by software
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_DCLS

5.1.30 Voltage Detector

Name	*Description*
ID	SM_HW_BG_VDECT
Description	Voltage detectors, including over-voltage detection and under-voltage detection, will be used to make sure that the voltage across the external resistor is between 1.5 V and 0.9 V, thus ensuring the output current is between 18.75 uA and 11.25 uA. If this voltage is out of range, BGERR will be asserted. Output bias current (15 uA) is mirrored internally and flows into an 80 KOhm resistor. Another set of voltage detectors are used to compare it against 1.65 V and 0.6 V. If this voltage is out of range, it means that the external resistor fails and BGERR will be asserted.
Ownership	Ambarella
SM is to against	Output current of BiasGen out-of-range
Software Related	No
Failure mode coverage	90%
Initialization	N/A

Operation	N/A
Error indication	Assert safety error signal
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_BG_ABIST and SM_HW_DCLS

5.1.31 Voltage Detector Self-Checking BIST

Name	*Description*
ID	SM_HW_BG_ABIST
Description	– During analog BIST, voltage detectors will be tested by internal voltage references to make sure it is working correctly – Analog BIST runs once after power reset – The self-test can make sure that voltage detector can cover a range of 0.9 V to 1.5 V, which is the same coverage range required under normal function.
Ownership	Ambarella
SM is to against	Faults in voltage detector
Software Related	No
Failure mode coverage	90%
Initialization	NA
Operation	N/A
Error indication	Assert safety error signal
Error detect time	Power-on test

Measure to prevent latent fault	None (this safety mechanism is to detect latent fault)

5.1.32 AHB Decoder Master ID Checking Logic

Name	*Description*
ID	SM_HW_BM_ID_CHK
Description	In SM_HW_BM_ID_CHK safety mechanism, a master ID checking logic is implemented in the AHB decoder to check bus master of the transaction. The System Integrator is responsible for implementing the safety mechanism software by configuring the master ID checking logic.
Ownership	Ambarella and System Integrator
SM is to against	Fault on address multiplexing path from NIC-400 to AHB address decoding logic
Software Related	Yes
Failure mode coverage	80%
Initialization	Software sets the registers in AXI configuration space for each AHB / APB client to determine if this client could be accessed by both Cortex-R52 and Cortex-A53 or by Cortex-R52 only.
Operation	- When master ID checking logic detects a transaction to the client from Cortex-A53 is not permitted, it will issue an error flag to AHB configuration space register, and AHB slave will issue ERROR response. - When software receive ERROR response, it will stop current execution and enter exception handler; software shall read AHB configuration space register to check which client is accessed unexpected.
Error indication	Assert safety error signal by software
Error detect time	99 ms

Measure to prevent latent fault	None
SDK Usage	The AHB Decoder Master ID Checking Logic enforces bus-level access control by validating the master ID of every transaction on the AHB bus. This mechanism prevents unauthorized hardware masters (e.g., a misconfigured DMA engine or a compromised QM subsystem) from accessing memory-mapped registers or peripherals that are restricted to safety-critical masters, ensuring freedom from interference at the bus interconnect level. **Detected Error Types:** **Unauthorized Master Access:** A bus transaction from a master whose ID is not permitted to access the target address region, indicating either a software misconfiguration or a hardware fault causing incorrect master ID propagation. **Illegal Peripheral Access:** A QM or lower-privilege master attempting to read or write safety-critical peripheral registers reserved for the safety processor (R52) or safety software. **Bus Routing Fault:** A hardware-level error causing a transaction to be attributed to an incorrect master ID, indicating a fault in the interconnect fabric. **Assumptions of Use** 1. Master ID Privilege Analysis The System Integrator shall define the complete AHB master access policy for the target system, identifying which masters are permitted to access each peripheral or memory-mapped region. This analysis shall cover both safety-critical (ASIL) and QM masters. 2. SDK-Based Configuration The System Integrator shall configure the AHB decoder master ID checking logic using the provided API (AmbaRTSL_AxiSetClientAccess()), following the reference implementation in the bootloader. The configuration must be applied before any application-level bus traffic is initiated. 3. Policy Validation The System Integrator shall validate the configured access policy through integration testing, verifying that all illegal access attempts are correctly detected and reported, and that all legitimate transactions proceed without false positives. 4. CEHU Integration and Error Response AHB master ID violations shall be reported to the CEHU. The System Integrator shall implement an error handler on the R52 to capture violation events, identify the offending master, and initiate a Safe State transition if the violation indicates a safety-critical interference condition. Note: When the AHB slave returns an ERROR response, the Cortex-A53 enters an exception handler. The System Integrator shall implement this handler to assert a safety error signal, ensuring the violation is escalated to the hardware safe state reaction framework and not silently ignored.

	SDK API and integration flow: Configure the AHB decoder master-ID access in the bootloader stage via AmbaRTSL_AxiSetClientAccess() (example; the integrator may adjust for their configuration).

5.1.33 Fault Injection Logic

Name	*Description*
ID	SM_HW_FAULT_INJ
Description	In this safety measure, fault injection logic is implemented in every error signal which is collected in AMBA ECRU except some of the errors from Cortex-A53 and Cortex-R52 which can't be injected. This safety mechanism is to detect latent fault in AMBA ECRU. The System Integrator is responsible for the safety mechanism software implementation which triggers the fault injection.
Ownership	Ambarella and System Integrator
SM is to against	Latent fault in AMBA ECRU
Software Related	Yes
Failure mode coverage	70%
Initialization	N/A
Operation	In startup test, SW shall set safety_mode in CEHU to disable safety pin. Then, SW shall inject faults by programming error_injection registers in each client of AMBA ECRU. Finally, SW shall check the status of CEHU to see if the injected fault is detected.
Error indication	Keep in safe state
Error detect time	Power-on test

Measure to prevent latent fault	None (this safety mechanism is to detect latent fault)

5.1.34 CAN RX Node Detect Error Status

Name	*Description*
ID	SM_HW_CAN_RE
Description	– In error status report safety mechanism - There are five types of error(s) in CAN bus protocol: - Bit error: The bit value that is monitored differs from the bit value sent (detected at TX) - Stuff error: detects the sixth consecutive equal bit in a frame. (detected at TX and RX) - CRC error: calculated CRC sequence does not equal the received one (detected at RX) - Form error: Fixed-form bit contain one or more illegal bits (detected at TX and RX) - ACK error: TX node does not receive dominant (0) at ack field (detected at TX) - When error is detected, the TX or RX node will send error flag (consecutive six dominant bits) on bus, and all nodes will detect stuff error and then send error flag again and then sync to same idle state. - When any random hardware fault corrupts the TX packet at the path between CRC encoder and CAN_TX pad, RX node will detect error because of protocol violation. So any fault in the TX path of MBUF and Protocol engine can be detected. - In CAN bus protocol, hardware can detect bit error, form error and ACK error, so any fault from RX external pin to RX path in Protocol engine can be detected. - There are TX/RX error counters recording frequency of TX and RX errors. When error number reach some threshold, error state (ACTIVE, PASSIVE, BUS-OFF) will be changed and assert interrupt. Software shall detect error by checking the interrupt status. - When any “stuck at” faults corrupt the encoded frame, the error counter will increase all the time, and CAN node will enter BUS-OFF state. - Then SW can determine if safety error in CEHU should be asserted to enter safe state. System Integrator is responsible for the safety mechanism software implementation to detect the reported error from CAN controller.
Ownership[	Ambarella and System Integrator

SM is to against	Any errors in transmitted packets and protocol violation
Software Related	Yes
Failure mode coverage	99%
Initialization	N/A
Operation	SW check interrupt status to see if there is error. HW will send error frame when error is detected.
Error indication	Assert safety error signal by software
Error detect time	99 ms
Measure to prevent latent fault	See 5.3.11
SDK Usage	The CAN RX Node Detect Error Status mechanism leverages the CAN bus protocol's built-in error detection to identify hardware faults and signal integrity violations in real time. The CAN controller monitors each transmitted and received frame against five protocol-defined error rules. When any rule violation is detected, the controller asserts an interrupt, increments the TX or RX error counter, and—if the error persists—escalates to the BUS-OFF state. The safety application can then assert a safety error signal to trigger the configured safe state reaction. **Detected Error Types:** **Bit Error:** The bit value monitored on the bus differs from the bit value sent. Detected at the TX node, indicating a fault in the TX output path between the CRC encoder and the CAN_TX pad. **Stuff Error:** Six or more consecutive bits of equal polarity are detected, violating the bit-stuffing rule. Detected at both TX and RX nodes; all receiving nodes simultaneously flag the error and resynchronise. **CRC Error:** The CRC sequence calculated by the receiver does not match the CRC appended by the transmitter. Detected at the RX node, indicating a fault corrupting the data payload in transit. **Form Error:** A fixed-form field (e.g., end-of-frame, interframe space) contains one or more illegal bits. Detected at both TX and RX nodes, indicating a protocol-level encoding failure. **ACK Error:** The transmitter does not receive a dominant bit during the ACK slot, meaning no receiver acknowledged the frame. Detected at the TX node, indicating that no node on the bus received the frame correctly.

	Bus-Off Condition: The TX error counter exceeds 255, causing the CAN node to enter the BUS-OFF state and cease all bus activity. Indicates a persistent or repetitive hardware fault in the TX path. **Assumptions of Use** 1. CAN Error Interrupt Handler The System Integrator shall implement an interrupt service routine (ISR) triggered by the CAN error interrupt. The ISR shall read the CAN controller status registers to identify the error type and the affected controller (CAN0–CAN5), then invoke the AmbaRTSL_CAN.c error notification path to report the event to the safety layer. 2. ECRU Status Mapping **Upon receiving a CEHU amba_safety error notification, the System Integrator shall read the ECRU status registers to identify the error source. The CAN controller error bits are located at the following positions in error_log_1:** CAN0 = bit 7, CAN1 = bit 8, CAN2 = bit 9, CAN3 = bit 10, CAN4 = bit 11, CAN5 = bit 12. 3. Error Counter Monitoring and Escalation Policy The System Integrator shall define threshold policies for the TX and RX error counters. When a counter reaches the PASSIVE threshold (128) or the BUS-OFF threshold (256), the safety application shall treat the event as a persistent fault and evaluate whether a Safe State transition is required within the FTTI. 4. CEHU Integration and Safe State Response The System Integrator shall configure the CEHU to map CAN error signals to the appropriate error reaction for the target application. Upon detecting a BUS-OFF condition or a pattern of repeated errors exceeding the defined threshold, the safety application shall assert a safety error signal and execute the user-defined Safe State procedure (e.g., deactivate the CAN-dependent safety function, alert the vehicle safety manager, or initiate a controlled system reset). **SDK API and integration flow:** To receive the notification, register a CEHU interrupt callback with AmbaCEHU_IntCallbackRegister(AMBA_CEHU_INT_CALLBACK_f pCallBackFunc). When a CEHU amba_safety error occurs, the user application reads the error status via AmbaCEHU_ErrorGet(UINT32 InstanceID, UINT32 ErrorID, UINT32 *Value) to determine the source: each CAN controller (CAN0-CAN5) maps to a specific CEHU error ID, and a returned Value of 1 indicates that controller has reported an error. The user application then executes its user-defined recovery procedure for the identified controller. *Note: The reference code / sample code is provided to demonstrate usage only and shall not be used as production code.*

5.1.35 VOUT Checksum

Name	*Description*

ID	SM_HW_VOUT_CS
Description	VOUT calculates checksum on input stream of display section every frame, and stores the checksum in a software readable register. Software will decide when to read the checksum and compares the number between frames to detect the frozen frame. System Integrator is responsible for implementing the safety mechanism software to read out the checksum and frozen frame comparison.
Ownership	Ambarella and System Integrator
SM is to against	Any random hardware fault that causes output of frozen images
Software Related	Yes
Failure mode coverage	90%
Initialization	N/A
Operation	SW can check VOUTD_luma / chroma_stream_checksum registers to detect frozen image.
Error indication	Assert safety error signal by software
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_LBIST
SDK Usage	The VOUT Checksum mechanism provides a frame-level data integrity monitor for the video output pipeline. The VOUT hardware calculates a checksum over the input stream of each display frame and stores the result in a software-readable register. By comparing the checksum values between consecutive frames, the safety application can detect "frozen frame" conditions — a safety-critical failure mode where the display is powered on but is presenting a static, stale image instead of live camera or sensor data. **Detected Error Types:** **Frozen Frame:** Two or more consecutive frames produce an identical checksum, indicating that the display pipeline is outputting a static image and the live video feed has stalled or been interrupted.

	Display Pipeline Stall: A hardware or software failure in the VOUT subsystem that causes the frame buffer to stop updating while the display controller continues outputting data. **Silent Video Source Failure:** The upstream video source (e.g., ISP or DSP output) has halted, but the display controller continues rendering the last valid frame without error indication. **Assumptions of Use** 1. Software Implementation of Frozen Frame Detection The System Integrator shall implement a safety-critical software routine that periodically invokes AmbaVout_GetDisp0Status() and/or AmbaVout_GetDisp1Status() to retrieve the per-frame checksum and the pFreezeDetect output. This routine shall be executed at the system's defined diagnostic monitoring rate. 2. Comparison Strategy and Threshold The System Integrator shall define the number of consecutive matching checksums that constitutes a confirmed frozen frame event. A single matching frame may indicate a brief stall; multiple consecutive matches indicate a persistent failure. The threshold shall be derived from the system's FTTI for the "display availability" Safety Goal. 3. Coverage of Both Display Channels If both VOUT display channels (Disp0 and Disp1) are used for safety-relevant output (e.g., surround-view or rear-view camera), the System Integrator shall implement frozen frame detection independently for each channel. 4. Error Handling and Safe State Upon detection of a frozen frame condition exceeding the defined threshold, the safety application shall flag the video output as unreliable, notify the driver or system safety manager, and transition to a Safe State (e.g., display a warning overlay, revert to a backup video source, or disable the automated driving feature relying on that camera feed) within the FTTI. **SDK API and integration flow:** Read the VOUT frame checksum / freeze status via AmbaVout_GetDisp0Status() and AmbaVout_GetDisp1Status(); the freeze result is returned in the pFreezeDetect output parameter. Signature: UINT32 AmbaVout_GetDisp0Status(UINT32 *pStatus, UINT32 *pFreezeDetect).

5.1.36 VOUT Data Input FIFO Underflow Detection

Name	*Description*
ID	SM_HW_UNDERFLOW_VOUT
Description	The FIFO underflow detection can detect the problem when VOUT needs to generate data to DSI-PHY, but the FIFO is

	empty. In this case, VOUT will generate garbage data (not desired video data) to keep the frame rate and report the error.
Ownership	Ambarella
SM is to against	Insufficient input data to VOUT
Software Related	No
Failure mode coverage	99%
Initialization	N/A
Operation	N/A
Error indication	Assert safety error signal
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_LBIST

5.1.37 VOUT MIPI FIFO Overflow Detection

Name	*Description*
ID	SM_HW_OVERFLOW_VOUT
Description	When something goes wrong in DSI-PHY, or in DSI-PHY and outside panel / SOC connection, DSI-PHY may hang or respond to VOUT handshake request slower than expected. This will lead to incorrect data output. The MIPI FIFO overflow detection can detect when the performance drop downstream VOUT starts to cause incorrect data output.
Ownership	Ambarella
SM is to against	No response (hang) or slow response from DSI-PHY

Software Related	No
Failure mode coverage	99%
Initialization	N/A
Operation	N/A
Error indication	Assert safety error signal
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_LBIST

5.1.38 Duplicated Safety Critical IRQ

Name	*Description*
ID	SM_HW_GIC_DUP_IRQ
Description	The safety mechanism, SM_HW_GIC_DUP_IRQ, is implemented in hardware. A safety critical IRQ is cloned in source end and the IRQ pair is connected to two inputs of GIC. System Integrator is responsible for analyzing the safety critical IRQ and this safety mechanism software.
Ownership	Ambarella and System Integrator
SM is to against	Spurious missing IRQ (for safety critical IRQs only) and random hardware faults on IRQ wiring between IP and GIC.
Software Related	Yes
Failure mode coverage	99%
Initialization	If the safety critical function is in use, software should enable both channels to monitor if they behave the same way.

Operation	If one of safety critical GIC channel asserts, the software should check if its counterpart behaves in the same way.
Error indication	Assert safety error signal by software
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_GIC_DUP_IRQ (software compare)

5.1.39 CPU Error Monitoring Unit

Name	*Description*
ID	SM_HW_CEMU
Description	– This safety mechanism is used to detect any wrong address and L2 error signals from Cortex-A53. - Read / Write address signals from Cortex-A53 are compared with predefined address range. Any address outside of predefined address range is considered as invalid address. - Read / Write IDs from Cortex-A53 that indicate which CPU core issue the read / write request are recorded when invalid address occur. - L2 error signals from Cortex-A53 (nEXTERRIRQ & nINTERRIRQ) are OR-ed with address checking error to generate a consolidated error signal to CEHU. System Integrator is responsible for implementing and integrating the software for this safety mechanism.
Ownership	Ambarella and System Integrator
SM is to against	Wrong address and L2 error signals (nEXTERRIRQ and nINTERRIRQ) from Cortex-A53
Software Related	Yes
Failure mode coverage	60%
Initialization	Software shall define valid address ranges by setting addr_start_0~9 and addr_end_0~9 in axi_config registers. Then, set range_ck_en_0~9

	in axi_config registers to enable this address checking feature. For the detailed programming flow, please refer to Programming Reference Manual.
Operation	N/A
Error indication	Assert safety error signal
Error detect time	99 ms
Measure to prevent latent fault	None

5.1.40 APB Watchdog Timer

Name	*Description*
ID	SM_HW_APB_WDT
Description	– An APB watchdog timer is used to check if there is Cortex-A53, Cortex-R52, or AMBA bus hang; if CPU doesn't reload watchdog timer internal counter before it reaches zero, then the watchdog timer would assert error. – Software warm reset shall be enabled if there is no external WDT available. System Integrator is responsible for implementing the software to enable the WDT safety mechanism.
Ownership	Ambarella and System Integrator
SM is to against	Hang on Cortex-A53, Cortex-R52 or AMBA bus.
Software Related	Yes
Failure mode coverage	99%
Initialization	Please refer to Programming Reference Manual.
Operation	N/A

Error indication	Assert safety error signal or assert warm reset to the SEooC
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_DCLS and SM_SW_PO_WDT_CHK

5.1.41 Fault Injection for DCLS in Cortex-R52

Name	*Description*
ID	SM_HW_FAULT_INJ_CR52
Description	Cortex R52 is protected by the DCLS design. This safety measure is to detect its latent fault by forcing comparison logic of the DCLS design fail and check if the (internal) safety error is asserted or not. System Integrator shall implement this safety mechanism software.
Ownership	Ambarella and System Integrator
SM is to against	Latent faults of Cortex R52 DCLS comparison logic
Software Related	Yes
Failure mode coverage	90%
Initialization	In startup test, setup axi_config module to enable "Force Comparison Fail (FRC)" (Enable Comparison (EN) is on by default). For this test, error pins to flag safety error should be masked by software.
Operation	Check if corresponding error bits inside AMBA ECRU module are asserted.
Error indication	Keep in safe state
Error detect time	Power-on test
Measure to prevent latent fault	N/A (this safety mechanism is to detect latent faults)

5.1.42 ECC Corrected-Error Counter

Name	*Description*
ID	SM_HW_ECC_CORR_CNT
Description	– Implementation details: In the SEooC, the following groups of memories are SECDED ECC protected. - FMEM in SMEM cluster - Channel register memories in SMEM cluster - Sync counters in SMEM cluster - PMEM SRAMs in VMEM - FM SRAMs in XMEM Each group has: - a saturating counter keeping track of how many single-bit errors have been corrected. For every cycle, this counter only counts up to one corrected error; - a debug port accessible (R/W) threshold indicating at what count the HW will report an error. That is, if the corrected count exceeds the threshold, an error will be reported to CEHU. 1. In SMEM, these counters are 12-bits each for the channel register and sync counter groups (which do scrubbing to write back corrected values), and 24-bits for the FMEM counter group (does not do scrubbing). The threshold's default value is its maximum value (all 1s). 2. In VMEM this counter is 22-bits. The threshold's default value is 127. At the next debug port address, SW can read current corrected error count. 3. In XMEM, this counter is 8-bits. The threshold's default value is 10. SW can read current corrected error count via debug port. - a mechanism that would prevent an error from being sent to CEHU for corrected single-bit errors, regardless of the count. 1. In SMEM and VMEM, this is achieved by a disable / mask bit. 2. In XMEM, a threshold value of 255 is interpreted as infinity to achieve the same purpose.
Ownership	Ambarella
SM is to against	Permanent failures in SRAM with ECC protection
Software Related	Yes

Failure mode coverage	90%
Initialization	Setup the threshold registers if needed.
Operation	N/A
Error indication	Assert safety error signal by software
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_LBIST
SDK Usage	The ECC Corrected-Error Counter mechanism monitors for the accumulation of single-bit ECC corrections across multiple internal SRAM groups within the DSP and vector processing subsystems (SMEM, VMEM, and XMEM). Each SRAM group maintains an independent saturating counter that tracks corrected single-bit errors. If any counter exceeds its software-configurable threshold, a safety error signal is asserted. This mechanism distinguishes between isolated transient upsets (acceptable) and a pattern of accumulating corrections (indicative of a developing permanent fault). **Monitored Memory Groups and Counter Specifications:** **SMEM (FMEM):** 24-bit saturating counter; no scrubbing; default threshold = maximum (all 1s). **SMEM (Channel Register and Sync Counter memories):** 12-bit saturating counters; scrubbing enabled; default threshold = maximum (all 1s). **VMEM (PMEM SRAMs):** 22-bit saturating counter; default threshold = 127. **XMEM (FM SRAMs):** 8-bit saturating counter; default threshold = 10. **Detected Error Types:** **Single-Bit Error (SBE) Rate Exceedance:** The corrected error count for any SRAM group exceeds the programmed threshold, indicating that the transient error rate is beyond the acceptable baseline, suggesting SRAM degradation. **Latent Fault Progression:** Repeated corrections in the same memory cell or region indicate the onset of a permanent stuck-at fault. **Assumptions of Use** 1. Threshold Configuration and Safety Analysis The System Integrator shall review the default threshold values for each memory group (SMEM, VMEM, XMEM) and, based on a system-level safety analysis, determine whether the defaults are appropriate or need

	to be adjusted. For safety-critical applications, thresholds shall be set to detect degradation before an uncorrectable double-bit error can occur. 2. Error Monitoring via Debug Port The System Integrator shall implement a periodic monitoring routine that reads the current corrected error counts from the debug port for VMEM and XMEM. This polling interval must be shorter than the DTTI. 3. CEHU Fault Response Upon a threshold violation reported by CEHU, the safety application on the Cortex-R52 shall log the affected memory group, invalidate any data computed using that memory, and evaluate whether a warm reset of the affected subsystem can restore operation. If the fault is persistent, the system shall transition to a Safe State. 4. XMEM Infinity Mode The System Integrator shall note that for XMEM, setting the threshold register to 255 effectively disables CEHU reporting for that group. This setting shall only be used when the safety analysis confirms that XMEM SBEs do not pose a risk to the Safety Goal. **SDK API and integration flow:** This safety mechanism is implemented inside the microcode (ucode); the System Integrator shall check the error reported from the DSP driver or the CEHU.

5.1.43 SRAM Duplication

Name	*Description*
ID	SM_SRAM_DUP
Description	The SRAM macro is duplicated
Ownership	Ambarella
SM is to against	Any random hardware fault in SRAM macro.
Software Related	No
Failure mode coverage	99%
Initialization	N/A
Operation	N/A

Error indication	Assert safety error signal
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_LBIST (if applicable)

5.1.44 SRAM SECDED for Address and Data

Name	*Description*
ID	SM_SRAM_SECDED_ADDR_DATA
Description	Hsiao code is used to perform single-error-correction-double-error-detection for address and data of SRAM macro.
Ownership	Ambarella
SM is to against	Faults in SRAM cell array, control logic, address decoder, word lines, data path, and bit lines.
Software Related	Yes
Failure mode coverage	Over 90% on average
Initialization	For the case that the hardware can read an uninitialized address, such as SMEM, software shall enable self-initialization feature to put each entry to a valid state.
Operation	N/A
Error indication	Assert safety error signal
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_LBIST (if applicable)

SDK Usage	The SRAM SECDED (Single-Error Correction, Double-Error Detection) for Address and Data is a hardware-implemented fault detection and correction mechanism using Hsiao code. It is applied to SRAM macros where both the address and data paths are protected. For single-bit errors occurring anywhere in the protected address or data word, the hardware automatically corrects the error before it propagates. For double-bit errors, the hardware flags the event without correction, enabling the system to detect uncorrectable faults. **Detected Error Types:** **Single-Bit Error (SBE):** A single bit flip in the stored address or data, which is automatically corrected by the Hsiao code hardware. The correction is transparent but may be tracked via the ECC error counter (SM_HW_ECC_CORR_CNT). **Uncorrectable Double-Bit Error (DBE):** Two simultaneous bit errors within the same ECC word, which cannot be corrected by SECDED. The hardware detects and flags this event, triggering an error report to CEHU. **Address Decoder Fault:** A stuck-at fault in the address path covered by the ECC that prevents correct SRAM cell selection, detected via the address parity component of the Hsiao code. **Assumptions of Use** 1. Transparency of Operation The SRAM SECDED mechanism operates autonomously within the hardware and ucode. Single-bit corrections are performed transparently without software intervention. The System Integrator does not need to explicitly enable this mechanism, as it is active by default on all protected SRAM macros. 2. DBE Error Monitoring via CEHU The System Integrator shall ensure that CEHU error reports from SECDED-protected SRAMs are handled by the safety application on the Cortex-R52. A DBE event shall be treated as a critical, non-recoverable hardware fault requiring immediate transition to a Safe State. 3. Integration with ECC Corrected-Error Counter The System Integrator shall use the ECC Corrected-Error Counter mechanism (SM_HW_ECC_CORR_CNT) in conjunction with this SM to monitor the rate of SBE corrections. An elevated SBE rate may indicate the onset of a permanent SRAM fault even before a DBE occurs. 4. Validation at Power-On The System Integrator shall ensure that the PO MBIST (SM_HW_PO_MBIST) is executed at every power cycle to validate the integrity of SECDED-protected SRAMs before normal operation begins. **SDK API and integration flow:** This safety mechanism is implemented inside the microcode (ucode); the System Integrator shall check the error reported from the DSP driver or the CEHU.

5.1.45 SRAM SECDED for Data Only

Name	*Description*
ID	SM_SRAM_SECDED_DATA
Description	Hsiao code is used to perform single-error-correction-double-error-detection for data of SRAM macro.
Ownership	Ambarella
SM is to against	Faults in SRAM cell array, control logic, data path, and bit lines.
Software Related	No
Failure mode coverage	over 90% on average
Initialization	N/A
Operation	N/A
Error indication	Assert safety error signal
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_LBIST (if applicable)

5.1.46 SRAM Parity for Address and Data

Name	*Description*
ID	SM_SRAM_PAR_ADDR_DATA
Description	Add parity bit to detect single bit error for address and data
Ownership	Ambarella

SM is to against	Faults in SRAM cell array, control logic, address decoder, word lines, data path, and bit lines.
Software Related	No
Failure mode coverage	Over 90% on average
Initialization	N/A
Operation	N/A
Error indication	Assert safety error signal
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_LBIST (if applicable)

5.1.47 SRAM Parity for Data Only

Name	*Description*
ID	SM_SRAM_PAR_DATA
Description	Add parity bit to detect single bit error for data
Ownership	Ambarella
SM is to against	Faults in SRAM cell array, control logic, data path, and bit lines.
Software Related	No
Failure mode coverage	Over 90% on average
Initialization	N/A
Operation	N/A

Error indication	Assert safety error signal
Error detect time	99 ms
Measure to prevent latent fault	SM_HW_LBIST (if applicable)

5.1.48 PO MBIST

Name	*Description*
ID	SM_HW_PO_MBIST
Description	PO MBIST (Power On MBIST) is executed once every power cycle for the ECC protected SRAMs in VMEM and SMEM to flag any SRAM faults which could be latent due to ECC. The MBIST is a library binary provided by Ambarella.System Integrator is responsible for integrating it into the product software.
Ownership	Ambarella and System Integrator
SM is to against	Latent faults in ECC protected SRAMs in VMEM & SMEM
Software Related	Yes
Failure mode coverage	80%
Initialization	Refer to the initialization flow of SM_HW_LBIST.
Operation	N/A
Error indication	SW determines to remain in safe state by reading error bits in IST STATUS register
Error detect time	Power-on test
Measure to prevent latent fault	N/A (this safety mechanism is to detect latent faults)

SDK Usage	The Power-On MBIST (Memory Built-In Self-Test) is executed once per power cycle against the ECC-protected SRAMs within the VMEM and SMEM subsystems. Because the SECDED ECC logic can silently mask single-bit faults in SRAMs that contain stale data, a comprehensive MBIST is required to detect latent faults in these memories before operational data is loaded. The MBIST is implemented as a binary library and is triggered from within the bootstrap code, providing diagnostic coverage comparable to SM_HW_LBIST (5.1.17) for the SRAM array. **Detected Error Types:** **Latent SRAM Faults (ECC-Masked):** Permanent stuck-at faults within SRAM bit-cells that are currently being silently corrected by the ECC logic. These faults would become uncorrectable if an adjacent bit also fails. **Hard SRAM Cell Defects:** Permanent physical defects (open, short, or stuck-at) within the SRAM array that are revealed by the MBIST pattern sequences. **Address Decoder Failures:** Faults in the SRAM row or column address decoders that prevent correct cell selection. **Peripheral Circuit Failures:** Failures in the sense amplifiers, write drivers, or precharge circuitry that prevent reliable read/write operations. **Assumptions of Use** 1. Mandatory Execution at Every Power Cycle The System Integrator shall ensure the PO MBIST is executed during every power-on boot sequence before safety-critical applications are initialized. The MBIST must complete with a PASS result before the system transitions from the "Start-up" phase to "Normal Operation." 2. Integration via Bootstrap Code The System Integrator shall integrate the Ambarella-provided MBIST library binary into the production software bootstrap (AmbaBST_BIST.asm), following the same approach as SM_HW_LBIST. The MBIST must be invoked before any application data is written to VMEM or SMEM. 3. Result Retrieval and Evaluation The System Integrator shall implement a safety application routine that reads the MBIST result via the provided SDK API after boot. A FAIL result shall prevent the system from entering normal operation and shall trigger a system-level fault response (e.g., safety error notification, fallback to a reduced operating mode, or system halt). 4. Failure Handling If the PO MBIST reports a failure, the System Integrator shall treat this as a permanent hardware fault. The system must not enter operational mode, and the failure shall be reported to the diagnostic log and, where applicable, to an external safety monitor or system controller. **SDK API and integration flow:** The safety application reads the power-on BIST result via AmbaCEHU_BstBistResultGet(UINT32 *pResult); a return value of 0 indicates no error and a non-zero value indicates a BIST error.

	Note: The reference code / sample code is provided to demonstrate usage only and shall not be used as production code.

5.2 Software Safety Mechanisms

A SW-SM (software safety mechanism) is a safety mechanism to be implemented in software to diagnose faults without dedicated hardware logic on silicon.

5.2.1 Software Test for Power Supply Monitor

Name	*Description*
ID	SM_SW_PSMON_TEST
Description	This mechanism is used to test the availability of power monitors once per driving cycle. SW intentionally sets a wrong voltage range to see if the safety error output of PSMON is asserted. System Integrator is responsible for implementing and integrating this safety mechanism software.
Ownership	System Integrator
SM is to against	Any latent faults in power monitor
Software Related	Yes
Failure mode coverage	60%
Initialization	• After chip power on reset, SW should configure the PSMON in test mode • Set abist_en=1'b0 to test the power sensing block functionality • Set the under voltage and over voltage target range • Toggle uvov_swap signal and read pm_obsv<5:0> and pm_errorb output • Check the pm_obsv<5:0> with expected value • Check if CEHU receives pm_errorb from each PSMON • Repeat above procedure for 4 times. • Ignore the related observations if any monitor path that is powered down
Operation	Assert safety error signal if error from the power monitor is not received
Error indication	Assert safety error signal by software
Error detect time	Power-on test

5.2.2 Software Check Initial Safety Error State

Name	*Description*
ID	SM_SW_CHK_SAFETY_ERR
Description	According to TLSS-1, safety error indicators output by sub-components shall be in active state. Software shall check the safety error status in both CEHU and AMBA_ECRU after a global reset event. The safety error status register in CEHU should be all ones except • Bit [37:32]: WDT external safety errors • These signals may be used as reset signals to external element(s), and thus is waived to avoid interfering other element(s). SW can check WDT internal safety errors at Bit 0 to Bit 5 instead. • Bit 51: APB_DBG timeout error • This timeout error is not flop output. SW shall check APB_DBG safety error flag at bit 31 instead. • Bit 94: ijtag safety error • This is not flop output but gated version of xx_jtag_rst_l which is tied to low externally. • Bit 102: OTP safety error • This is cleared by VROM boot code. • Bit 107: SW-ARM safety error • This is the register used by SW only. Instead of checking initial safety error state of SW-ARM safety error, SW can check bit-102 of AMBA_ECRU safety error status, which is at the same reset domain • Bit 108: test-mode safety error • This is not flop output, but wired to xx_test_mode, which is tied to low externally. • Bit 113: VROM safety error. • This is cleared by VROM boot code. System Integrator is responsible for implementing this safety mechanism software checker.
Ownership	System Integrator
SM is to against	Any fault causing reset signal for each IP stuck at inactive state
Software Related	Yes
Failure mode coverage	100%
Initialization	Software should check these status after power on reset or warm reset before clearing.

Operation	SW should check CEHU error_bit_vector_0~4 registers and ECRU error_log0~3 registers right after power on reset or global reset to know if reset signals work. If the values are not consistent with the description above, SW should not leave safe state.
Error indication	Assert safety error signal by software
Error detect time	99 ms

5.2.3 Software Watchdog Timer

Name	*Description*
ID	SM_SW_WDT
Description	The Arm / Cortex software implements interrupt service routine monitoring to check the duration for specified ISR to prevent unexpected delay or hang. System Integrator is responsible for implementing this safety mechanism monitor software.
Ownership	System Integrator
SM is to against	Any random hardware fault that can cause a hang
Software Related	Yes
Failure mode coverage	100%
Initialization	A software timer shall be started after software has configured the IP function for certain task.
Operation	Software shall wait for an operation done indicator (such as the completion interrupt). Simultaneously, software shall check the timer periodically. If the IP has not yet asserted the operation done indicator in a predefined time interval, it's assumed that the IP is hanging.
Error indication	Assert safety error signal by software

Error detect time	99 ms

5.2.4 Software Test Clock Monitor

Name	*Description*
ID	SM_SW_CKMON_TEST
Description	After chip power-on reset, software shall perform the test for all clock monitors by setting an illegal upper-bound and lower-bound to false-trigger safety error and see if the error is propagated to CEHU correctly. System Integrator is responsible for implementing and integrating this safety mechanism software.
Ownership	System Integrator
SM is to against	Latent fault of clock monitor comparator.
Software Related	Yes
Failure mode coverage	60%
Initialization	• Enable clock source to get target frequency • Set clock monitor upper_bound to max and set lower_bound to zero • Program CEHU to receive error from each clock monitor • Enable each clock monitor • Check if CEHU receives error from each clock monitor • Clear error of CEHU clock monitor error bit
Operation	Assert safety error if error from clock monitor not received
Error indication	Assert safety error signal by software
Error detect time	Power-on test

5.2.5 Software Check POC Settings

Name	*Description*
ID	SM_SW_POC_CHK
Description	Software periodically check if POC settings are identical to pre-determined value. System Integrator is responsible for implementing this safety mechanism software.
Ownership	System Integrator
SM is to against	Any fault causing wrong POC setting loaded to configuration register.
Software Related	Yes
Failure mode coverage	100%
Initialization	N/A
Operation	Periodically readout and check RCT system configuration registers value.
Error indication	Assert safety error signal by software
Error detect time	99 ms

5.2.6 Sensor Embedded Data ACK to SW

Name	*Description*
ID	SM_SW_SENSOR_EMB_ACK
Description	External video sensor may send V/H-sync ACK / NACK flags as part of embedded auxiliary data. The embedded/auxiliary data is captured by IDSP / VIN and stored to SMEM for software processing. Software can then process the embedded/auxiliary data and determine if the sensor has acknowledged the reception of V/H-sync signal within the expected duration. System Integrator is responsible for implementing the sensor control software and the embedded data handling software.

	Ambarella is responsible for reading out the embedded data for System Integrator to handle.
Ownership	System Integrator and Ambarella
SM is to against	Fault on V/H-sync signal to external video sensor
Software Related	Yes
Failure mode coverage	99%
Initialization	Software should enable the embedded streams via IDSP ORC command, and setup the appropriate SMEM channels to capture the embedded data.
Operation	Software should periodically process the stored embedded data to determine if the sensor received the V/H-sync within expected duration.
Error indication	Assert safety error signal by software
Error detect time	99 ms

5.2.7 Software Watchdog Timer on ORCs

Name	*Description*
ID	SM_SW_WDT_ORC
Description	A timer implemented by software so that software runs on Cortex-R52 and the software that runs on ORCs shall have some message passing periodically through DRAM which can detect hang in ORCs. Ambarella is responsible for the software running on ORCs and Cortex-R52 for this safety measure. System Integrator is responsible for integrating and verifying both software.
Ownership	Ambarella and System Integrator

SM is to against	Any random hardware fault that can cause the ORCs to hang.
Software Related	Yes
Failure mode coverage	100%
Initialization	Software should initialize the message passing data structure in DRAM before it wakes up the ORC.
Operation	Software that runs on the ORC should pass a heartbeat message to Cortex-R52 at least once every 99 ms to be monitored by Cortex-R52 software to make sure the ORC is not hanging.
Error indication	Assert safety error signal by software
Error detect time	99 ms
SDK Usage	The Software Watchdog Timer on ORCs (SM_SW_WDT_ORC) is a software-based liveness monitoring mechanism for the ORC processors. It operates by requiring the ORC software to increment a shared counter in DRAM at a defined periodic rate. The safety application running on the Cortex-R52 monitors this counter and detects if the ORC has hung or stalled by checking whether the counter has advanced within the expected time window. **Detected Error Types:** **ORC Software Hang:** The ORC software fails to execute the counter-update routine within the expected interval, indicating a deadlock, infinite loop, or unhandled exception in ORC firmware. **ORC Hardware Stall:** The ORC processor itself becomes unresponsive due to a hardware fault (e.g., pipeline freeze, instruction memory fault), preventing software execution. **DRAM Communication Failure:** The ORC is unable to write the counter value to DRAM due to a bus fault or memory access failure, resulting in the R52 observing a stale counter value. **DSP Pipeline Freeze:** The DSP pipeline enters a state where the ORC is kept busy waiting for hardware resources that are no longer responding, preventing normal counter updates. **Assumptions of Use** 1. Monitor Initialization and Configuration The System Integrator shall initialize the ORC monitoring infrastructure using AmbaDSP_MonitorInit() and configure the monitoring parameters using AmbaDSP_MonitorConfig() as provided in the SDK reference code. The configuration must define the expected counter update period and the acceptable deviation window.

	2. R52 Monitoring Implementation The System Integrator shall implement the monitoring software on the Cortex-R52 to periodically read the ORC liveness counter from DRAM. The polling interval must be set such that a hung ORC is detected and responded to within the system's FTTI. 3. Watchdog Timeout Calibration The timeout threshold shall be derived from the worst-case ORC task execution time under maximum system load. The threshold must be long enough to avoid false positives during heavy DSP processing, but short enough to detect genuine ORC failures within the FTTI. 4. Error Handling and Recovery Upon detection of a stalled ORC counter, the R52 safety application shall attempt a warm reset of the ORC subsystem if the safety analysis permits a recovery attempt. If the counter fails to resume after reset, the system shall invalidate all DSP outputs and transition to a Safe State (e.g., disabling perception features and notifying the driver). **SDK API and integration flow:** Monitor the ORC counter from the R52 via AmbaDSP_MonitorInit() and AmbaDSP_MonitorConfig() (DspMonitor reference code). *Note: The reference code / sample code is provided to demonstrate usage only and shall not be used as production code.*

5.2.8 Software Check Timestamp Interface

Name	*Description*
ID	SM_SW_CHK_ORC_INTF
Description	Software needs timestamp as the reference timing. Two timers in Ethernet instances are used to generate the timestamps for software to read. One is the primary timer, and the other is the shadow. Software shall compare the timer values to detect faults between the timers and SW. System Integrator is responsible for implementing and integrating this safety mechanism software.
Ownership	System Integrator
SM is to against	Any fault on the path from timestamp generators to SW.
Software Related	Yes
Failure mode coverage	99%

Initialization	If any of two Ethernet instances is not enabled in system application, software shall force the unused Ethernet instance in RMII mode to make ptp_timestamp_o signal running.
Operation	Software shall periodically check whether the timers are running and the difference of two timestamps is in a reasonable range.
Error indication	Assert safety error signal by software
Error detect time	99 ms
SDK Usage	The Software Check Timestamp Interface (SM_SW_CHK_ORC_INTF) verifies the integrity of the timestamp reference used by safety software. Two Ethernet (ENET) instance timers generate the timestamps that software reads as its timing reference; the SDK provides the API to configure this check so that a fault on the path from the timestamp generators to software is detected. **Assumptions of Use:** The System Integrator shall follow the SDK API and integration flow below and use the AmbaDSP_MainSafetyCfg API to implement the checker within the user safety application, selecting a timer threshold (msec) appropriate to the system timing requirements. **SDK API and integration flow:** The integrator shall (1) enable ENET, (2) determine the ENET timer threshold, and (3) boot the DSP and invoke AmbaDSP_MainSafetyCfg(0U, msec, 0). When the difference between the ENET timers exceeds the configured threshold, a CEHU error is raised.

5.2.9 Software Watchdog Timer on VIN

Name	*Description*
ID	SM_SW_WDT_VIN
Description	A watchdog timer implemented by software to check if VIN receives the first frame after a pre-defined time. System Integrator is responsible for implementing the safety software to check whether Vin receives the first frame.
Ownership	System Integrator
SM is to against	Any random hardware fault that causes the system to hang

Software Related	Yes
Failure mode coverage	100%
Initialization	The timer shall be enabled after VIN and MIPI-DPHY are configured to receive image data, and the sensor is configured to start sending data
Operation	Check if VIN receives the first frame after a pre-defined time, which is determined by the frame-rate, and must be less than DTTI (99 ms)
Error indication	Assert safety error signal by SW
Error detect time	99 ms
SDK Usage	The VIN First-Frame Watchdog is a software-based safety mechanism designed to ensure the availability and readiness of the video input pipeline during the transition from "System Initialization" to "Active Streaming." By monitoring the arrival of the very first frame from the sensor within a deterministic time window, the system can detect critical startup failures that would otherwise result in a "black screen" or silent failure of the perception stack. **Detected Error Types:** **Sensor Initialization Failure:** The sensor fails to start streaming due to power-up sequencing issues, I2C configuration errors, or internal firmware hangs. **Link/Interface Failures:** Physical layer issues (e.g., MIPI CSI-2) or incorrect clocking that prevent the first packet from reaching the VIN module. **DSP Configuration Mismatch:** Incorrect resource allocation in the DSP that prevents the pipeline from binding to the incoming stream. **Startup Latency Violations:** Scenarios where the sensor takes longer than the safety-critical limit to produce data, potentially delaying the availability of safety features (e.g., Rear View Camera display). **Assumptions of Use** 1. Configuration of Timeout Thresholds The System Integrator shall utilize the AmbaDSP_ResourceLimitConfig() API to define the maximum allowable startup latency. Specifically, the integrator must populate the MaxVinTimeout field within the AMBA_DSP_LIVEVIEW_RESOURCE_s structure. This value must be derived from the sensor's worst-case power-on-to-streaming specification. 2. Software Watchdog Implementation The System Integrator is responsible for implementing the monitoring logic. This software shall:

	Start the watchdog timer simultaneously with the "Start Streaming" command sent to the DSP/Sensor. Use the DSP status APIs to poll for the successful reception of the first frame. Explicitly clear the watchdog only upon receipt of a valid "First Frame" interrupt or status flag. 3. Reference to Sample Implementation The System Integrator should refer to the provided sample code for the correct sequence of the AmbaDSP_ResourceLimitConfig call and the integration of the AMBA_DSP_RESOURCE_s structure. The integrator may adapt the sample code flow to match the specific system architecture, provided the timeout check remains deterministic. 4. Error Handling and Recovery If the first frame is not received within the MaxVinTimeout period, the safety application shall: Trigger an immediate "Vision Path Unavailable" fault. Attempt a re-initialization of the sensor and VIN module if the safety analysis allows for a retry. 5. Synchronization with System FTTI The System Integrator shall ensure that the MaxVinTimeout is calibrated such that the time to detect a non-starting sensor, plus the time to transition the system to a Safe State, is within the Fault Tolerant Time Interval (FTTI) of the relevant Safety Goal (e.g., "Display image within 2.0 seconds of reverse gear engagement"). **SDK API and integration flow:** Configure the VIN first-frame timeout via AmbaDSP_ResourceLimitConfig() (field path AMBA_DSP_RESOURCE_s -> AMBA_DSP_LIVEVIEW_RESOURCE_s -> MaxVinTimeout). *Note: The reference code / sample code is provided to demonstrate usage only and shall not be used as production code.*

5.2.10 Software checksum calculation for coding out stream

Name	*Description*
ID	SM_SW_CODE_CRC
Description	There is a checksum generating engine inside the coding unit. Software can feed data from SMEM into this engine to generate a checksum. The value of the checksum can be cleared and read by software via special purpose registers (SPR) from the coding ORC.
Ownership	Ambarella

SM is to against	Non-transient random hardware fault in coding unit datapath
Software Related	Yes
Failure mode coverage	60%
Initialization	Software shall clear this checksum before running a test frame.
Operation	After running the test frame, software shall issue linear DMA loads to bring back the coding out stream from SMEM into the checksum generating engine to generate a checksum. Software shall read the checksum and compare it with the golden value. If there is a mismatch, software can write to SPR_CEHU_ERR to signal an error to CEHU.
Error indication	Assert safety error signal by software
Error detect time	99 ms
SDK Usage	The Software Checksum Calculation for Coding Output Stream (SM_SW_CODE_CRC) leverages a hardware checksum generation engine embedded within the coding unit. This engine accumulates a checksum over data fed from SMEM through the coding pipeline. The ORC software reads and compares this hardware-generated checksum against an expected golden value at the completion of the DSP test frame, providing end-to-end data integrity verification of the coding data path. **Detected Error Types:** **Coding Data Path Corruption:** A permanent or transient fault within the SMEM or the coding pipeline that results in incorrect data being processed, producing a checksum mismatch against the expected golden value. **SMEM Read Fault:** An error in the SMEM read path that delivers corrupted data to the coding unit, detectable via the checksum discrepancy. **Checksum Register Fault:** A fault in the Special Purpose Register (SPR) used to hold the accumulated checksum, causing an incorrect value to be read by the ORC. **Systematic Processing Error:** A deterministic fault in the coding unit's data processing logic that consistently produces incorrect output, detectable through the golden checksum comparison. **Assumptions of Use** 1. Test Frame Integration **The System Integrator shall integrate the checksum verification into the VDSP test frame execution flow using the provided APIs:**

	AmbaDSP_TestFrameConfig() and AmbaDSP_TestFrameCtrl(). The test frame shall be configured to exercise the full coding data path and generate a checksum over the expected output data. 2. Golden Checksum Derivation and Storage The System Integrator shall derive the golden checksum value from a validated reference run of the coding pipeline on fault-free hardware. The golden value shall be stored in a read-protected, integrity-checked memory location to prevent it from being corrupted and masking a fault. 3. Execution Frequency and FTTI Compliance The System Integrator shall configure the test frame execution rate such that the maximum interval between consecutive checksum verifications is less than the DTTI defined in the system safety plan. 4. Error Handling Upon detection of a checksum mismatch, the ORC shall notify the R52 safety application via the DSP driver event reporting mechanism. The R52 shall treat the mismatch as a data integrity fault, invalidate the coding output for the affected frame, and transition the system to a Safe State within the remaining FTTI budget. **SDK API and integration flow:** The VDSP test frame generates and compares the checksum inside the ORC; configure via AmbaDSP_TestFrameConfig() and AmbaDSP_TestFrameCtrl().

5.2.11 Software Periodic Register Read Test

Name	*Description*
ID	SM_SW_CYC_REGRD
Description	Software to read the configuration register of all safety-related AHB / APB bus clients, AXI configuration space, and NIC-400 GPV, and compare it against the expected value periodically. It cannot detect the read-only type configuration and thus, the diagnostic coverage highly depends on the ratio of read/writable register. System Integrator is responsible for implementing and integrating this safety mechanism software.
Ownership	System Integrator
SM is to against	To detect random HW failure of configuration registers defined above.
Software Related	Yes

Failure mode coverage	60% to 90%
Initialization	If a client is reset, software shall read and compare first before any operation to make sure it's reset properly.
Operation	SW to read out configuration registers defined above and compare their values against expected ones.
Error indication	Assert safety error signal by software
Error detect time	99 ms

5.2.12 Software Periodic Read After Exclusive-Write Test

Name	*Description*
ID	SM_SW_CYC_RAEW
Description	Periodically perform exclusive write by Cortex-R52 and read back by Cortex-A53 to detect stuck-at fault on AXI bus lock signal. System Integrator is responsible for implementing the safety mechanism software.
Ownership	System Integrator
SM is to against	Any stuck-at fault on AXI client's lock signal.
Software Related	Yes
Failure mode coverage	100%
Initialization	N/A
Operation	Software shall ask Cortex-R52 to write pre-defined data to a specific region with exclusive access, and also ask Cortex-A53 to write another pre-defined data to the same location before Cortex-R52's exclusive assess is done and read data from this location to check if data is overwritten by Cortex-A53.

Error indication	Assert safety error signal by software
Error detect time	99 ms

5.2.13 Software GDMA Test Pattern

Name	*Description*
ID	SM_SW_GDMA_TEST
Description	Issue the test patterns to detect faults in the combination logic of the GDMA engine. This test pattern covers all 6 opcodes supported by the GDMA and also covers the maximum width and maximum height supported by the GDMA. System Integrator is responsible for implementing and integrating the safety mechanism into the final system software.
Ownership	System Integrator
SM is to against	Any random hardware fault in the combinational logic used for the GDMA operation.
Software Related	Yes
Failure mode coverage	60%
Initialization	N/A
Operation	The test pattern should consist of 12 GDMA operations, 2 for each Opcode, in any order. Not all 12 operations have to be run consecutively, if there is not enough time for the entire test pattern to run. For each Opcode, 1 operation should be run with Width = 0x1fff, Height = 0 and 1 operation should be run with Width = 0, Height = 0xfff. Each operation can be run with a random legal combination of Source1Format, Source2Format, and DestinationFormat. The operations should start with known source data in DRAM, so that the destination data written back out to DRAM can be compared with the expected value
Error indication	Assert safety error signal by software
Error detect time	99 ms

5.2.14 Software SD Error Checker

Name	*Description*
ID	SM_SW_SD_ERR_CHK
Description	SD bus protocol utilizes CRC for data integration check. Software shall monitor if the CRC error events are triggered unusually often. System Integrator is responsible for implementing the monitor software.
Ownership	System Integrator
SM is to against	Any permanent fault in CRC encoder/decoder or SD bus interface that causes CRC error asserted all the time.
Software Related	Yes
Failure mode coverage	99%
Initialization	N/A
Operation	Software checks SD error status register whenever the SD transaction is over. If SD error happens, software may try to take proper action to recover SD controller and SD device. If the error continuously happen, software shall treat this as a permanent fault.
Error indication	Assert safety error signal by software
Error detect time	99 ms
SDK Usage	The Software SD Error Checker (SM_SW_SD_ERR_CHK) monitors the frequency of CRC errors occurring on the SD bus interface. The SD bus protocol includes hardware CRC checks on all data transfers; a single CRC error may indicate a transient upset, but an elevated or sustained error rate is indicative of a degrading physical interface, a failing SD device, or a connector fault. This software mechanism detects abnormal SD error rates before they result in uncorrectable data loss. **Detected Error Types:** **CRC Error Rate Exceedance:** The frequency of CRC errors on SD read or write operations exceeds a defined threshold, indicating interface degradation or persistent signal integrity issues.

	Persistent Transfer Failure: Repeated CRC errors on the same sector or transfer, indicating a permanent fault in the SD device or the SD controller's physical layer. **Silent Data Corruption Risk:** A sustained pattern of correctable CRC errors that, if unmonitored, could indicate the interface is approaching a state where uncorrectable errors become likely. **SD Device Degradation:** Accumulation of errors associated with a specific SD device indicative of wear-out or early-life failure. **Assumptions of Use** 1. Error Counter Implementation The System Integrator shall implement a software error counter that is incremented each time AmbaSD_ReadSector() (or equivalent write API) returns a CRC error result code. This counter shall be maintained as a rolling window or periodic accumulator to measure the error rate rather than the cumulative total. 2. Threshold Definition and Safety Analysis The System Integrator shall define the maximum acceptable CRC error rate based on the system's safety analysis. This threshold shall distinguish between occasional transient bit-flips (tolerable) and a systematic interface failure (safety-relevant). The threshold and monitoring window shall be documented in the system's HSI. 3. Monitoring Period and DTTI Compliance The error rate monitoring window shall be set such that a degraded SD interface is detected and responded to within the system's DTTI. The monitoring routine shall execute as a periodic safety task on the Cortex-R52. 4. Error Handling and Safe State Upon exceeding the defined error threshold, the safety application shall log the event, cease relying on the SD device for safety-critical data reads, and if the SD device stores safety-critical data (e.g., calibration files or maps), transition the system to a Safe State or switch to a redundant storage source within the FTTI. **SDK API and integration flow:** The return value of AmbaSD_ReadSector() reflects CRC or other errors; the safety application monitors these results. Refer to the DDG document.

5.2.15 Software Watchdog Timer for VOUT

Name	*Description*
ID	SM_SW_WDT_VOUT
Description	A watchdog timer implemented by software to check if VOUT sends out the first frame sync after a pre-defined time.

	System Integrator is responsible for implementing the safety mechanism software to setup the watchdog timer and check if VOUT outputs the first frame in expected timeout value.
Ownership	System Integrator
SM is to against	Detect any random hardware fault that causes unexpected delay or hang
Software Related	Yes
Failure mode coverage	100%
Initialization	Timer should be enabled after VOUT is enabled and input data is ready
Operation	1. Software shall check if VOUT sends out the first frame sync after a pre-defined time, which is determined by the frame-rate, and must be less than DTTI (99ms). 2. After first frame, software could keep monitoring the time interval between VOUT frame sync and compare with a pre-defined time to detect hang or unintended frame rate drop.
Error indication	Assert safety error signal by software
Error detect time	99 ms

5.2.16 Software Watchdog Timer for Slave Controller

Name	*Description*
ID	SM_SW_WDT_SLAVE
Description	A watchdog timer implemented by software to check whether the safety mechanism for the slave controller, such as [AR-18] or [AR-19], has been executed periodically. See also 5.3.18 and 5.3.19 . System Integrator is responsible for implementing this safety mechanism software with the target system configuration.
Ownership	System Integrator

SM is to against	Detect any random hardware fault that causes hang.
Software Related	Yes
Failure mode coverage	100%
Initialization	Watchdog timer should be enabled after the slave controller is enabled.
Operation	Software shall check if the slave controller receives as least one predetermined data from an external master controller within 99 ms periodically.
Error indication	Assert safety error signal by software
Error detect time	99ms

5.2.17 DMA Interrupt Rate Checker

Name	*Description*
ID	SM_SW_QM_DMA_CHK
Description	Software to detect whether I2S/UART DMA request is stuck at 1. System Integrator is responsible for implementing the safety mechanism software.
Ownership	System Integrator
SM is to against	Stuck-at -1 fault on I2S/UART DMA request
Software Related	Yes
Failure mode coverage	100%
Initialization	N/A
Operation	• Software shall record the DMA interrupt rate in normal use case.

	• Software shall check the DMA interrupt rate in run time; if the interrupt rate is much higher than expected, software shall stop assigning the DMA channel to I2S/UART so that I2S/UART won't interfere with the DMA engine anymore. If needed, the software could assert safety error to indicate TX/RX REQ to DMA engine is stuck at 1.
Error indication	Assert safety error signal by software
Error detect time	99 ms

5.2.18 Software OTP-Data Consistency Check

Name	*Description*
ID	SM_SW_OTP_DCC
Description	Software to check whether amba_id is correct. System Integrator is responsible for implementing the safety mechanism software.
Ownership	System Integrator
SM is to against	Any random hardware fault in amba_id entry in OTP memory
Software Related	Yes
Failure mode coverage	99%
Initialization	N/A
Operation	• Software shall read amba_id after system boot up and compare the value stored in RCT. • If there is any CRC error, it will be caught by software reading amba_id.
Error indication	Assert safety error signal by software
Error detect time	99 ms

5.2.19 Software GPIO Check

Name	*Description*
ID	SM_SW_GPIO_CHK
Description	Software to periodically check the consistency of GPO and its routed-back GPI state (if in GPO mode), or check the consistency of main and shadow GPI state (if in GPI mode). System Integrator is responsible for implementing the safety mechanism software and integrating the final software to meet the safety requirement for the target system.
Ownership	System Integrator
SM is to against	stuck-at fault on GPIO
Software Related	Yes
Failure mode coverage	100%
Initialization	– If the tested GPIO is in GPO mode, one redundant GPIO pin should be designated as GPI and connected to the tested GPIO. – If the tested GPIO is in GPI mode, one redundant GPIO pin should be designated as GPI and received the same input source of the test GPIO.
Operation	– If the GPIO is in GPO mode, software shall periodically read the status of routed-back GPI to check the consistency of GPO and GPI state. – If the GPIO is in GPI mode, software shall periodically read the status of shadow GPI to check the consistency of main and shadow GPI state.
Error indication	Assert safety error signal by software
Error detect time	99 ms

5.2.20 Software GIC Checker

Name	*Description*
ID	SM_SW_GIC_CHK
Description	SM_SW_GIC_CHK is to detect if the interrupt is stuck at 1. Note that this cannot be detected if GPIO interrupt is stuck at 0 because the timing to assert interrupt for slave controller is not deterministic. System Integrator is responsible for implementing the software mechanism and integrating the final software.
Ownership	System Integrator
SM is to against	• Unused IRQ asserts. • Falsely triggered IRQ. • An IRQ not served. • Wrong IRQ priority. • An IRQ is distributed to wrong CPU cores.
Software Related	Yes
Failure mode coverage	99%
Initialization	• Configure required GIC channels. • Configure required peripherals for specific applications.
Operation	The following operations are recommended to detect the failure modes in GIC. • **Unused IRQ asserts** • On Cortex-R52: i. When an IRQ is asserted, the interrupt handler on Cortex-R52 shall get the IRQ ID and check if it is an enabled (valid) interrupt. Note that every interrupt enabled by SW on Cortex-R52 will be recorded as a valid interrupt. There are 2 information used to define a valid interrupt. • GIC register GICD_ISENABLER • A SW enabled flag on memory, Cortex-R52 interrupt enabling function shall set up a SW flag to mark this enabled interrupt. ii. If the interrupt is not enabled (not used), the interrupt handler on Cortex-R52 should notify the SW safety monitor on Cortex-R52 to handle. • On Cortex-A53: When boot-up, every interrupt shall hook to a handler

	function as its default ISR even it's not enabled. This handler function should stall target Cortex-A53 core in an infinite loop by default. Every device drivers on Cortex-A53 shall assign an interrupt handler to the corresponding interrupt before enabling it. • **Falsely triggered IRQ** This safety measure is delegated to be implemented at the device driver's interrupt handler. Following describes the approach: b. An interrupt is raised via GIC to trigger the peripheral interrupt handler on Cortex-A53. c. The interrupt handler reads interrupt status register to check if this asserted interrupt matches register content. d. If not, this is a "Spurious IRQ", the SW safety monitor on Cortex-R52 should be raised to handle. • **An IRQ not served** This safety measure is delegated to be implemented by each device driver handling this interrupt. A SW safety monitor on Cortex-R52 periodically reads out the status in share memory, which is written by device driver on Cortex-A53, to detect this failure mode. • For periodical interrupts (for example, VIN, VOUT), the IRQ handlers on Cortex-A53 shall write the timestamps into the shared memory for the safety monitor on Cortex-R52 to double check. • For peripherals that have transactional commands with a completion interrupt (for example, SPI/SSI), they shall implement software watchdog timeouts mechanism on Cortex-A53 to detect the missed IRQ event. Once detected, Cortex-A53 device driver should notify the safety monitor running on Cortex-R52. • **Wrong IRQ priority** e. On IRQ assertion, the interrupt handler on Cortex-A53 shall read GIC register to check current toggled interrupt. f. If a pending higher priority interrupt is found in status register out of the current interrupt, this is a wrong IRQ priority failure mode. g. On detection of this failure mode, the device driver on Cortex-A53 should notify the SW safety monitor running on Cortex-R52. • **An IRQ is distributed to wrong CPU core** h. On IRQ assertion, the interrupt handler on Cortex-A53 gets the IRQ ID and check if current core ID matches the configured target core ID for this interrupt handler. The device driver on Cortex-A53 should setup the right target core ID when initializing and store this information in memory as golden data. The interrupt handler on Cortex-A53 should check if current core ID is the target core ID stored in the shared memory data.

	i. On detection of this failure mode, Cortex-A53 device driver should notify the SW safety monitor running on Cortex-R52.
Error indication	Assert CEHU error pins by software
Error detect time	99 ms

5.2.21 Software Redundant Timer

Name	*Description*
ID	SM_SW_RT
Description	There are three INTERVAL TIMERS implemented in hardware. Each of INTERVAL TIMER contains 10 independent timers. If software calls one timer in INTERVAL TIMER for safety-related task, it should call another timer as the redundant timer for the same task and compare the timers' values periodically. The redundant timer can be counted in different precision compared to the main functional timer. System Integrator is responsible for implementing the safety mechanism in the system software.
Ownership	System Integrator
SM is to against	Any fault resulting in incorrect timer count
Software Related	Yes
Failure mode coverage	90%
Initialization	• Initialize functional timer configuration. • Initialize the same configuration for redundant one (if the counting clock is different in redundant timer, the threshold shall be converted to a corresponding value)
Operation	• SW shall use redundant timer's value to check if the functional timer's value is reasonable. • SW shall check if the timer's value is decreasing to detect common cause failure.

Error indication	Assert safety error signal by software
Error detect time	99 ms

5.2.22 Data Integrity Check With Checksum or Duplicate Copy

Name	*Description*
ID	SM_SW_CA53_DATA_CHK
Description	Safety-critical software units running on Cortex-A53 are required to double-check the correctness of critical data in memory with a verifier running on Cortex-R52. For example, using CRC; or using duplicate copies. System Integrator is responsible for implementing the safety software to compute and verify the data integrity of safety critical software executed by Cortex-A53.
Ownership	System Integrator
SM is to against	Data corruption: reading or writing of invalid data, or accessing unintended locations
Software Related	Yes
Failure mode coverage	60%
Initialization	Initialization is specific to each safety-critical software unit implementing data cyclic redundancy check
Operation	Software unit computes a checksum against protected data and stores it in memory on Cortex-A53. A verifier on Cortex-R52 computes the a checksum against the same protected data area and verifies that the checksum is identical to the one stored.
Error indication	Assert safety error signal by software
Error detect time	99 ms

5.2.23 L2 Cache Monitor

Name	*Description*
ID	SM_SW_CA53_L2_MON
Description	Periodic Cortex-A53 L2 cache self-diagnostic with Cortex-R52 verifier/monitor. System Integrator shall look for ARM's help to develop the safety software onCortex-A53 to detect the L2 cache failure of Cortex-A53.
Ownership	System Integrator
SM is to against	Permanent hardware fault in the L2 cache
Software Related	Yes
Failure mode coverage	99%
Initialization	N/A
Operation	Execute known memory access patterns to assess the performance of L1/L2 cache by reading bus/memory cycles as well as L1D/L2D accesses from the performance monitor unit.
Error indication	Assert safety error signal by software
Error detect time	99 ms

5.2.24 Livelock Check

Name	*Description*
ID	SM_SW_CA53_LIVE
Description	Safety-critical software units running on Cortex-A53 are required to implement self-checks against a monitor running on Cortex-R52 to make sure forward progress is no stuck. System Integrator shall look for ARM's help to develop the safety

	software on Cortex-A53 to detect the execution stuck on Cortex-A53.
Ownership	System Integrator
SM is to against	Unforeseen software stall
Software Related	Yes
Failure mode coverage	60%
Initialization	Initialization is specific to each safety critical software unit implementing livelock check.
Operation	Software watchdog timer or external monitor
Error indication	Assert safety error signal by software
Error detect time	99 ms

5.2.25 Instruction Set Monitor

Name	*Description*
ID	SM_SW_CA53_INS_MON
Description	Periodic Cortex-A53 comprehensive-instruction-set self-diagnostic routine with Cortex-R52 verifier / monitor. Safety-critical software units running on Cortex-A53 are required to perform redundant and diversified computations and have the results verified by a monitor running on Cortex-R52. System Integrator shall look for ARM's help to develop the safety software on Cortex-A53 to do full instruction set diagnostics on Cortex-A53.
Ownership	System Integrator
SM is to against	Any fault which leads to incorrect Arm instruction results

Software Related	Yes
Failure mode coverage	99%
Initialization	N/A
Operation	Systematic execution in categories of general, data transfer, floating-point, SIMD scalar, SIMD vector, and cryptographic instructions.
Error indication	Assert safety error signal by software
Error detect time	99 ms

5.2.26 Exception Monitor

Name	*Description*
ID	SM_SW_CA53_EXC_MON
Description	Periodic Cortex-A53 CPU exception self-diagnostic routine with Cortex-R52 verifier / monitor. System Integrator shall look for ARM's help to develop the safety software on Cortex-A53 to do the periodic Cortex-A53 CPU exception self-diagnostics and monitoring.
Ownership	System Integrator
SM is to against	Failure to process synchronous and/or asynchronous exceptions
Software Related	Yes
Failure mode coverage	99%
Initialization	N/A

Operation	Execute self-diagnostic to trigger synchronous and asynchronous exceptions and verify that those modes are entered and exited properly as expected.
Error indication	Assert safety error signal by software
Error detect time	99 ms

5.2.27 Performance Monitor

Name	*Description*
ID	SM_SW_CA53_PF_MON
Description	Periodic Cortex-A53 branch predictor self-diagnostic routine with Cortex-R52 verifier / monitor. Safety-critical software units running on Cortex-A53 are required to measure the timing of execution of critical functions for monitors running on Cortex-R52 to check that the measurements stay within predefined deadlines. System Integrator shall look forARM's help to develop the safety software on Cortex-A53 to detect the execution deadline failure on Cortex-A53.
Ownership	System Integrator
SM is to against	CPU performance loss due to branch prediction circuit not executing correctly
Software Related	Yes
Failure mode coverage	99%
Initialization	N/A
Operation	Execute known instruction pattern and capture performance monitor unit for CPU cycles, branch prediction speculative execution count, branch prediction mis-predicted count, and branch immediately executed count to verify they fall within specified thresholds.

Error indication	Assert safety error signal by software
Error detect time	99 ms

5.2.28 Software SRAM-ECC Self Diagnostic

Name	*Description*
ID	SM_SW_SRAM_DIAG
Description	Single bit permanent fault in bit-cells of SRAM with SECDED-ECC protection is latent fault. Software shall run test pattern on it if SRAM in Cortex-R52 and Cortex-A53 is software-accessible. System Integrator is responsible for implementing this self-diagnostic software.
Ownership	System Integrator
SM is to against	Permanent fault in bit-cells of software accessible SRAM with SECDED-ECC protection
Software Related	Yes
Failure mode coverage	99%
Initialization	For a software accessible SRAM with SECDED-ECC protection, software writes test pattern to all entries of the SRAM. If ECC engine reports data error corrected, software shall write test pattern again. If correctable error is reported again, then the correctable error should be regarded as permanent fault in SRAM bit-cell.
Operation	N/A (run during power up or power down)
Error indication	Assert safety error signal by software
Error detect time	99 ms

5.2.29 GPIO Register Read/Write Test

Name	*Description*
ID	SM_SW_GPIO_REG_RW
Description	Perform register read/write test to dummy bits of GPIO module to detect common cause failure on GPIO module's clock. System Integrator is responsible for implementing this safety mechanism software.
Ownership	System Integrator
SM is to against	Common cause failure on GPIO's clock.
Software Related	Yes
Failure mode coverage	90%
Initialization	N/A
Operation	Once software enables one GPIO module with GPI function, it shall take unused bits as dummy bits and do configuration register read/write test periodically to check if the clock works or not.
Error indication	Assert safety error signal by software
Error detect time	99 ms

5.2.30 Timer Read Test

Name	*Description*
ID	SM_SW_TMR_RD_TEST
Description	RCT timer is protected by lockstep. Software shall check if the timer starts counting if RCT timer is enabled. System Integrator is responsible for implementing the safety mechanism software.

Ownership	System Integrator
SM is to against	Common cause failure on the clock signal of RCT timers
Software Related	Yes
Failure mode coverage	99%
Initialization	N/A
Operation	Periodically check if the timer is counting and if RCT timer is in use.
Error indication	Assert safety error signal by software
Error detect time	99 ms

5.2.31 Configuration SRAM Redundancy Information

Name	*Description*
ID	SM_SW_CFG_SRAM_RI
Description	Perform data read/write to SRAM in scratchpad with predefined redundant information to detect common-cause failure on SRAM's clock. System Integrator is responsible for implementing the safety mechanism for this requirement.
Ownership	System Integrator
SM is to against	Common-cause failure on clock of SRAM in scratchpad
Software Related	Yes
Failure mode coverage	100%
Initialization	N/A

Operation	Software shall add the predefined redundant information (like CRC or similar) to the data written into the SRAM in scratchpad. • The redundant information is encoded by the software before programming data into the SRAM. • The redundant information shall be decoded and checked by the software before using the data read from the SRAM.
Error indication	Assert safety error signal by software
Error detect time	99 ms

5.2.32 Test WDT Timeout alarm to CEHU as startup test

Name	*Description*
ID	SM_SW_PO_WDT_CHK
Description	To test WDT timeout alarm to CEHU correctly.
Ownership	System Integrator
SM is to against	Any fault on the paths between WDT and CEHU
Software Related	Yes
Failure mode coverage	60%
Initialization	In startup test, set a timeout threshold to WDT and wait until it timeouts, and read CEHU safety error status to confirm whether WDT timeout event occurred.
Operation	N/A
Error indication	Keep in safe state
Error detect time	Power-on test

5.2.33 DSP Health and Data Plausibility Monitor

DSP Health and Data Plausibility Monitor	
ID	SM_SW_DSP_HEALTH_MON
Description	This safety mechanism employs a hierarchical approach to ensure the integrity of DSP data. The Cortex-A53 acts as a "Data Aggregator and Checker," while the Cortex-R52 acts as the "Safety Gatekeeper." This setup detects not only the "Alive" status of the DSP and A53 but also the logical plausibility of the DSP data before it reaches the safety-critical layers.
Ownership	Ambarella and System Integrator
SM is to against	Loss of DSP/A53 liveness (hang or crash) and implausible or incorrect DSP output data reaching the safety-critical layers.
Software Related	Yes
Error indication	Reported to the user application through the SDK API; refer to the API design document.
Error detect time	99 ms
SDK Usage	This safety mechanism employs a hierarchical approach to ensure the integrity of DSP data. The Cortex-A53 acts as a "Data Aggregator and Checker," while the Cortex-R52 acts as the "Safety Gatekeeper." This setup detects not only the "Alive" status of the DSP and A53 but also the logical plausibility of the DSP data before it reaches the safety-critical layers. **Detected Error Types:** DSP Data Implausibility: The A53 detects that DSP output values are out-of-range, inconsistent, or physically impossible based on defined safety rules. A53 Processing Failure: The A53 stops sending the heartbeat, indicating a hang, OS crash, or failure in the plausibility logic. DSP-to-A53 Communication Error: Failures in the Message Dispatcher between the DSP and A53 that prevent data from being validated. Temporal Violations: The R52 detects that the A53 heartbeat is delayed, indicating an overload or task starvation on the A53 application processor. **Assumptions of Use** 1. A53 Plausibility and Heartbeat Generation The System Integrator shall implement a safety-related task on the Cortex-A53 that: Consumes DSP Messages: Receives data via the Message Dispatcher.

	Performs Plausibility Checks: Validates the DSP data against application-specific bounds (e.g., size of frame data, frame timestamp, or frame number consistency). Signs the Heartbeat: Only if the DSP data is deemed plausible, the A53 shall generate a heartbeat message. If the DSP data is implausible, the A53 must either stop the heartbeat or set an "Error" flag within the heartbeat payload. 2. R52 Safety Monitor Implementation The System Integrator shall implement a windowed watchdog monitor on the Cortex-R52. This monitor must: Supervise the A53: Verify that the A53 heartbeat arrives within the specified time window. Verify Health Payload: Inspect the heartbeat message for any error flags reported by the A53's internal plausibility checks. 3. Direct Error Reporting to CEHUThe System Integrator shall configure the Cortex-R52 to trigger the Central Error Handling Unit (CEHU) immediately if:The A53 fails to send a heartbeat (Total system failure).The A53 sends a heartbeat containing a "DSP Implausible" or "Internal Error" status.The heartbeat arrives with an invalid sequence counter (Data corruption). 4. Failure Response and FTTI The System Integrator must ensure the total time—from a DSP error occurring to the R52 is within the system's Fault Tolerant Time Interval (FTTI). Safe State: Upon detection, the R52 should be capable of forcing the system into a safe state (e.g., ignoring following frames or resetting the system). 5. Task Priority and Resource IsolationThe System Integrator shall ensure that the heartbeat generation task on the A53 is assigned a high priority. On the R52, the monitor must run in a high-priority, deterministic execution environment. **SDK API and integration flow:** No configuration is required to enable this monitor; it runs inside the image framework. The System Integrator retrieves the health report through the generic Image Main interface AmbaImgMain_ConfigEx(UINT32 VinId, UINT64 Param, UINT64 Param2), which returns the software counter of the task run cycle (image-task liveness) together with the reported DSP/A53 health and plausibility status. If a fault is detected, the System Integrator deactivates the image framework via AmbaImgMain_Config(UINT32 VinId, UINT64 Param) using LIVEVIEW_CMD_STOP, SYNC_CMD_STOP and STATISTICS_CMD_STOP.

5.2.34 Software Frame Monitor

Software Frame Monitor	
ID	SM_SW_FRAME_MON

Description	The Software Frame Monitor is a safety-critical software module that provides temporal supervision for periodic data streams. It identifies timing violations at the software level and reports these anomalies to the Cortex-R52. This mechanism is essential for detecting "Late" or "Lost" data that could lead to stale information being used by safety algorithms.Detected
Ownership	Ambarella and System Integrator
SM is to against	Timing faults in periodic DSP/VIN frame streams (late, lost, jittered, or out-of-rate frames) that could cause stale data to be used by safety algorithms.
Software Related	Yes
Error indication	Reported to the user application through the SDK API; refer to the API design document.
Error detect time	99 ms
SDK Usage	**Frame Monitor** The Software Frame Monitor is a safety-critical software module that provides temporal supervision for periodic data streams. It identifies timing violations at the software level and reports these anomalies to the Cortex-R52. This mechanism is essential for detecting "Late" or "Lost" data that could lead to stale information being used by safety algorithms.Detected Error Types: Frame Timeout: The elapsed time since the last valid frame exceeds the safety threshold, indicating a producer hang or transport blockage. Temporal Jitter: Arrival times fluctuate beyond the allowed tolerance, indicating non-deterministic scheduling in the data source (e.g., the DSP or A53). High-Frequency Violations: Frames arrive too quickly, potentially indicating a logic error in the frame dispatcher. Inconsistent Periodicity: Detection of an unstable frame rate that may compromise the control loop stability. **Assumptions of Use** 1. R52 Error Reception and Logic The System Integrator shall implement a dedicated error-handling routine on the Cortex-R52 to receive and process error messages from the Software Frame Monitor. Since the monitor does not trigger the CEHU directly, the R52 software must be the "Decision Maker." The integrator must ensure that the communication path between the Frame Monitor and the R52 is reliable and has a deterministic latency. 2. Software-Based Error Reporting (Dispatcher Integration) The System Integrator shall utilize the Message Dispatcher to route error packets from the Frame Monitor to the R52. This error message must be treated with the highest priority to

	ensure that the R52 is notified of the timing violation before the next safety-critical processing cycle begins. 3. Application-Specific Error Handling Upon receiving a timing error from the Frame Monitor, the R52 software shall execute the following:Data Invalidation: Mark the current data frame associated with that timing error as "Invalid" to prevent downstream consumers from using corrupted or stale data.Counter-Measure Execution: Based on the system safety analysis, the R52 shall decide whether to attempt a re-synchronization, request a re-transmission, or trigger a local safety state. 4. Safety State Transition (R52-Led) The System Integrator shall ensure that if the Software Frame Monitor reports persistent errors (exceeding a user-defined threshold), the R52 software initiates a transition to a Safe State. Since the CEHU is not automatically triggered, the R52 must have the authority to command other system components (e.g., the A53 or external actuators) to enter a safe mode. 5. FTTI Management The System Integrator must verify that the cumulative time for:The Frame Monitor to detect the timeout.The error message to be dispatched and received by the R52.The R52 to execute the safety reaction....all fit within the Fault Tolerant Time Interval (FTTI) of the specific Safety Goal. **SDK API and integration flow:** No configuration is required to enable this monitor; it runs inside the image framework. The System Integrator retrieves the health report through the generic Image Main interface AmbaImgMain_ConfigEx(UINT32 VinId, UINT64 Param, UINT64 Param2), which returns the software timing-checker results and the sync-control FIFO status for the periodic DSP/VIN frame streams. If a timing fault is detected, the System Integrator deactivates the image framework via AmbaImgMain_Config(UINT32 VinId, UINT64 Param) using LIVEVIEW_CMD_STOP, SYNC_CMD_STOP and STATISTICS_CMD_STOP.

5.3 Assumption-of-Use (AoU)

An assumption of use (AoU) is a safety requirement with any of the following attributes, and hence needs system integrator to take special case.

- Assumption made of the system design and manufacturing.
- Assumption made of the external element(s) or software, which is not for diagnostic
- Assumption made of the software operation which depends on the external element(s).
- Assumption made of the software that Ambarella cannot provide SDK or reference code for.

The following table shows the template used to describe the AoUs in the following sub-sections.

Description	*The description for this assumption of use.*		
Failure Mode Coverage	*The failure mode to be covered by this AoU. This field is applicable only if the AoU is for diagnostic.*		
Safe State	*The recommended method to signal the safe state. This field is applicable only if the AoU is for diagnostic.*		
ASIL	*The target ASIL rating of this AoU*	DTTI	*The recommended Diagnostic Test Time Interval. This field is applicable only if the AoU is for diagnostic.*

5.3.1 [AR-01] Monotonic counter, write_lock, and data_invalid fields in OTP shall be written in the reverse bit order to the other entry

Description	The monotonic counter field of OTP may be updated from time-to-time, and therefore cannot be protected by CRC. SW shall program monotonic counter in one entry, and program in another entry with the reverse bit order. Whenever software reads this entry, it shall compare the value of this entry with the other entry with the reverse bit order for data integrity. The same mechanism shall be applied to write_lock and data_invalid fields in OTP.		
Failure Mode Coverage	To cover any fault that causes wrong data in monotonic counter, write_lock, and data_invalid fields in OTP with at least medium effectiveness.		
Safe State	Assert safety error signal by software		
ASIL	C	DTTI	99ms

5.3.2 [AR-02] System developer shall implement a measure on the system level to monitor the operating temperature of the SEooC

Description	This is derived from CON-12. System developer must ensure proper thermal design on system level to detect if the CV2A operating temperature stays within the datasheet limit.		
Failure Mode Coverage	To detect any fault that causes the temperature of the SEooC out of range with at least medium effectiveness.		
Safe State	Switch off all power rails of the SEooC		
ASIL	C	DTTI	99ms

5.3.3 [AR-03] Software shall use Cortex A53 STL to cover main processor and pipelines

Description	Software test libraries (STL) is provided by ARM® for additional diagnostic coverage on main processor and pipelines of Cortex A53.		
Failure Mode Coverage	To detect permanent fault in main process and pipelines of Cortex A53 with low effectiveness.		
Safe State	Assert safety error signal by software		
ASIL	C	DTTI	99ms

5.3.4 [AR-04] Safety Error pins of CEHU shall be pulled down by external resistors

Description	The safety error indicators of the SEooC are active-low and shall be pulled down externally to deal with the fault that the bonding wires of the safety error pins are open so that CEHU cannot derive the safety error status to the SYSTEM.
ASIL	C

5.3.5 [AR-05] 24 MHz clock with 50 ppm tolerance shall be provided as the reference clock of the SEooC

Description	The 24 MHz reference clock is the clock source of the internal PLL. The system integrator shall ensure the correctness and the accuracy of the reference clock so that the generated clocks run at the expected frequency.
ASIL	C

5.3.6 [AR-06] 32.768 kHz clock with 100 ppm tolerance shall be provided as the RTC clock of the SEooC

Description	The 32.768 kHz RTC clock is used as reference clock to cross check the accuracy of the 24 MHz reference clock.
ASIL	C

5.3.7 [AR-07] JTAG interface shall be pulled to inactive state externally

Description	According to CON-3, debug mode must not be used during a safety related application. To ensure this, JTAG_RST_L must be pulled down externally. JTAG_CLK, JTAG_TDI and JTAG_TMS must be pulled up externally.
ASIL	C

5.3.8 [AR-08] Only video sources (sensors) that support MIPI interface with ECC and CRC functions, and a reasonable level of internal safety mechanism shall be used, and these functions shall be enabled when SEooC is used to capture the video input

Description	ECC and CRC functions in video sources (sensors) are assumed to be enabled to detect any fault causing data error on the video input interface.
ASIL	C

5.3.9 [AR-09] If the external video source (sensor) uses V/H-sync from SEooC, then the video source (sensor) shall use MIPI interface with ECC and CRC functions, and shall communicate the V/H-sync acknowledgement flag back to SEooC as part of the embedded data

Description	SW shall check the V/H-sync flag captured by VIN to make sure the sensor is receiving V/H-sync from IDSP correctly.
ASIL	C

5.3.10 [AR-10] The external video sources (sensors) shall have a measure against faults in their clock and supply inputs

Description	The system developer shall implement clock and voltage monitors on the video source (sensor).
ASIL	C

5.3.11 [AR-11] If a CAN controller is connected to the CAN bus, SW shall add redundant information to check data integrity

Description	To provide end-to-end data protection on the CAN bus, redundant information, such as CRC, shall be added in the data field of data packets. This safety measure can also help on detecting latent fault of error checking logic in CAN controller.		
Failure Mode Coverage	To detect faults in TX/RX path of CAN interface with at least medium effectiveness.		
Safe State	Assert safety error signal by software		
ASIL	C	DTTI	99ms

5.3.12 [AR-12] If a display panel is connected to the element via MIPI-DSI interface, it shall be able to report error status according to MIPI D-PHY specification v1.1

Description	To support the safety mechanism, SM_HW_ERPT, the external receiver of MIPI-DSI shall be able to report the error status of the transmitted data according to MIPI D-PHY specification v1.1.
ASIL	C

5.3.13 [AR-13] A power monitor for AVDD18 power supply shall be implemented in the system level

Description	The system developer shall implement an external power monitor for AVDD18 power supply to ensure the correctness of BiasGen function and internal power monitors.		
Failure Mode Coverage	To cover any fault in AVDD18 power supply with at least medium effectiveness		
Safe State	Power all power rails of the SEooC off.		
ASIL	C	DTTI	99ms

5.3.14 [AR-14] Initialization is necessary before safe access to DDR if it's possible to read from uninitialized DDR address

Description	This AoU is to avoid false alarm from ECC Decoder. All data stored in external DDR device are assumed to be protected by ECC. If any DRAM client reads an uninitialized DDR address, the returned read data will be unknown. A DRAM BIST and Scrubbing Engine (DBSE) is implemented in the SEooC. Formatting DDR can be achieved by setting DBSE_MODE to 0 and DBSE_BIST_PATTERN to 0x1 in DBSE module.
ASIL	C

5.3.15 [AR-15] If a CAN controller is connected to the CAN bus, SW shall send a packet periodically to detect permanent fault on CAN interface

Description	This is to detect permanent fault on CAN-TX and RX paths, which are connected to external CAN transceiver. SW shall have packets exchange with other nodes periodically with a pre-define protocol or mechanism to detect the permanent fault on CAN interface. Such "protocol" would be designed and decided by application of end-customers.		
Failure Mode Coverage	To cover DC fault model on CAN interface to IOPAD with at least medium effectiveness.		
Safe State	Assert safety error signal by software		
ASIL	C	DTTI	99ms

5.3.16 [AR-16] SW shall perform read test to external I2C slave after read/write operation to detect any fault that occurs on the I2C interface.

Description	This is to make sure I2C master in the SEooC is able to access the external slave device correctly. Software reads some predetermined data from the external I2C slave device after any read/write operation to check if the read data is as expected.		
Failure Mode Coverage	To cover any fault that causes wrong data transmission via I2C master with at least medium effectiveness.		
Safe State	Assert safety error signal by software		
ASIL	C	DTTI	99ms

5.3.17 [AR-17] SW shall perform read test to external SSI slave after read/write operation to detect any fault occurred on SSI interface.

Description	This is to make sure SSI master in the SEooC is able to access the external slave device correctly. Software reads some predetermined data from the external SSI slave device after any read/write operation to check if the read data is as expected.		
Failure Mode Coverage	To cover any fault that causes wrong data transmission via SSI master with at least medium effectiveness.		
Safe State	Assert safety error signal by software		
ASIL	C	DTTI	99ms

5.3.18 [AR-18] SW shall use checksum communication to detect any fault occurred on I2C slave interface.

Description	This is to detect any fault occurred on I2C slave interface. It's assumed that the external master devices shall send the data packets with redundant information (such as CRC, or similar one). The chosen redundant information technology must be robust enough to ensure at least 90% diagnostic coverage. Once the packet is received by I2C slave, software shall read data via APB bus and check the correctness of the redundant information before using data in safety-related information.		
Failure Mode Coverage	To cover any fault occurred on I2C slave interface with at least medium effectiveness.		
Safe State	Assert safety error signal by software		
ASIL	C	DTTI	90ms

5.3.19 [AR-19] SW shall use checksum communication to detect any fault occurred on SSI slave interface.

Description	This is to detect any fault occurred on SSI slave interface. The external SSI master transmits data with predetermined checksum mechanism to the SSI slave controller and software shall verify if the checksum is correct.		
Failure Mode Coverage	To cover any fault occurred on SSI-slave interface with at least medium coverage.		
Safe State	Assert safety error signal by software		
ASIL	C	DTTI	90ms

5.3.20 [AR-20] If data to be stored in NAND device is safety-related, it shall be protected by a predefined redundant information.

Description	This is to provide end-to-end data protection between DRAM and NAND device. System shall add the predefined redundant information (like CRC, or similar one) for the safety-related message stored in the external NAND device. The redundant information is encoded by software before programming data into the storage. The redundant information shall be decoded and checked by software before using the read data for the safety-related application.		
Failure Mode Coverage	To detect any fault that causes wrong data transfer between DRAM and the external NAND flash with high effectiveness.		
Safe State	Assert safety error signal by software		
ASIL	C	DTTI	99ms

5.3.21 [AR-21] If data to be stored in SD device is safety-related, it shall be protected by a predefined redundant information.

Description	This is to provide end-to-end data protection between DRAM and SD device. This safety measure can also be used to detect latent fault of CRC decoder in SD controller. System shall add the predefined redundant information (like CRC, or similar one) for the safety-related message stored in the external SD/eMMC device. The redundant information is encoded by software before programming data into the storage. The redundant information shall be decoded and checked by software before using the read data for the safety-related application.		
Failure Mode Coverage	To detect any fault that causes wrong data transfer between DRAM and the external SD/eMMC device with high effectiveness.		
Safe State	Assert safety error signal by software		
ASIL	C	DTTI	99ms

5.3.22 [AR-22] If data to be stored in SPI NOR device is safety-related, it shall be protected by a predefined redundant information.

Description	This is to provide end-to-end data protection between DRAM and SPI-NOR device. System shall add the predefined redundant information (like CRC, or similar one) for the safety-related message stored in the external spi-NOR device. The redundant information is encoded by software before programming data into the storage. The redundant information shall be decoded and checked by software before using the read data for the safety-related application.		
Failure Mode Coverage	To detect any fault that causes wrong read data from SPI-NOR device with at least medium effectiveness.		
Safe State	Assert safety error signal by software		
ASIL	C	DTTI	99ms

5.3.23 [AR-23] An external MCU with an independent clock source shall be implemented if TLSS-2 is chosen to represent the safe state of SEooC

Description	This is derived from TLSS-2 and CON-11 to avoid common cause failure on the clock signal of the external MCU.
ASIL	C

5.3.24 [AR-24] GPI shall be duplicated for safety-related general purpose input

Description	It is possible that incorrect GPI data is received from external device due to faults on GPI pins. For safety critical input, GPI shall be duplicated and receive the same input from external device. Then, SW shall compare whether the GPI data from the two input pins are the same or not.
ASIL	C

5.3.25 [AR-25] GPO shall be routed back to GPI for safety-related general purpose output

Description	It is possible that incorrect GPO data is transmitted to external device due to faults on GPO pins. For safety critical output, GPO shall be routed back to GPI. Then, SW shall compare whether the transmitted output data to GPO and the received input data from routed GPI consist.
ASIL	C

5.3.26 [AR-26] MMC, SD, or SDIO device shall be connected to SD or SDIO controller as embedded

Description	Connecting MMC or SDIO devices as embedded helps to reduce compatibility issues.
ASIL	C

5.3.27 [AR-27] SW shall move BST to TCM and check data integrity before the execution

Description	NAND controller is QM in HW. If the CPU uses the SRAM in NAND controller during the BST stage to execute boot code, any fault causing wrong data in SRAM may lead to a corrupt, but successful boot procedure resulting in an undetected unsafe state. Therefore, SW must move the BST code to TCM in Cortex-R52 before start execution to achieve end-to-end protection.
ASIL	C

5.3.28 [AR-28] If TLSS-2 is chosen to represent the safe state of SEooC, the external MCU shall check the consistency of two safety error pins from CEHU

Description	This is derived from TLSS-2. The SEooC contains two copies of CEHU with separate safety error pins. If TLSS-2 is chosen to represent the safe state of the SEooC, the external MCU shall check the consistency of the safety error pins to detect the random hardware fault in CEHU and the safety error pins.		
Failure Mode Coverage	To cover any fault in CEHU and the safety error pins with at least medium effectiveness.		
Safe State	Reset or power down the SEooC		
ASIL	C	DTTI	1ms

5.3.29 [AR-29] An external watchdog timer with an independent clock source shall be implemented if TLSS-4 is chosen to represent the safe state of SEooC

Description	This is derived from TLSS-4 and CON-11 to avoid common cause failure on the clock signal of the external watchdog timer.
ASIL	C

5.3.30 [AR-30] Customer shall verify that all software run on Cortex and VP perform correctly and within accepted ASIL during development and deployment

Description	If the software running on Cortex or VP are incorrect, HW has no way of detecting or preventing this. It's user's responsibility to verify that the software performs correctly. This verification may include the use of divergent and redundant algorithms or performance monitors in software.
ASIL	C

6 System-Level Integration

6.1 Safety Related External Components

6.1.1 External Power Supply

The on-chip power monitors take AVDD18 as the reference voltage to monitor all the other power supplies to the SEooC. In case of a detected out-of-spec external power supply condition, the SEooC triggers an error reaction as specified in Section 5.1.1 .

As described in Section 5.3.12 , the external power monitor shall be implemented to monitor AVDD18 power supply.

The following figure shows a conceptual view of the connection with the external power supplies.

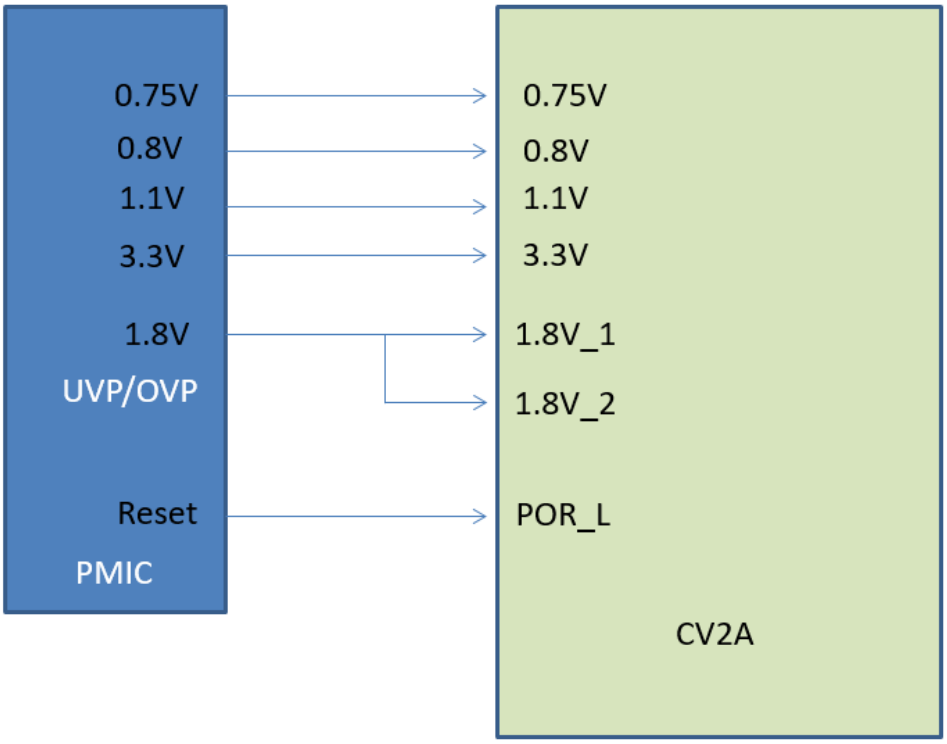


Figure 5 Conceptual View of the Connection with External power Supplies

6.1.2 External Watchdog

An external watchdog timer is expected to continuously monitor the SEooC. This external watchdog timer serves two purposes:

1. Initialization of a system safe state, if the SEooC detects an error and indicates it through one of the following assumed top-level safe state (TLSS).
   - TLSS-1: Error indication by forcing power down or cold reset.

- TLSS-2: Error indication by dedicated safety error pins.
- TLSS-4: Error indication by deactivating an external watchdog timer interface

2. Independent detection of certain failure modes that are not detected by the SEooC (e.g. frozen SEooC) and initiation of a system safe state.

Note that the dedicated error output (TLSS-2) can either be processed by the external watchdog timer or by another external MCU that is capable to initiate the system safe state.

Dependent failures between the SEooC and the external watchdog timer or MCU must be avoided. For that an independent clock source is mandatory (CON-11). Other common cause initiators shall be reviewed in a system-level dependent failure analysis.

The following figure shows a conceptual view of the connection with the external watchdog timer.

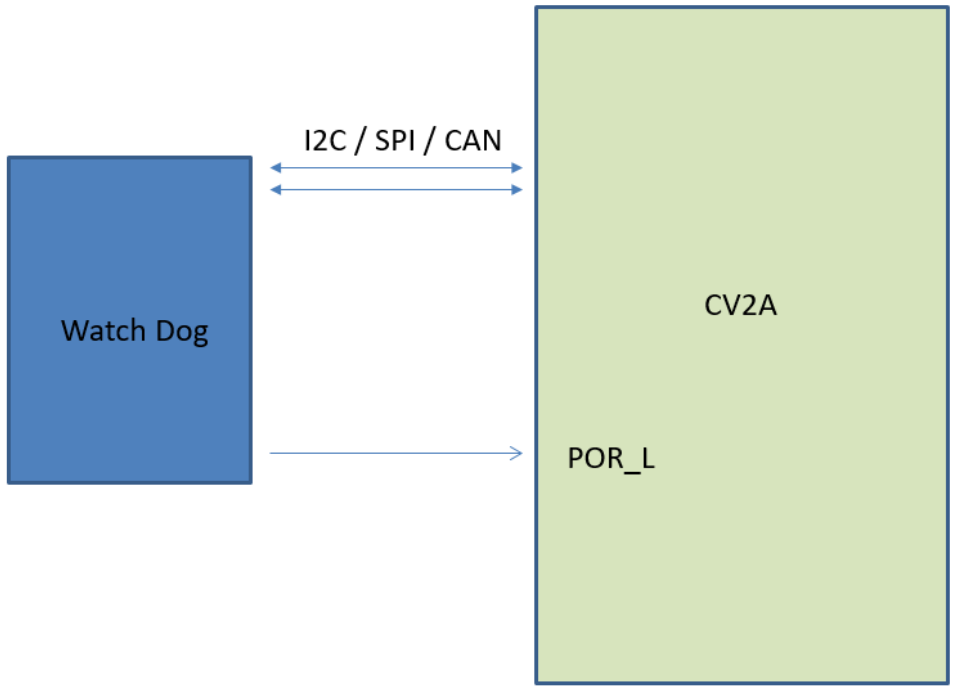


Figure 6 Conceptual View of the Connection with External Watchdog Timer

6.1.3 Allocation of Functions among the SEooC and other Involved Elements

The following diagram shows how SEooC should be connected - specifically safety-related connections.

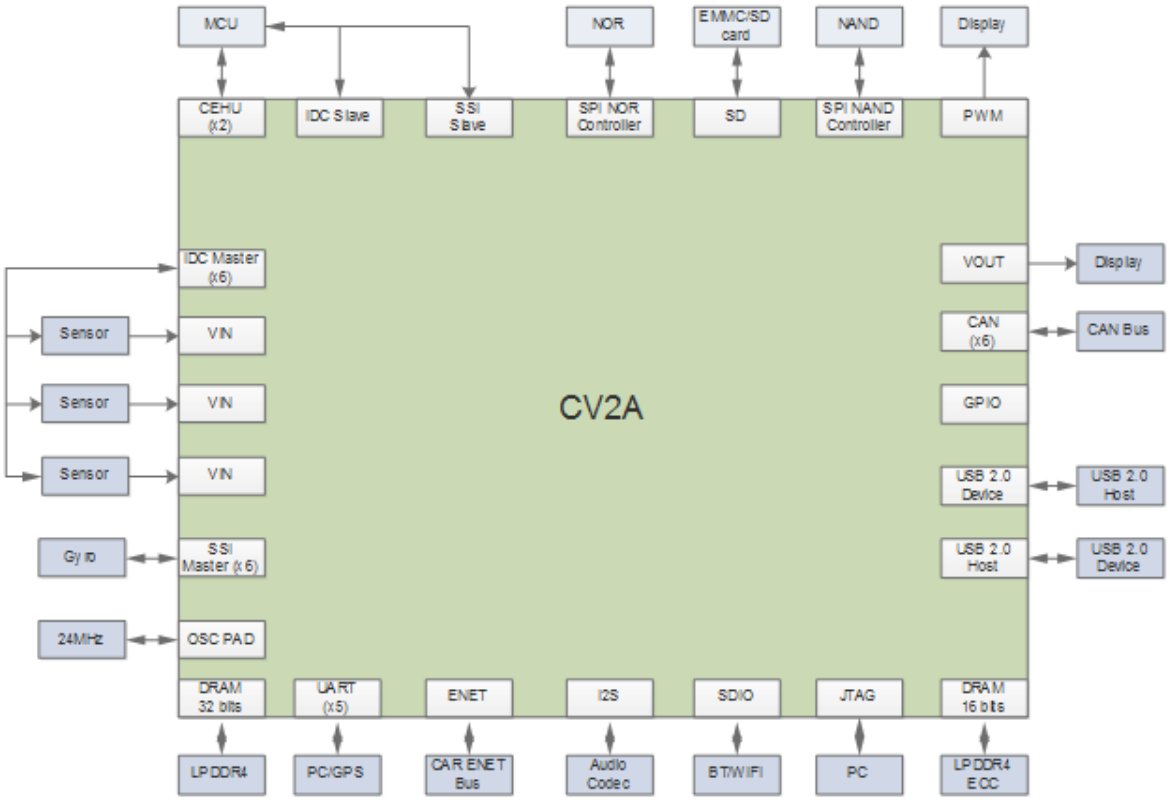
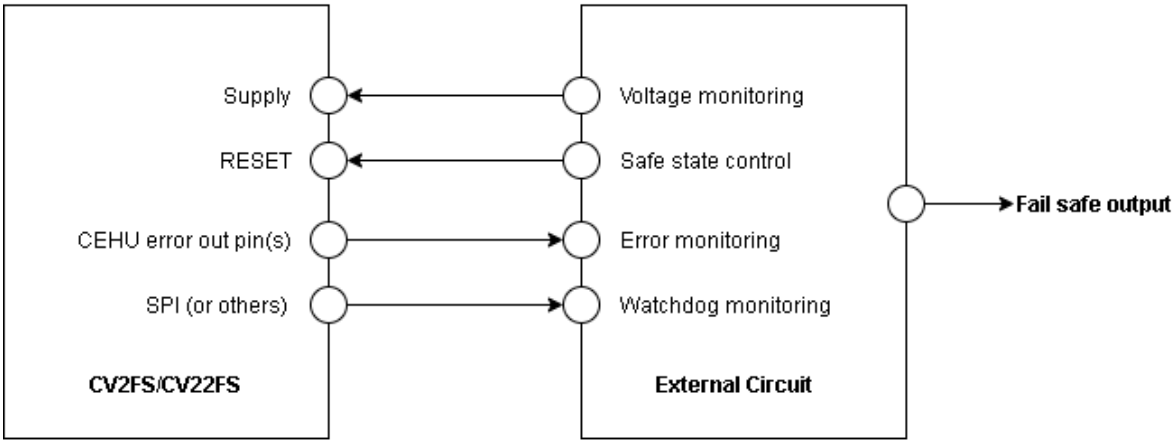


Figure 7 Examples of Integrating SEooC Into a System

Note:

The SEooC supports both USB 2.0 host and device interfaces, but only one of them can be enabled at a time.

There is no separated power and clock for R52 cores.
Following diagram shows the proposed external circuit.



CV2FS/CV2FS integrated safety concept enables SPF and LF to be detected with high diagnostic coverage. However not all CMF maybe detected. As a result, in order to detect failures that may not be detected by CV2FS/CV2FS, it is assumed that there will be some separate means to bring the system into safe state.

The following are the assumed functions by the external circuit:
1. Power supply monitoring
2. Error out monitoring (reading CEHU ERR pins)
3. External watchdog (via SPI, I2C, or CAN)

Figure 8 Integrating SEooC into a System

For details on the power supply for the SEooC and such, refer to Section 5.3.1 in the SEooC document.

7 Safety Analysis

This chapter summarizes the safety analysis performed for CV2FS, CV22FS and CV2FSA including the results that are relevant for the integrator and user.

7.1 Failure Modes, Effects, and Diagnostic Analysis (FMEDA)

An FMEDA has been performed for CV2FS, CV22FS and CV2FSA, as a quantitative analysis of random hardware faults, according to ISO 26262-5:2018 clause 8 and 9. For the evaluation of safety goal violations due to random hardware failures, the PMHF method (ISO 26262-5, clause 9.4.2) has been applied. The conditions and results of this analysis are summarized in the following sub-chapters. A more detailed FMEDA extract is available under NDA.

Note: The numbers may need to be recalculated based on the temperature profile.

7.1.1 Sources of Permanent Hardware Failure Rates

Failure rates for permanent faults in the SEooC have been estimated based on ISO 26262-11:2018, clause 4.6.2.1.1 (former IEC/TR 62380:2004). Failure rates are calculated separately for silicon die and package.

Failure rates have been calculated for the “Passenger Compartment” mission profile (including temperature profile and temperature cycling information) from ISO 26262-11:2018 (clause 4.6.2.1.1, Figure 16) or IEC/TR 62380:2004 (Section 5.8.3, Table 11).

Note: To avoid an artificial failure rate reduction, the failure rate calculation is based on operating time only, without consideration for OFF-time. This is achieved by forcing $\tau_{off} = 0$. If the resulting failure rates are used in a system-level fault tree analysis (FTA), they must therefore be integrated over operating time instead of total lifetime.

Attention: The system integrator must adjust the failure rates for the SEooC to the actual mission profile of the target application. The changed temperature and cycling conditions are expected to result in different permanent base failure rates, which may also affect the FMEDA results. Contact the designated sales representative to request for help on this task.

Note that the number may need to be updated based on customer's temperature profile. To avoid an artificial rate reduction, OFF time is suppressed.

7.1.2 Source of Transient Hardware Failure Rate

Failure rates for transient faults in the SEooC’s embedded memories and other memory elements (flip-flops, latches) are based on foundry measurement data:

Samsung's foundry data for 10 nm low power plus (10LPP) process.

7.1.3 FMEDA Results

The table below summarizes the FMEDA results for "Passenger Compartment" mission profile, assuming that all safety mechanisms and assumptions-of-use are implemented and used as described in Chapter 5.

Table 2 FMEDA Results for CV2FS/CV22FS (ASIL C)

Types	**SPFM**	**LFM**	**λ_{RF}**	**$\lambda_{DFP,\ latent}$**
Silicon die – Permanent faults	97.01%	96.85%	1.43 FIT	1.46 FIT
Silicon die – Transient faults	99.64%	N/A	5.41 FIT	N/A
Package – Permanent faults	99.41 %	97.43%	1.74 FIT	7.56 FIT
Total	-	-	**8.58 FIT**	**9.02 FIT**

Table 3 FMEDA Results for CV2FSA (ASIL C)

Types	**SPFM**	**LFM**	**λ_{RF}**	**$\lambda_{DFP,\ latent}$**
Silicon die – Permanent faults	97.01%	96.85%	1.43 FIT	1.46 FIT
Silicon die – Transient faults	99.64%	N/A	5.41 FIT	N/A
Package – Permanent faults	99.38 %	97.78%	1.83 FIT	6.53 FIT
Total	-	-	**8.67 FIT**	**7.99 FIT**

7.1.4 Top-level Failure Modes and Failure Modes Distribution

All faults in the SEooC have been mapped to effects on top-level, which are the top-level failure modes (TLFM).

The following table shows the distribution of residual failure rates among the potential top-level failure modes for the SEooC.

Table 4 Top-Level Failure Modes

Failure Mode	Description	Silicon Permanent (1.43 FIT)	Silicon Transient (5.41 FIT)	Package (1.74 / 1.83 FIT)
[TLFM-01]	Erroneous image signal processing	14%	16%	32%
[TLFM-02]	Erroneous video encoding and decoding	8%	12%	13%
[TLFM-03]	Erroneous CV flow processing	41%	36%	13%
[TLFM-04]	Erroneous real-time data processing and interrupt handling	9%	16%	13%
[TLFM-05]	Erroneous on-chip data communication and storage	5%	6%	0%
[TLFM-06]	Erroneous configuration	4%	4%	0%
[TLFM-07]	Erroneous I/O communication	14%	6%	18%
[TLFM-08]	Unexpected performance degradation	3%	3%	0%
[TLFM-09]	Dependent failure between safety function and safety mechanism	3%	0%	11%

Note: The failure rate distribution for package does not change significantly from CV2FS to CV2FSA, despite the change in the absolute residual failure rate value.

7.2 Design Failure Modes and Effects Analysis (DFMEA)

A design FMEA has been performed for the critical modules in CV2 ASIL, to identify and analyze potential failure modes of the SEooC blocks and sub-blocks. This DFMEA focuses on systematic errors, their root causes and effects, complementing the analysis of random hardware faults in the FMEDA.

For failure modes that were identified to have a high risk / high priority (RPN>100), measures were defined to reduce these risks to an acceptable low level (RPN≤100). While most of these measures have been implemented or realized during the SEooC development project, some are left as requirements to be fulfilled by the integrator or user of the SEooC.

7.3 Dependent Failure Analysis (DFA)

A DFA has been performed for CV2FS / CV22FS / CV2FSA, to identify single events or single causes that could bypass or invalidate a required independence or freedom from interference between given elements, and thereby lead to the violation of a safety requirement.

The following architectural features have been considered (ref. ISO 26262-9 clause 7.2):

- Similar and dissimilar redundant elements
- Different functions implemented with identical hardware elements
- Functions and their respective safety mechanisms
- Physical distance between hardware elements, with or without barrier
- Common external resources

The following dependent failure initiators have been considered (ref. ISO 26262-9 clause 7.4.4):

- Random hardware failures
- Development faults
- Manufacturing faults
- Installation faults
- Environmental factors
- Failures of common external resources or information
- Stress due to specific situations
- Aging and wear

The DFA has shown that sufficient prevention and detection measures are in place to avoid or control dependent failures. Most of these measures have been implemented or realized during the SEooC development project, but some are left as requirements to be fulfilled by the integrator or user of the SEooC.

Please refer to the DFA report *CV2A_DFA_spreadsheet.xlsx* [D6854_0].

8 Development Lifecycle

8.1 About Ambarella's Development Cycle

The Ambarella ASIL SoC development lifecycle is based on an ISO 9001 certified quality management system. Ambarella is conscientious in merging ISO 26262 processes to address functional safety requirements.

Project initiation starts with a product requirement description (PRD) document from marketing, which is the basis of the context and customer requirements. Feasibility and risk analysis are conducted before the kickoff review to confirm the project. Ambarella executes development interface agreements (DIAs) with key suppliers so it can have supplier management under control.

Detailed planning follows immediately at both general project level and functional safety level to ensure proper resource alignment, roles and responsibilities, and much more. Ambarella provides appropriate training to its development teams and plans for confirmation reviews with an independent assessment company.

Based on the PRD, Ambarella starts the process of SEooC definition to define top-level safety requirements with ASIL targets, top-level safe states, along with assumptions of use and constraints. The SEooC document also defines functional boundaries, interfaces, and preliminary allocation of functions to elements.

Since Ambarella's SoC is really an element instead of a system per ISO26262, Ambarella’s technical safety requirement (TSR) and system architecture design (i.e. TSC - Technical Safety Concept) are developed concurrently and iteratively. TSRs are allocated to hardware and software directly and use a separate V-model process to guide the development. Safety analysis is performed to identify failure modes and come up with safety measures/mechanisms, which further refine the TSRs.

When the architectural design is complete, Ambarella starts to design the micro-architecture design. And, when that is done, design of the chip and the software stack begin. During the design process, Ambarella applies FMEDA analysis to meet target FIT rates. Architectures develop a golden model of the hardware design to aid verification and software development.

The pre-silicon verification process is hierarchical. Testing is first done at block level, followed by cluster level, then the full-chip level. Once the chip design is functionally verified, the physical design process begins. This process includes closing timing, adding design-for-test blocks (DFT), place-and-route, running design rule checks specific to a process defined by the foundry. The chip design is sent to foundry for tape-out/wafer fabrication. After that the wafers are assembled and tested before going to warehouse.

Post-silicon verification includes three categories of tests: (1) function verification, (2) characterization, and (3) qualification. Post-silicon testing validates the safety design in silicon. When function verification is complete, Ambarella are ready to release the safety manual and safety analysis report, and can offer engineering samples to customers. When (2) and (3) are complete, Ambarella is ready to release for production.

Ambarella applies both ATE and ATB bench tests in production testing to ensure a very low defect rate.

Ambarella reviews suppliers’ manufacturing production plans and control plans to ensure that safety-related special characteristics are controlled throughout of the fabrication-assembly-testing stages. After release for production, Ambarella applies field monitoring process and lifelong configuration & change management process to address the functional safety needs.

8.2 ISO 26262 Lifecycle Tailoring During SEooC Development

The CV2 ASIL has been developed as a hardware SEooC according to ISO 26262-10, clause 9. The table below summarizes the ISO 26262:2018 lifecycle tailoring with the parts, clauses and work products that were included in the scope of the SEooC development project. Requirements and work products that have been tailored out are shown as struck-out (~~Example~~), and they may have to be completed during the system development project and SEooC integration.

Table 5 ISO 26262 Lifecycle Tailoring Applied During SEooC Development

Part 2 - Management of functional safety	
5 Overall safety management	5.5.1 Org.-specific rules and processes for functional safety 5.5.2 Evidence of competence management 5.5.3 Evidence of a quality management system 5.5.4 Identified safety anomaly reports (if applicable)
6 Project dependent safety management	~~6.5.1 Impact analysis at the item level~~ ~~6.5.2 Impact analyses at element level (if applicable)~~ 6.5.3 Safety plan 6.5.4 Safety case 6.5.5 Confirmation measure reports 6.5.6 Release for production report
7 Safety management regarding production, operation, service and decommissioning	7.5.1 Evidence of safety management regarding production, operation, service and decommissioning
Part 3 - Concept phase	
~~5 Item definition~~	~~5.5.1 Item definition~~
~~6 Hazard analysis and risk assessment~~	~~6.5.1 Hazard analysis and risk assessment report~~ ~~6.5.2 Verification report of the HARA~~
~~7 Functional safety concept~~	~~7.5.1 Functional safety concept~~ ~~7.5.2 Verification report of the FSC~~

Part 4 - Product development at the system level	
5 General topics for the product development at system level	(none)
6 Technical safety concept	6.5.1 Technical safety requirements specification 6.5.2 Technical safety concept 6.5.3 System architectural design specification 6.5.4 Hardware-software interface (HSI) specification 6.5.5 Specification of requirements for production, operation, … 6.5.6 Verification report for TSR, TSC, SYS, HSI, … 6.5.7 Safety analyses reports
7 System and item integration and testing	7.5.1 Integration and test strategy 7.5.2 Integration and test report
8 Safety validation	8.5.1 Safety validation specification 8.5.2 Safety validation report
Part 5 - Product development at the hardware level	
5 General topics for the product development at hardware level	(none)
6 Specification of hardware safety requirements	6.5.1 Hardware safety requirements specification (including test and evaluation criteria) 6.5.2 Hardware-software interface specification (refined) 6.5.3 Hardware safety requirements verification report
7 Hardware design	7.5.1 Hardware design specification 7.5.2 Hardware safety analysis report 7.5.3 Hardware design verification report 7.5.4 Specification of requirements for production, operation, …
8 Evaluation of the hardware architectural metrics	8.5.1 Analysis of the effectiveness of the architecture of the item to cope with the random hardware failures 8.5.2 Verification review report of evaluation …

9 Evaluation of safety goal violations due to random hardware failures	9.5.1 Analysis of safety goal violations due to random hardware failures 9.5.2 Specification of dedicated measures for hardware, if needed, including the rationale regarding the effectiveness of the dedicated measures 9.5.3 Verification review report of evaluation …
10 Hardware integration and verification	10.5.1 Hardware integration and verification specification 10.5.2 Hardware integration and verification report
Part 6 - Product development at the software level	
5 General topics for the product development at software level	5.5.1 Documentation of the software development environment
6 Specification of software safety requirements	6.5.1 Software safety requirements specification 6.5.2 Hardware-software interface specification (refined) 6.5.3 Software verification report
7 Software architectural design	7.5.1 Software architectural design specification 7.5.2 Safety analysis report 7.5.3 Dependent failures analysis report 7.5.4 Software verification report (refined)
8 Software unit design and implementation	8.5.1 Software unit design specification 8.5.2 Software unit implementation
9 Software unit verification	9.5.1 Software verification specification 9.5.2 Software verification report (refined)
10 Software integration and verification	10.5.1 Software verification specification (refined) 10.5.2 Embedded software 10.5.3 Software verification report (refined)
11 Testing of the embedded software	11.5.1 Software verification specification (refined) 11.5.2 Software verification report (refined)

Annex C (normative) Software configuration	C.5.1 Configuration data specification C.5.2 Calibration data specification C.5.3 Configuration data C.5.4 Calibration data C.5.5 Software verification specification (refined) C.5.6 Software verification report (refined) C.5.7 Software architectural design specification (refined) C.5.8 Documentation of the software development env. (refined)
Part 7 - Production and operation	
5 Planning for production, operation, service and decommissioning	5.5.1 Safety-related content of the production plan 5.5.2 Safety-related content of the production control plan, including the test plan 5.5.3 Producibility requirements specification 5.5.4 Production process capability report 5.5.5 Safety-related content of the service plan 5.5.6 Safety-related content of the service instructions 5.5.7 Safety-related content of the information made available to the user (safety manual) 5.5.8 Safety-related content of the decommissioning instructions 5.5.9 Operation, service and decommissioning requirements specification 5.5.10 Safety-related content of the rescue services instructions
6 Production	6.5.1 Control measures report 6.5.2 Production process capability report
7 Operation, service and decommissioning	7.5.1 Field observation instructions
Part 8 - Supporting processes	
5 Interfaces within distributed developments	5.5.1 Supplier selection report 5.5.2 Development interface agreement (DIA) 5.5.3 Supplier's safety plan 5.5.4 Functional safety assessment report 5.5.5 Supply agreement

6 Specification and manage-ment of safety requirements	(none)
7 Configuration management	7.5.1 Configuration management plan
8 Change management	8.5.1 Change management plan 8.5.2 Change request 8.5.3 Impact analysis and change request plan 8.5.4 Change report
9 Verification	9.5.1 Verification plan 9.5.2 Verification specification 9.5.3 Verification report
10 Documentation	10.5.1 Documentation management plan 10.5.2 Documentation guideline requirements
11 Confidence in the use of software tools	11.5.1 Software tool criteria evaluation report 11.5.2 Software tool qualification report
12 Qualification of software components	12.5.1 Software component documentation 12.5.2 Software component qualification report 12.5.3 Software component qualification verification report
13 Qualification of hardware components	13.5.1 Hardware element evaluation plan 13.5.2 Hardware element test plan (if applicable) 13.5.3 Hardware element evaluation report for hardware elements
14 Proven in use argument	14.5.1 Description of candidate for proven in use argument 14.5.3 Proven in use analysis reports
15 Interfacing an application that is out of scope of ISO 26262	15.5.1 Base vehicle manufacturer or supplier guideline
16 Integration of safety-related systems not developed according to ISO 26262	16.5.1 Safety rationale

Part 9 - ASIL-oriented and safety-oriented analyses	
5 Requirements decomposition with respect to ASIL tailoring	5.5.1 Update of architectural information 5.5.2 Update of ASIL as attribute of safety requirements and elements
6 Criteria for coexistence of elements	6.5.1 Update of ASIL as attribute of sub-elements of the element
7 Analysis of dependent failures	7.5.1 Dependent failures analysis 7.5.2 Dependent failures analysis verification report
8 Safety analyses	8.5.1 Safety analyses 8.5.2 Safety analyses verification report
Part 10 - Guideline on ISO 26262	
9 Safety element out of context	9.1 SEooC assumptions and requirements specification

8.3 Recommendations to Customer

Customer is expected to perform all the other activities during system integration which are tailored out from the Ambarella ISO 26262 life cycle.

We recommend that customers shall reach Ambarella team to address the safety requirements and the real case implementation that needs software involved.

8.4 Assumptions

Any use of this chip conforms to the guidelines and specifications of the safety manual. Customer is responsible for scheduling safety tasks during normal operation within the FTTI time to ensure safety of all embedded vision co-processors and assumes all risk if Ambarella's low-level microcode is modified.

9 Abbreviations

The following lists the descriptions of the abbreviations used in this document.

Abbreviation	Description
ADAS	Advanced Driver Assistance System
AMBA	Advanced Micro-controller Bus Architecture
AoU	Assumption of Use
AR	Assumed Requirement (to the user / customer)
ASIL	Automotive Safety Integrity Level
BGERR	BiasGen Error
BSD	Blind Spot Detection
BST	Bootstraper
BTA	Bus-Turn-Around
CAN	Controller Area Network
CEHU	Central Error Handling Unit
CMBE	Continuous Multiple-Bit Error
CMS	Camera Monitoring Systems
CON	Constraints
CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
CV	Computer Vision
DAG	Directed Acyclic Graph

DBSE	DRAM BIST and Scrubbing Engine
DCB	Direct Chaining Buffers
DCLS	Delayed lockstep
DCS	Display Command Set
DDR	Dual Data Rate
DES	Design measure
DFA	Dependent Failure Analysis
DFMEA	Design Failure Modes and Effect Analysis
DFT	Design For Test
DMA	Direct Memory Access
DRAM	Dynamic Random-Access Memory
DPF	Dual Point Fault
DSI	Display Serial Interface
DSM	Driver State Monitoring
DTTI	Diagnostic Test Time Interval
ECC	Error Correction Codes
ECRU	Error Collecting and Recording Unit
eMMC	Embedded Multi Media Card
FBGA	Fine Ball Grid Array
FD	Flexible Data rate
FEX	Feature Extraction

FIFO	First In First Out
FIT	Failure in Time (1 FIT = 10^{-9} h^{-1})
FMEA	Failure mode and effects analysis
FMEDA	Failure Mode, Effect and Diagnostic Analysis
FRC	Force Comparison Fail
FTTI	Fault Tolerant Time Interval
GDMA	GRAPHICS DMA
GIC	Generic Interrupt Controller
GPIO	General Purpose I/O
HDR	High Dynamic Range
HSI	Hardware-software interface
I2C	Inter-Integrated Circuit communication protocol
I2S	Integrated Interchip Sound
ICG	isolation clock gating
IDSP	Image Digital Signal Processor
IR	InfraRed
IRQ	Interrupt ReQuest
IST	In-System-Test
JTAG	Joint Test Action Group
LBIST	Logic Built-In Self-Test
LED	Light-Emitting Diode

LFM	Latent Fault Metric
LPDDR	Low-Power Dual Data Rate
MBUF	message buffer
MCU	Motor Control Unit ECU
MIPI	Mobile Industry Processor Interface
MIPI CSI	MIPI Camera Serial Interface
MIPI DSI	MIPI Display Serial Interface
MIPI HS	MIPI High Speed
MIPI LP	MIPI Low Power
MMC	Multimedia Card
NDA	Non-Disclosure Agreement
NFR	Non-Functional Requirement
NIC-400	Network Interface Controller
NIC-400 GPV	NIC-400 Global Programmers View
ORC	Open RISC Core
OTP	One-Time Programming
OVP	Over-Voltage Protection
PLL	Phase-Locked Loop
PMIC	Power Management IC
PO MBIST	Power On MBIST
POC	Power-On Configuration

POR	Power-On Reset
PRM	Programming Reference Manual
PROD	Production
PSMON	Power supply monitor
PWM	Pulse Width Modulation
QM	Quality Management
RF	Residual Fault
RMII	Reduced Media Independant Interface
RPN	Risk Priority Number
RTC	Real-Time Clock
RTL	Register Transfer Language
SD	Secure Digital Memory Card
SECDED	Single bit Error Correction and Double bit Error Detection
SEooC	Safety Element out of Context
SIMD	Single Instruction Multiple Data
SM	Safety Mechanism
SoC	System on a Chip
SPFM	Single Point Fault Metric
SPI	Serial Peripheral Interface
SPI-NOR	Serial NOR Flash
SPR_TOR	special purpose register timeout reason

SRAM	Static Random-Access Memory
SS	Safe State
SSI	Synchronous Serial Interface
SSIS	Synchronous Serial Interface Slave
STL	Software test libraries
SW	Software
SYS	System Design
TCM	Tightly Coupled Memory
TLFM	Top-Level Failure Modes
TLSR	Top-level Safety Requirement
TLSS	Top-Level Safe State
TSC	Technical Safety Concept
TSR	Technical Safety Requirement
UART	Universal Synchronous Asynchronous Receiver Transmitter
UC	Use Case
UVP	Under-Voltage Protection
VDSP	Video Data Signal Processing
VIN	Video input
VMB	VMEM Buffer
VMEM	Dedicated software-managed memory for the vector processor
VOUT	Video output

VP	Vector Processor
WDT	Watchdog Timer
XMEM	Cross bar memory

10 Disclaimer

All Ambarella design specifications, datasheets, drawings, files, and other documents (together and separately, “materials”) are provided on an “as is” basis, and Ambarella makes no warranties, expressed, implied, statutory, or otherwise with respect to the materials, and expressly disclaims all implied warranties of noninfringement, merchantability, and fitness for a particular purpose. The information contained herein is believed to be accurate and reliable. However, Ambarella assumes no responsibility for the consequences of use of such information.

Ambarella reserves the right to correct, modify, enhance, improve, and otherwise change its products and services at any time and to discontinue any product or service without notice. Customers should obtain the latest relevant information before placing orders and should verify that such information is current and complete.

All products are sold subject to Ambarella’s terms and conditions of sale supplied at the time of order acknowledgment. Ambarella warrants performance of its hardware products to the specifications applicable at the time of sale in accordance with its standard warranty. Testing and other quality control techniques are used to the extent Ambarella deems necessary to support this warranty.

Ambarella assumes no liability for applications assistance or customer product design. Customers are responsible for their products and applications using Ambarella components. To minimize the risks associated with customer products and applications, customers should provide adequate design and operating safeguards and ensure their products and applications meet applicable standards and safety requirements.

Ambarella does not warrant or represent that any license, either expressed or implied, is granted under any patent right, copyright, mask work right, or other intellectual property right relating to any combination, machine, or process in which Ambarella products or services are used. Information published or provided by Ambarella regarding third-party products or services does not constitute a license from Ambarella to use such products or services or a warranty or endorsement thereof. Use of such information may require a license from a third party under the patents or other intellectual property of the third party, or a license from Ambarella under the patents or other intellectual property of Ambarella.

Reproduction of information from Ambarella documents is not permissible without prior approval from Ambarella.

Ambarella products are not authorized for use in safety-critical applications (such as life support) where a failure of the product would reasonably be expected to cause severe personal injury or death, unless officers of the parties have executed an agreement specifically governing such use. Customers acknowledge and agree that they are solely responsible for all legal, regulatory and safety-related requirements concerning their products and any use of Ambarella products in such safety-critical applications, notwithstanding any applications-related information or support that may be provided by Ambarella. Further, customers must fully indemnify Ambarella and its representatives against any damages arising out of the use of Ambarella products in such safety-critical applications.

Ambarella products are neither designed nor intended for use in military/aerospace applications or environments. Customers acknowledge and agree that any such use of Ambarella products is solely at the customer's risk, and they are solely responsible for compliance with all legal and regulatory requirements in connection with such use.

11 Reference Documents

This document is based on the *exida* template "Safety Manual Microcontroller-IC, V1-R0"; all rights reserved.

The following documents are referenced in this document.

ID	Document name	Description
DTS_1000002, DTS_1000006, DTS_1000002	CV2FS Datasheet	The datasheet of CV2FS products
CV22FS-DTS-002, CV22FS-DTS-003, CV22FS-DTS-004	CV22FS Datasheet	The datasheet of CV22FS products
DTS_1000106	CV2FSA Datasheet	The datasheet of CV2FSA product
CV2FS-SHA-001	CV2FS System hardware	Application notes of CV2FS system hardware
CV22FS-SHA-001	CV22FS System hardware	Application notes of CV22FS system hardware
CV2FS-S03-001	CV2FS Hardware Programming Reference Manual	This document provides comprehensive information on each field of each register, as well as step-by-step programming instructions.

12 Revision History

Our goal is to provide our customers with the highest-quality documentation possible, and to continuously improve our publications to ensure that your experience with Ambarella's products is a positive one. If you have any questions or comments regarding this document, please contact the Technical Writing team at docs@ambarella.com. Your feedback is welcomed and appreciated.

NOTE: Page / chapter numbers for previous drafts may differ from those in the current version.

Table 6 Revision History

Version	Date	Authors	Comments
0.1	20 AUG 2020	Bob Kunz Asher Shih	Initial draft Added Section 5.3.2 for CR#18985
0.2	22 JUN 2021	Asher Shih	Added unique requirement ID for each Assumed Requirements in Section 5.3
0.3	10 Oct 2021	Asher Shih	Changed ASIL target from B to C Added [AR-31] in Section 5.3.31 Updated Table-2 and Table-3 for CR#21263
0.4	26 Oct 2021	Asher Shih	Refined the tables in Section 5.3 by removing the information not needed for users Refined descriptions in Chapter 5 to make it clear for users
0.5	10 Dec 2021	Bob Kunz Asher Shih	Updated the safety architecture in Sections 4.2 Updated the recommendations and assumptions in Sections 8.3 and 8.4 Added the ownership fields in Sections 5.1 and 5.2 to clarify the responsibility of each safety mechanism Removed on-chip temperature sensor from Section 5.1 since it's not a safety measure for fault detection Updated the information for SM_SW_PSMON_TEST, SM_HW_VIN_CRC, SM_HW_ASTUCK_TEST, and SM_SW_WDT_ORC
0.6	12 Dec 2021	Asher Shih	Reworded the text in Chapter 3 Updated the wording in Sections 5.2.7 and 5.2.28
0.7	2 Mar 2022	Asher Shih Bob Kunz	Updated Figure 2 to explicitly mention that the caches of Cortex-A53 is protected by ECC and parity (CR#21649) Updated the description in Section 4.2 (CR#21649)

0.8	26 May 2025	Asher Shih	Added CV2FSA as the product name of the SEooC. Updated λ_{RF} in Table 2. Added CV2FSA FMEDA results in Section 7.1.3. Added a note for Table 4. Updated the wording in Section 8.3. Added the reference documents in Chapter 11.